BEAST-MEV: Batched Threshold Encryption with Silent Setup for MEV Prevention

Jan Bormet
Technische Universität Darmstadt
Darmstadt, Germany
jan.bormet@tu-darmstadt.de

Arka Rai Choudhuri
zkBricks
Sacramento, USA
arkarai.choudhuri@gmail.com

Sebastian Faust
Technische Universität Darmstadt
Darmstadt, Germany
sebastian.faust@tu-darmstadt.de

Sanjam Garg
University of California, Berkeley
Berkeley, USA
sanjamg@berkeley.edu

Hussien Othman
Technische Universität Darmstadt
Darmstadt, Germany
hussien.othman@gmail.com

Guru-Vamsi Policharla
Commonware, Inc.
Palo Alto, USA
guruvamsi.policharla@gmail.com

Ziyan Qu
Technische Universität Darmstadt
Darmstadt, Germany
ziyan.qu@tu-darmstadt.de

Mingyuan Wang
University of Delaware
Newark, USA
mywang@udel.edu

Abstract

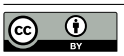
Threshold encrypted mempools protect the privacy of transactions up until the point their inclusion on chain is confirmed. They are a promising approach to protection against front-running attacks on decentralized blockchains.

Recent works have introduced two key properties that an encryption scheme must satisfy to operate efficiently on large-scale decentralized blockchains such as Ethereum: Silent Setup, as introduced by Garg et al. (CRYPTO'24), demands that a threshold encryption scheme does not require any interaction during the setup phase and only relies on the existence of Public Key Infrastructure. Batched Decryption, as introduced by Choudhuri et al. (USENIX'24), demands that an entire block containing B encrypted transactions can be decrypted using communication that is independent of (or sublinear in) B, without compromising the privacy of transactions that have not yet been confirmed.

While existing constructions achieve either Silent Setup or Batched Decryption independently, a truly decentralized and *scalable* encrypted mempool requires both properties to be satisfied simultaneously. In this work, we present the first "Batched Threshold Encryption scheme with Silent Setup" built using bilinear pairings. We provide formal definitions for the primitive and prove security in the Generic Group Model. We provide several optimizations and implement our scheme to evaluate its performance. Our experiments demonstrate its efficiency for deployment in blockchain systems.

CCS Concepts

• **Theory of computation** → **Cryptographic protocols**; • **Security and privacy** → *Public key encryption*; *Distributed systems security*.



This work is licensed under a Creative Commons Attribution 4.0 International License.
CCS '26, The Hague, Netherlands
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2871-6/2026/11
https://doi.org/10.1145/3830454.3846729

Keywords

batched threshold encryption, mempool privacy, silent setup

ACM Reference Format:
Jan Bormet, Arka Rai Choudhuri, Sebastian Faust, Sanjam Garg, Hussien Othman, Guru-Vamsi Policharla, Ziyan Qu, and Mingyuan Wang. 2026. BEAST-MEV: Batched Threshold Encryption with Silent Setup for MEV Prevention. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26), November 15–19, 2026, The Hague, Netherlands.* ACM, New York, NY, USA, 28 pages. https://doi.org/10.1145/3830454.3846729

1 Introduction

One of the most significant applications of public blockchains is decentralized finance (DeFi), in which users access innovative financial products to generate trading returns. Over the past few years, the DeFi ecosystem has experienced explosive growth in both locked capital and product innovation. For instance, Ethereum alone currently holds over $60 billion in DeFi deposits, and several competing networks each boast multi-billion-dollar ecosystems of their own. While DeFi has several advantages in terms of decentralization, and being permissionless, it is prone to market manipulation attacks from front-running, sandwiching, and more [4, 32, 53, 68, 77, 79, 82, 83].

To illustrate these attacks, let us recall how blockchains process transactions that are submitted by users to the blockchain miners. The miners collect these unprocessed transactions in a so-called *mempool*. Once a miner is elected as a block proposer, it selects some non-finalized transactions from the mempool and assembles them into a block. Crucially, since miners can decide upon the order of the transactions within a block, a malicious miner can, e.g., place a profitable transaction before an honest users' trade (known as frontrunning). Such re-ordering can reap huge financial profits, which is often referred to as *miner/maximal extractable value (MEV)*. The presence of MEV opportunities can lead to centralization issues, e.g., currently the majority of Ethereum blocks are produced by

just two block builders.[1] Since this can de-stabilize the underlying consensus mechanism [25], developing countermeasures against MEV attacks has gained huge interest both by academia and industry [10, 30, 34, 56].

Among the proposed mitigations, a particularly promising approach is to keep transaction content private upon entry into the mempool. By concealing transaction details (e.g., the type of swap a user wants to execute), this effectively prevents miners from selecting and ordering transactions based on their content, thereby preventing MEV. However, for the system to remain usable, transactions must be available in the clear *after* they are finalized. The naïve implementation, in which clients commit their transactions and later return to reveal them if selected, is inadequate since it places a burden on clients to remain online and invites the risk that clients refuse to reveal transactions they no longer deem favorable.

Instead, an appealing cryptographic solution is to use threshold encryption to construct *private mempools* [3, 8, 18, 22, 23, 30, 81]. In this setting, during a setup phase a set of n servers (often called a *committee*) jointly generates a public encryption key that is published on a blockchain and a decryption key that is shared among the servers. Users can encrypt their transactions under the public key and send them to the miners, where they queue in the mempool for processing. Later, once the miners have selected B encrypted transactions from the mempool and built a block, the members of the decryption committee broadcast partial decryptions, enabling anyone in the system to decrypt and recover the corresponding transaction data of the B selected transactions. The security guarantee is that as long as fewer than $t < n$ servers are malicious, re-ordering based on transaction content is prevented.

For use in practice, two further properties are important: (i) since the partial decryptions are stored (at least temporarily) on the blockchain, the size of each server's partial decryption should be a constant, independent of the number of selected transactions B; and (ii) ciphertexts that are *not* selected should *remain private*, a notion formalized as *pending transaction privacy* in [22]. Schemes satisfying these criteria are referred to as *batched* threshold encryption (BTE) schemes. While it is relatively easy to construct a scheme satisfying either of these properties in isolation, recent works have shown how to construct practical BTE schemes satisfying both properties (with various additional features) [3, 18, 22, 23].

One crucial bottleneck of current batched threshold encryption schemes (and in fact most threshold cryptosystems in general) is the requirement of an interactive setup. Indeed, all existing constructions of BTE schemes [3, 18, 22, 23] assume parties holding Shamir secret shares of the decryption key. In the blockchain setting, where no trusted party exists, this would require all committee members to run an interactive setup protocol, known as the distributed key generation (DKG) protocol. Running DKG is highly undesirable in practice due to its costly concrete efficiency. Moreover, in a permissionless setting, the committee is typically refreshed on a regular basis, which means that such a costly DKG protocol needs to be repeated as well every time a new committee is selected. For our application of encrypted mempools, the costs of DKG are particularly problematic as ideally we would like all miners to participate in the decryption process. Since major blockchains such as Ethereum are run by hundreds of thousands of miners (or validators in case of Ethereum), the costs of running DKG become effectively prohibitive. What we really need is a *non-interactive* and *one-time* setup that enables the formation of all future committees, when *no complex on-boarding or off-boarding* is required when miners join or leave the network. This notion, in the context of (standard) threshold encryption, was formalized and achieved as *silent threshold encryption* (STE) scheme by [45]. However, none of the previously discussed BTE schemes support a silent setup; and the STE scheme in [45] incurs the same communication overhead as standard threshold encryption for decrypting a batch of messages, i.e., the partial decryption size grows linearly with B for a batch of B messages.

Thus, although significant progress has been made in using threshold encryption for mempool privacy, incorporating various desirable features, none of the existing schemes are suitable for the large open networks characteristic of blockchain environments.

1.1 Our Contribution

In this work, we address the above challenge and build the first practical construction for batched threshold encryption, that does not rely on any (costly) interactive setup protocol.

- We construct an efficient CCA-secure *silent* batched threshold encryption scheme. Our construction requires no interaction among committee members during the setup phase, and no setup is needed per epoch. In contrast to most prior work on BTE [3, 22, 23], we achieve the additional benefit that ciphertexts are not tied to any specific block.
- As part of our construction, and of independent interest, we introduce the *first* efficient additive homomorphic silent threshold encryption scheme (HSTE).
- We propose several *novel* optimizations that enable efficient encryption and decryption of multiple messages simultaneously in our HSTE. Concretely, by applying these optimizations, the resulting ciphertext is reduced by a factor of 213× and partial decryptions are up to 3000× smaller. The techniques underlying these optimizations may also be of independent interest.
- We provide formal security proofs for our construction, covering practical attack scenarios relevant to encrypted mempools.
- We implement the fully secure version of our batched threshold encryption scheme, including all NIZK proofs required for CCA and rogue ciphertext security, in Rust and evaluate its performance in Section 8. Encryption takes ≈ 106 ms (≈ 9.1 ms for the encryption itself and ≈ 97 ms for generating the attached proofs) irrespective of other system parameters such as committee size n and batch size B, and verifying the proofs attached to a ciphertext takes ≈ 95 ms. The ciphertexts are ≈ 1.4 KiB in size—only 88% larger than ciphertexts from [45], which does not support batched decryption—plus ≈ 5.7 KiB for the attached proofs. Computing a constant-sized partial decryption (48 bytes) together with its 64-byte proof of correct decryption takes ≈ 33 ms at a batch size of 512, and reconstruction of all messages can be completed in 3.28 s in single-threaded mode. We emphasize that the reconstruction step can be parallelized to achieve more competitive decryption times.

[1] Notice that in current Ethereum, this is due to the proposer-builder separation where special parties, so-called builders, are responsible for pre-building blocks.

Subsequent work. Subsequent to the first version of this manuscript (as publicly available), Gong, Waters, Wee, and Wu [48] independently constructed a batched threshold encryption scheme with silent setup from pairings using different techniques. Their approach builds on the Boneh-Boyen IBE scheme [13] to obtain a batched IBE, and then incorporates the silent threshold IBE construction from [80] to eliminate the need for DKG.

The main advantage of their scheme over ours is a more efficient decryption procedure. Moreover, their ciphertexts are slightly shorter and do not require non-interactive zero-knowledge proofs to achieve CCA security. On the other hand, a significant drawback is that each ciphertext is bound to a specific block or epoch, which must be fixed at encryption time. Additionally, the size of the common random string in their construction is substantially larger than in ours: our construction requires approximately 414 KiB, whereas theirs requires 18,528 KiB.

We compare the parameter sizes in detail in Figure 9.

1.2 Technical Overview

To highlight shortcomings of prior batched threshold encryption (BTE) schemes [18, 22, 23], and to illustrate the technical challenges in making BTE *silent*, we first recall the core syntax of BTE.[2]

$\mathsf{KGen}(n, t)$ The key generation algorithm is specified with two parameters, n - the maximum size of the committee; and t - the desired threshold for the committee. The algorithm outputs a *single* public key pk for the committee along with n secret keys $\mathsf{sk}_1, \ldots, \mathsf{sk}_n$. While the public key is available to all users in the system, the secret key sk_i is handed out to a *pre-determined committee member*.

$\mathsf{Enc}(\mathsf{pk}, m)$ The encryption algorithm run by any user produces a ciphertext ct encrypting a message to m under the committee public key pk.

$\mathsf{BatchDec}(\mathsf{sk}_\ell, \mathsf{ct}_1, \cdots, \mathsf{ct}_B)$ The batched decryption algorithm run by the committee member in possession of the secret key sk_ℓ produces a *single* partial decryption pd_ℓ for the B ciphertexts.

$\mathsf{Comb}(\mathsf{pk}, \mathsf{ct}_1, \cdots, \mathsf{ct}_B, \{\mathsf{pd}_\ell\}_{\ell \in S})$ Given partial decryptions from a set S (of at least t) committee members, one can decrypt all B ciphertexts to obtain messages $m_1, \cdots, m_B$.

The non-triviality of a BTE scheme comes from the simultaneous requirements that partial decryptions can only be used for the ciphertexts they were computed for, while their size remains independent of B.

As noted in Section 1, running a DKG not only incurs the cost of an interactive protocol, but also fixes the committee and threshold, making them *static*. This restriction is undesirable in settings where committees (and thresholds) should evolve dynamically, without rerunning the DKG each time.

Instead, we want KGen to be run locally by any user, producing a key pair (pk, sk), with pk published. For any chosen committee C, the public keys $\{\mathsf{pk}_i\}_{i \in C}$ can then be aggregated by anyone into two keys: an encryption key ek (for encryption) and an aggregation key ak (for combining partial decryptions).

Encryption is now done with respect to ek instead of pk, and the combination uses ak instead of pk. The batch decryption procedure remains unchanged. These requirements for a (standard) threshold encryption scheme were introduced in [45], where this setup procedure, without interaction, is referred to as *silent*. However, when applied in the context of BTE, the known construction of silent threshold encryption (STE) yields partial decryptions of size $O(B)$. In the remainder of this section, we describe how to combine the techniques of BTE and STE, and the challenges that arise therein.

Batch Threshold Encryption of [18]. Our starting point is the BTE scheme presented in [18]. As we shall shortly see, this choice is informed by the structure of the BTE construction in [18], where the only use of DKG is for a vanilla threshold encryption scheme.

The batched threshold encryption scheme from [18] leverages key-homomorphic puncturable PRFs. Let PKE denote an additively homomorphic threshold encryption scheme, and let PRF be a key-homomorphic puncturable PRF. The scheme is as follows:

- Setup: The committee runs PKE.Setup in order to generate a distributed public key pk as well as private secret shares $\mathsf{sk}_1, \ldots, \mathsf{sk}_n$. Each committee member receives a secret share.
- Enc: Given a message m and a public key pk, the i-th client who is assigned the index $i \in [B]$, samples a PRF key k and punctures it at index i as $k^* \leftarrow \mathsf{Punct}(k, i)$. It computes the ciphertext $\mathsf{ct}_i = (k^*, \beta = m + \mathsf{PRF}(k, i), v = \mathsf{PKE.Enc}(\mathsf{pk}, k))$.
- BatchDec: Given a batch of ciphertexts $\{\mathsf{ct}_1, \ldots, \mathsf{ct}_{|B|}\}$, each party P_i computes $\mathsf{pd}_i \leftarrow \mathsf{PKE.Dec}(\mathsf{sk}_i, \sum_j (\mathsf{ct}_j).v)$, i.e., the message being decrypted is the summation of the v-part in each ciphertext.
- Comb: Given t partial decryptions recover $k = \sum_i k_i$ from $\mathsf{pd}_1, \ldots, \mathsf{pd}_t$, where k_i is the PRF key in the i-th ciphertext ct_i. Using k, we can compute $\mathsf{PRF}(k_i, i)$ for all $i \in [B]$ as follows:

$$\mathsf{PRF}(k_i, i) = \mathsf{PRF}(k, i) - \sum\nolimits_{j \neq i} \mathsf{PEval}(\mathsf{ct}_j.k^*, i).$$

  Given $\mathsf{PRF}(k_i, i)$, we can recover $m_i \leftarrow \mathsf{ct}_i.\beta - \mathsf{PRF}(k_i, i)$.

Note that in BatchDec, the parties aggregate all ciphertexts in the batch and reveal partial decryptions only for the aggregated ciphertext. To enable this step, the underlying scheme PKE must be additively homomorphic. The construction in [18] leverages threshold ElGamal in the exponent as the underlying PKE. As described above, one needs the key k to complete the Comb step. However, the ElGamal scheme does not support plaintext recovery when the message is large, as is the case with the key k. To get around this issue, in [18], the authors demonstrate a specific puncturable PRF scheme, where it suffices to obtain $[k]_1$,[3] the key k encoded in a group $\mathbb{G}_1$, to evaluate the PRF. At a high level, the PRF evaluation process requires one to pair $[k]_1$ (in the group $\mathbb{G}_1$ of an asymmetric pairing group) with an element in $\mathbb{G}_2$, to receive an output in the target group $\mathbb{G}_T$.

To follow this template with any threshold encryption scheme (not just STE), the scheme must: (i) support encrypting source group elements (say in $\mathbb{G}_1$), i.e., elements of the form $[k]_1$; and (ii) support additive homomorphism over the *field elements* underlying the group, i.e., encryption of $[k]_1$ and $[k']_1$ can be combined to get an encryption of $[k + k']_1$.

[2] We simplify the syntax here to convey the main ideas, and refer the reader to the technical section for details.

[3] We denote by $[x]_1$, $[y]_2$, and $[z]_T$, the field elements x, y, and z encoded in groups $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_T$, respectively.

Unfortunately, the STE scheme in [45] *does not* support homomorphism. We first describe how to modify their scheme to achieve a limited form of homomorphism, which, as we shall see, does not suffice to follow the template above yet.

Homomorphic STE.. We start by recalling the main ideas in the STE scheme [45]. Each party samples a BLS key pair, $(\mathsf{sk}, \mathsf{pk} = [\mathsf{sk}]_1)$ and publishes pk. At a high level, to encrypt a message to a committee C, the user first aggregates the public keys of the committee members to compute a committee encryption key ek, and then computes a witness encryption of the message m it wants to encrypt, to the encryption key ek and a randomly sampled tag $[\gamma]_2$ such that m can only be recovered if one obtains at least t signatures from C on the tag $[\gamma]_2$. In more detail,

Encryption. To encrypt a message in the target group, $m \in \mathbb{G}_T$, the user samples a random tag $[\gamma]_2$, encryption randomness $\mathbf{s}$ and computes the ciphertext as

$$\mathsf{ct} = ([\gamma]_2, \mathsf{ct}_1 = \mathbf{s}^\intercal \cdot \mathbf{A}, \mathsf{ct}_2 = \mathbf{s}^\intercal \cdot \mathbf{b} + m)\,.$$

The matrix $\mathbf{A}$ and vector $\mathbf{b}$ are specified below.

$$\mathbf{A} = \begin{bmatrix} \mathsf{ek} & [1]_2 & [Z(\tau)]_2 & [\tau]_2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & [\tau^2]_2 & [1]_2 & 0 & 0 & 0 \\ 0 & [\gamma]_2 & 0 & 0 & 0 & [1]_1 & 0 & 0 \\ [\tau^t]_1 & 0 & 0 & 0 & 0 & 0 & [1]_2 & 0 \\ [1]_1 & 0 & 0 & 0 & 0 & 0 & 0 & [\tau]_1 \end{bmatrix}$$

$$\mathbf{b} = (0, 0, 0, 0, [1]_T)^\intercal$$

While the exact structure of $\mathbf{A}$ is not important for this discussion,[4] it suffices to note that $\mathbf{A}$ encodes the aggregated encryption ek of the committee, the threshold t, by the way of $[\tau^t]_1$, and the tag $[\gamma]_2$.

Partial Decryption. The partial decryption pd_i is essentially a (BLS) signature σ_i on $[\gamma]_2$, computed as $[\sigma_i]_2 = \mathsf{sk} \cdot [\gamma]_2$.

Decryption. Given signatures from t committee members S, $\{\sigma_i\}_{i \in S}$ and their corresponding public keys $\{\mathsf{pk}_i\}_{i \in S}$, one can compute both the aggregated BLS signature $[\sigma^*]_2$ and, as described in [45], the witness $\mathbf{w}$ such that $\mathbf{A} \circ \mathbf{w} = \mathbf{b}$, where $\circ$ is the pairing operation. Given such a witness, the decryption process to recover m is $m \leftarrow \mathsf{ct}_2 - (\mathsf{ct}_1 \circ \mathbf{w})$.

Note that for the *same* committee C, with the same threshold t, the only change in $\mathbf{A}$ for each ciphertext is the tag $[\gamma]_2$. Instead of having a user choose $[\gamma]_2$ at the time of encryption, let us fix a *single* $[\gamma]_2$ to be used by all ciphertexts.[5] Thus, the $\mathbf{A}$ would be identical for each encryption. This gives us the necessary homomorphism. Specifically, consider any two ciphertexts $\mathsf{ct} = (\mathsf{ct}_1, \mathsf{ct}_2)$ and $\mathsf{ct}' = (\mathsf{ct}'_1, \mathsf{ct}'_2)$ of messages m and m', then by adding the ciphertexts,

$$(\mathsf{ct}_1 + \mathsf{ct}'_1, \mathsf{ct}_2 + \mathsf{ct}'_2) = ((\mathbf{s} + \mathbf{s}')^\intercal \cdot \mathbf{A}, (\mathbf{s} + \mathbf{s}')^\intercal \cdot \mathbf{b} + m + m'),$$

we obtain a ciphertext for the message $m + m'$.

An observant reader may have already noticed that, while fixing $[\gamma]_2$ to be a parameter used across all ciphertexts enables homomorphism, it also makes the scheme insecure as anyone in possession of the aggregated signature $[\sigma^*]_2$ (or the witness $\mathbf{w}$) can now decrypt *any* message.

To fix this issue, we need to look more closely at how $[\sigma^*]_2$ is used in the decryption process. In particular, the witness $\mathbf{w}$ is of the following structure,

$$\mathbf{w} = (\star, \star, \star, \star, \star, [\sigma^*]_2, \star, \star)^\intercal,$$

where we only highlight the relevant part of $\mathbf{w}$ that depends on $[\sigma^*]_2$. Now, recalling the decryption procedure, and focusing only on the relevant parts, we have

$$m \leftarrow \mathsf{ct}_2 - (\mathsf{ct}_1 \circ \mathbf{w}) = \mathsf{ct}_2 - \star - (\mathsf{ct}_{1,6} \circ [\sigma^*]_2) = \mathsf{ct}_2 - \star - ([\mathbf{s}_3]_1 \circ [\sigma^*]_2),$$

where $\mathsf{ct}_{1,6}$ is the sixth component of ct_1, and $[\mathbf{s}_3]_1$ is computed using the encryption randomness $\mathbf{s} = (\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3, \mathbf{s}_4, \mathbf{s}_5)$.

Thus, instead of the partial decryption $\mathsf{pd}_i = [\sigma_i]_2$, the committee members compute $\mathsf{pd}_i = [\mathbf{s}_3]_1 \circ [\sigma_i]_2$, by pairing the signature $[\sigma_i]_2$ with $\mathsf{ct}_{1,6}$. In a manner analogous to the BLS signature aggregation, one can aggregate the partial decryptions to compute $[\mathbf{s}_3]_1 \circ [\sigma^*]_2$. This serves two simultaneous purposes: (i) it binds the partial decryption to the ciphertext, as $[\mathbf{s}_3]_1$ is sampled for each ciphertext; and (ii) it continues to enable decryption. In the technical sections, we formally show that this modification is a secure silent threshold encryption scheme that supports homomorphism.

Homomorphism Mismatch. Unfortunately, the homomorphism that the above scheme supports is only for encryption of messages in the target group $\mathbb{G}_T$, whereas the previously outlined template requires an encryption scheme that supports homomorphism for encryption of messages in the source group.

On the one hand, we do not know how to modify the STE scheme further to support homomorphism in the source group. On the other hand, it seems inherent that we require homomorphism for the source group. This is because the message encrypted serves as the secret key of the puncturable PRFs, whose evaluation involves pairing the secret key with other group elements; therefore, it is inevitable that the secret key comes from a source group.

To solve this problem, we reconsider the [18] template. Instead of using the source group encoding of the key $[k]_1$, we use the above-described homomorphic STE scheme to encrypt the key in the target group $[k]_T$. Since one cannot use $[k]_T$ for the PRF evaluation, we would have to recover the plaintext k from $[k]_T$. Now, since k is large, it is not clear how the decryption works. Here, we resort to a classical trick. Namely, we will encrypt k in a bit-by-bit manner such that plaintext recovery of each bit is feasible, and then the key k can be reconstructed. This remains feasible as long as the number of homomorphic operations is bounded.[6]

Efficiency Optimizations. Unfortunately, the solution as presented above, while feasible, is practically too inefficient. The key reason is that the binary decomposition incurs an overhead of the PRF's key length λ_k (e.g., $\lambda_k = 256$ for $\lambda = 128$ bits of security), i.e., a single ciphertext would consist of λ_k STE ciphertexts encrypting the bits of k. To make our construction practical, we introduce several novel optimizations that reduce the ciphertext size.

- The first simple idea is that, instead of binary decomposition, we could use a larger base (say, $x = (x_1, \ldots, x_\ell)$ for some

[4] We refer the readers to the technical sections for a formal description of $\mathbf{A}$.

[5] Note that, we want $[\gamma]_2$ to be random and the corresponding field element γ to be unknown for the semantic security of the encryption. Hence, such a fixing can simply be given by a random oracle, for instance.

[6] One subtlety here is that: since each key k_i is binary decomposed and k is the sum of k_i's in a bit-by-bit sum manner, when we reconstruct k, we obtain additional leakage on k_i's beyond just their sum. Fortunately, [18] does allow such leakages, and this is not really an issue.

smaller ℓ), striking a balance between decryption hardness and ciphertext size overhead.

- Second, we observe that since the encryptor employs STE to encrypt a *batch* of plaintexts $(x_1, \ldots, x_\ell)$ (due to decomposition). Instead of naïvely encrypting each plaintext individually and incurring an overhead of ℓ, we ask the following:[7]

  Can we leverage *batch encryption* to obtain *(amortized) rate-1* Batch STE?

  We affirmatively answer this question. Since this may be of independent interest, we elaborate on this in detail later.
- Finally, we introduce a *general* technique to improve efficiency whenever one needs to send a target group element (as part of the ciphertext). In particular, if one wants to send a target group element h_T, one may *multiplicatively share* it as source group elements h_1 and h_2 such that $e(h_1, h_2) = h_T$. This is feasible as long as one knows the source group element (from either source group) that has the same discrete log as h_T and saves communication as target group elements are much larger than $\mathbb{G}_1$ and $\mathbb{G}_2$ elements combined.

Rate-1 Batch Encryption for STE.. Recall that STE encryption is essentially a witness encryption for the relation $\mathbf{A} \circ \mathbf{w} = \mathbf{b}$, where the encryptor picks randomness s and computes $(s^\intercal \cdot \mathbf{A}, s^\intercal \cdot \mathbf{b} + m)$. Below, we will refer to $s^\intercal \cdot \mathbf{A}$ as the *header* of the ciphertexts and $s^\intercal \cdot \mathbf{b} + m$ as the *payload*.

Now, if we need to encrypt with respect to this relation ℓ times using fresh randomness, the resulting relation essentially becomes

$$\underbrace{\begin{pmatrix} \mathbf{A} & & & \\ & \mathbf{A} & & \\ & & \ddots & \\ & & & \mathbf{A} \end{pmatrix}}_{\ell \text{ repetitions}} \circ \begin{pmatrix} \mathbf{w} & & & \\ & \mathbf{w} & & \\ & & \ddots & \\ & & & \mathbf{w} \end{pmatrix} = \begin{pmatrix} \mathbf{b} & & & \\ & \mathbf{b} & & \\ & & \ddots & \\ & & & \mathbf{b} \end{pmatrix}.$$

This naïve approach incurs a multiplicative overhead of ℓ in the ciphertext. Instead, we view a batch of ℓ encryptions as the single relation

$$\mathbf{A} \cdot (\mathbf{w}, \ldots, \mathbf{w}) = (\mathbf{b}, \ldots, \mathbf{b}).$$

The hope is that we only need to encrypt the header once (but the payload ℓ times), yielding ciphertexts of size $O(1) + \ell$ group elements (thus, achieving amortized rate-1). However, this is clearly insecure: since the same one-time pad is used across all instances, an adversary can easily compute, for instance, the differences between messages encrypted in different positions.

Our key idea is to rerandomize the above relations as follows:

$$\mathbf{A} \cdot (r_1 \cdot \mathbf{w}, \ldots, r_\ell \cdot \mathbf{w}) = (r_1 \cdot \mathbf{b}, \ldots, r_\ell \cdot \mathbf{b}),$$

where the randomness r_i is freshly sampled, and more importantly, revealed only through its group encoding.

This eliminates the malleability attack described above, while also enabling the reuse of the STE randomness $s^\intercal$ across ℓ ciphertexts. Since the header depends only on $s^\intercal$ (and not on the message m), it can be shared across all ℓ ciphertexts. Concretely, in STE encryption the header consists of 8 group elements, while the payload consists of a single group element. Thus, instead of 9ℓ group elements, the ciphertext size is reduced to $8 + \ell$, making the scheme considerably more efficient in practice.

The remaining challenge is to ensure that the witness $r_i \cdot \mathbf{w}$ is efficiently computable given only the group encoding of r_i. To address this, we carefully modify the setup of the STE construction in a way that preserves security. Thus, we obtain a batch STE encryption scheme with amortized rate-1. We leave it at this point and refer the reader to the technical sections for details.

[7] To illustrate this point, think of the classical ElGamal *batch* encryption. Namely, to batch encrypt $x_1, x_2, \ldots, x_\ell$ under public keys $\mathsf{pk}_1, \ldots, \mathsf{pk}_\ell$, one could encrypt it as $g^r, \mathsf{pk}_1^r \cdot g^{x_1}, \ldots, \mathsf{pk}_\ell^r \cdot g^{x_\ell}$, which is "rate-1". That is, ℓ encryptions require sending only $\ell + 1$ group elements.

1.3 Related Work

Beyond cryptographic solutions for mempool privacy, several alternative approaches address MEV. In Ethereum, Flashbots and MEV-boost are used to regulate MEV extraction through auctions [25, 34]. While reducing its negative effects, they do not eliminate the problem and introduce centralization risks. Solutions based on Trusted-Execution Environment (TEE) are also explored to solve MEV [10, 74]. The secret key of a public key encryption scheme is stored in secure enclaves, and the TEE will decrypt and publish the blocks of transactions encrypted to the corresponding public key. They inherit hardware trust assumptions and remain vulnerable to side-channel attacks [70, 78], making them risky solutions. Another line of work approaches the MEV problem by achieving fair ordering [55, 56, 59]. They enforce consensus over all validators on the transaction ordering, thus limiting any reordering by a single block producer. However, only weakened order fairness has been achieved so far, which does not eliminate MEV entirely. MEV-resistant DEXs such as [51, 63] employ game-theoretic mechanisms to prevent MEV. These works are mostly complex and specific to certain applications, and cannot be used as a general solution. Motivated by these limitations, we build a mempool encryption scheme with the desired properties that practically mitigates MEV.

We present a more comprehensive overview of related work in Appendix A.

2 Preliminaries

Notation. We denote the security parameter with $\lambda \in \mathbb{N}$. We write $\mathrm{negl}(\lambda)$ for functions that are negligible in λ. For integers a, b with $a \le b$, we denote the sets $[a, b] = \{a, a+1, \ldots, b-1, b\}$ and $[b] = \{1, 2, \ldots, b\}$. We write $y \leftarrow A(x; r)$ to denote the execution of algorithm A on input x with randomness r, which outputs y. If A is executed with randomness that is sampled uniformly at random from the appropriate space, we abbreviate as $y \leftarrow\!\$\ A(x)$. We use $\approx_c$ to denote computational indistinguishability of random variables. For a probabilistic algorithm A, we write $y \in A(x)$ to denote that $\Pr[A(x) = y] > 0$.

Polynomials in $\mathbb{F}_p$. We define $\mathbb{H}_n = \{\omega, \omega^2, \ldots, \omega^n\}$ to be the multiplicative subgroup generated by the n-th primitive root of unity $\omega \in \mathbb{F}_p$ (i.e. $\omega^n = 1$) with $|\mathbb{H}_n| = n$. Let $Z(x) = \prod_{i \in n}(x - \omega^i) = x^n - 1$ be the vanishing polynomial on $\mathbb{H}_n$. Let $L_i(x)$ be the Lagrange coefficient of interpolation at position x for index i with respect to $\mathbb{H}_n$ such that $L_i(\omega^j) = 0$ for all $j \neq i$ and $L_i(\omega^i) = 1$.

Univariate Sumcheck. We make use of the univariate sum check lemma [9, 69] generalized to inner products [21].

Lemma 1 (Univariate Sumcheck). *Let* $A(x) = \sum_{i=1}^{|\mathbb{H}|} a_i \cdot L_i(x)$, *and* $B(x) = \sum_{i=1}^{|\mathbb{H}|} b_i \cdot L_i(x)$. *It holds that*

$$A(x)B(x) = \frac{\sum_i a_i b_i}{|\mathbb{H}|} + Q_x(x)x + Q_Z(x)Z(x),$$

where both Q_x *and* Q_Z *are polynomials of degree* $\leq |\mathbb{H}| - 2$ *defined as*

$$Q_x(x) = \sum_i a_i b_i \frac{L_i(x) - L_i(0)}{x}, \text{ and}$$
$$Q_Z(x) = \sum_i a_i b_i \frac{L_i^2(x) - L_i(x)}{Z(x)} + \sum_{i \neq j} a_i b_j \frac{L_i(x)L_j(x)}{Z(x)}.$$

Pairings. A pairing ensemble $\mathcal{G} = (\mathbb{G}_1, [1]_1, \mathbb{G}_2, [1]_2, \mathbb{G}_T, p, \circ)$ is an ensemble of cyclic groups $\mathbb{G}_1$ with generator $[1]_1$, $\mathbb{G}_2$ with generator $[1]_2$, and $\mathbb{G}_T$ of prime order p with a pairing operation $\circ\colon \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ that satisfies *bilinearity* and *non-degeneracy*. We usually write $\circ$ infix.

For some of our proofs, we will use the Generic Group Model (GGM) [62, 72] and its *master theorem* [14], [19, Theorem 1]. We recall relevant definitions and theorems in Appendix B.1.

Cryptographic Building Blocks. We rely on a *commitment scheme* and *non-interactive zero-knowledge proofs* (NIZKs) as building blocks.

At a high level, a *commitment scheme* allows to commit to a message m, generating a commitment com and an opening r such that: (a) the commitment com does not leak information about the message m (hiding); and (b) it is infeasible to find an opening r' such that com opens to a *different* message $m' \neq m$.

In non-interactive zero-knowledge proofs (of knowledge), a prover convinces a verifier in a non-interactive manner that they know a witness ω to a statement χ such that the statement-witness pair (χ, ω) is in some relation R. Crucially, the proof π cannot leak any additional information about ω (zero-knowledge), and a malicious prover should not be able to convince the verifier without knowledge of ω (knowledge-soundness / simulation-extractability).

Key-Homomorphic PPRFs Finally, we use *key-homomorphic puncturable pseudorandom functions* (PRF), which extends a standard PRF with a *key-homomorphism* (i.e., $\mathsf{PRF}(k_1, i) + \mathsf{PRF}(k_2, i) = \mathsf{PRF}(k_1 + k_2, i)$), and *puncturability*. Puncturability allows the key k to be "punctured" at a point i, producing a key k^*, which allows the PRF to be evaluated on any index except i. In more detail, this implies that, given k^*, $\mathsf{PRF.Eval}(k, i)$ looks random to a computationally bounded adversary.

Definition 2 (Key-Homom. Puncturable PRFs). A key-homomorphic puncturable PRF family with bounded domain $[B]$ is a tuple of PPT algorithms $\mathsf{PRF} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Punct}, \mathsf{Eval}, \mathsf{PEval})$ such that

- $\mathsf{pp} \leftarrow\!\$\ \mathsf{Setup}(1^\lambda, B)$: Given security parameter λ as well as domain bound B, the Setup algorithm returns public parameters pp that are implicit input to all subsequent algorithms.
- $k \leftarrow\!\$\ \mathsf{KGen}(1^\lambda)$: Given the security parameter λ, the key generation algorithm returns a key k.
- $k^* \leftarrow \mathsf{Punct}(k, i^*)$: Given a key k, and an index $i^* \in [B]$, the puncturing algorithm outputs a punctured key k^*.
- $y \leftarrow \mathsf{Eval}(k, i)$: Given key k and index $i \in [B]$, the Eval algorithm returns the evaluation y.
- $y \leftarrow \mathsf{PEval}(k^*, i^*, i)$: Given a punctured key k^* for index $i^* \in [B]$ and an index $i \neq i^*$ with $i \in [B]$, the punctured evaluation algorithm returns the evaluation y.

Sometimes we write $\mathsf{PRF}(k, i)$ as a shorthand for $\mathsf{PRF.Eval}(k, i)$. We require the PRF to be correct and key-homomorphic, i.e., for all $\lambda, B \in \mathbb{N}$, for all $k_1, k_2 \in \mathsf{KGen}(\lambda)$, and all $i, i^* \in [B]$ with $i \neq i^*$ it holds that

- $\mathsf{Eval}(k_1, i) + \mathsf{Eval}(k_2, i) = \mathsf{Eval}(k_1 + k_2, i)$, and
- $\mathsf{Eval}(k_1, i) = \mathsf{PEval}(\mathsf{Punct}(k_1, i^*), i^*, i)$.

Additionally, we require that the outputs of the PRF are *pseudorandom*, which we formally define in Appendix B.4.

In our constructions, we instantiate the PRF with a pairing-based construction from [29] where keys $k \in \mathbb{F}_p$ are field elements, punctured keys $k^* \in \mathbb{G}_1$ are source-group elements, and evaluations $y \in \mathbb{G}_T$ are target-group elements.

In the following, we denote as λ_k the size of the key in bits. For BLS-12-381, we have that $\lambda_k = 256$ for $\lambda = 128$.

We formally define these building blocks within Appendix B.

3 Definitions

To model security, we elevate the definition of a batched threshold encryption scheme [18] to the silent setup setting [45].

Definition 3 (Silent Batched Threshold Encryption). A *Silent Batched Threshold Encryption* scheme (SBTE) consists of a tuple of PPT algorithms $\mathsf{SBTE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{KVfy}, \mathsf{Prep}, \mathsf{Enc}, \mathsf{Vfy}, \mathsf{BatchDec}, \mathsf{PDVfy}, \mathsf{Comb})$ with the following syntax.

- $\mathsf{pp} \leftarrow\!\$\ \mathsf{Setup}(1^\lambda, B_{\max}, N)$: This algorithm initializes the scheme, receiving the security parameter $\lambda \in \mathbb{N}$, the maximum batch size $B_{\max}$, and the maximum number of members per committee N. It produces public parameters pp, which are an implicit input to all subsequent algorithms.
- $(\mathsf{pk}, \mathsf{sk}) \leftarrow\!\$\ \mathsf{KGen}(\mathsf{pp})$: The key generation algorithm returns a key pair (pk, sk).
- $1/0 \leftarrow \mathsf{KVfy}(\mathsf{pk})$: The key verification algorithm gets a public key pk and outputs 1 to indicate the public key is valid and 0 otherwise.
- $(\mathsf{ek}, \mathsf{ak}) \leftarrow \mathsf{Prep}(C, \{\mathsf{pk}_i\}_{i \in C})$: Prep is a deterministic algorithm that receives a committee $C \subseteq [N]$ of at most N parties and their public keys. It outputs a concise encryption key ek and aggregation key ak.
- $(\mathsf{ct}, \pi^{\mathsf{ct}}) \leftarrow\!\$\ \mathsf{Enc}(\mathsf{ek}, m, i, t)$: Given an encryption key ek, a message m, an index i, and a threshold $1 \leq t \leq N$, the encryption algorithm outputs the ciphertext ct for batch position i, along with a proof π^{ct}.
- $1/0 \leftarrow \mathsf{Vfy}(\mathsf{ek}, \mathsf{ct}, \pi^{\mathsf{ct}})$: Vfy is a deterministic algorithm that takes as input an encryption key ek, a ciphertext ct, and a proof π^{ct}. It outputs 1 if the proof is valid and 0 otherwise.
- $(\mathsf{pd}, \pi^{\mathsf{pd}})/\perp \leftarrow \mathsf{BatchDec}(\mathsf{sk}, \{\mathsf{ct}_i\}_{i \in [B]})$: Given a secret key sk and a batch of ciphertexts $\{\mathsf{ct}_i\}_{i \in [B]}$ where $B \leq B_{\max}$, the decryption algorithm generates a partial decryption pd along with a proof π^{pd}, or returns $\perp$.
- $1/0 \leftarrow \mathsf{PDVfy}(\mathsf{pd}, \{\mathsf{ct}_i\}_{i \in [B]}, \mathsf{pk}, \pi^{\mathsf{pd}})$: Given a partial decryption pd, a batch of ciphertexts $\{\mathsf{ct}_i\}_{i \in [B]}$, a public key

pk, and a proof π^{pd}, the partial decryption verification algorithm returns 1 to indicate that the partial decryption is valid and 0 otherwise.

- $\{m_i/\perp\}_{i\in[B]} \leftarrow \mathsf{Comb}(\mathsf{ak}, \{\mathsf{pd}_j\}_{j\in S}, \{\mathsf{ct}_i\}_{i\in[B]})$: The combining algorithm takes as input an aggregation key ak, a set of partial decryptions $\{\mathsf{pd}_j\}_{j\in S}$, and a batch of ciphertexts $\{\mathsf{ct}_i\}_{i\in[B]}$. It outputs decrypted messages $\{m_i\}_{i\in[B]}$ or an error symbol $\perp$.

We require that $B_{\max}$ and N are polynomials in λ.

Correctness. We require that SBTE schemes are *correct*, i.e., that, absent of adversarial interference, decryption of any set of up to $B_{\max}$ ciphertexts yields the correct messages. We defer a formal correctness definition to Appendix B.5.

Efficiency. The size of the partial decryption output by BatchDec must be sublinear in B, i.e., $o(B)$. Additionally, we require that each party executes BatchDec independently, that is, BatchDec is non-interactive and its runtime must be independent of N, i.e., it should run in time $\mathsf{poly}(B, \lambda)$. Setup must run in time $\mathsf{poly}(N, B_{\max}, \lambda)$, Enc in $O(1)$, and Comb in $\mathsf{poly}(N, B_{\max}, \lambda)$.

Remark 4. *For simplicity, we present our construction for committees of size $|C| = N$. However, we note that our constructions can easily be extended to support smaller committees $|C| < N$.*

Remark 5. *We explicitly define* Enc *to output a separate proof π^{ct} and introduce a dedicated verification algorithm* Vfy *for ciphertexts, rather than incorporating verification into* BatchDec. *This design reflects the requirement that ciphertexts must be publicly verifiable. Public verifiability is particularly important in the mempool application, where ciphertexts must be verifiable by block proposers.*

Following [18], we first introduce our definitions and constructions in the *coordinated* setting, where each ciphertext in a batch has a unique index i that is known at the time of encryption. Note that we can use the same sub-batching technique as [18] to remove coordination.

To model security, we adjust the definition of [18] of B-IND-CCA and rogue ciphertext security to the silent-setup setting.

*Definition 6 (*SB-IND-CCA*-security).* We say that an SBTE scheme Σ is SB-IND-CCA-secure if for all PPT-adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ there exists a negligible function negl such that

$$\Pr[\text{Game-SB-IND-CCA}^{\Sigma}_{\mathcal{A}}(1^\lambda) = 1] \le \frac{1}{2} + \mathsf{negl}(\lambda),$$

where Game-SB-IND-CCA is defined in Figure 1.

Rogue Ciphertext Security. [18] introduced the notion of Rogue Ciphertext Security to provide security against an adversary that introduces malformed ciphertexts into the batch of ciphertexts being decrypted such that correctness no longer holds for honestly generated ciphertexts. We extend the notion to the silent setup setting, where the adversary can generate the key pair and control partial decryptions on behalf of an arbitrary set of committee members. On a high level, an adversary can win Game-Rogue in one of two ways: it can create a batch of ciphertexts around an *honest encryption of a message m_i at position i*, such that all ciphertexts are valid, but either (a) at least one partial decryption by an honest committee member is invalid, or (b) decrypting the batch yields a message other than m_i at position i (or fails entirely).

Definition 7 (Rogue Ciphertext security). An SBTE scheme Σ is rogue ciphertext secure if for all PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ there exists a negligible function negl such that

$$\Pr[\text{Game-Rogue}^{\Sigma}_{\mathcal{A}}(1^\lambda) = 1] \le \mathsf{negl}(\lambda),$$

where Game-Rogue is defined in Figure 1.

Remark 8. *Note that Silent Threshold Encryption* STE *is a special case of* SBTE *with $B_{\max} = 1$. Therefore, we do not give a separate definition for* STE. *In* STE, *we call the decryption algorithm* Dec *instead of* BatchDec.

4 Additive Homomorphic STE

In this section, we present a variant of the construction from [45], modified to support additive homomorphism. This property is essential for applying the framework of [18] to construct a batched threshold encryption scheme, as we discuss in Section 5.

4.1 The Σ_{HSTE} Construction

Below, we describe a homomorphic version of the CPA-secure silent threshold encryption scheme due to [45]. We focus solely on the core algorithms in this description. Note that the presentation does not strictly adhere to the formal definition of STE.

Setup: Sample $\tau \leftarrow\!\$\ \mathbb{F}$ and $\gamma \leftarrow\!\$\ \mathbb{F}$ and publish the $\mathsf{crs} = ([\gamma]_2, [\tau^1]_1, \ldots, [\tau^n]_1, [\tau^1]_2, \ldots, [\tau^n]_2)$. Note that the tag γ is now included in the crs, allowing it to be reused across all ciphertexts.

KeyGen: The i-th party for $i \in [n]$ samples $\mathsf{sk}_i \leftarrow\!\$\ \mathbb{F}_p$ and publishes $\mathsf{pk}_i = ([\mathsf{sk}_i]_1, [\mathsf{sk}_i \cdot \tau]_1, \ldots, [\mathsf{sk}_i \cdot \tau^{n-1}]_1)$.

Preprocessing: Compute the encryption public key $\mathsf{ek} = [\mathsf{SK}(\tau)]_1 = [\sum_{i=1}^{n} \mathsf{sk}_i \cdot L_i(\tau)]_1$.

Before proceeding with the construction, we provide a brief overview of the ideas behind the encryption and decryption procedures. The encryption procedure is formulated as a witness encryption scheme for a certain relation, making decryption possible only with knowledge of the corresponding witness. We will now describe this particular relation; the details will become clear shortly. The linear relation is as follows: $\mathbf{A} \circ \mathbf{w} = \mathbf{b}$, where

$$\mathbf{A} = \begin{bmatrix} [\mathsf{SK}(\tau)]_1 & [1]_2 & [Z(\tau)]_2 & [\tau]_2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & [\tau^2]_2 & [1]_2 & 0 & 0 & 0 \\ 0 & [\gamma]_2 & 0 & 0 & 0 & [1]_1 & 0 & 0 \\ [\tau^t]_1 & 0 & 0 & 0 & 0 & 0 & [1]_2 & 0 \\ [1]_1 & 0 & 0 & 0 & 0 & 0 & 0 & [\tau]_1 \end{bmatrix}$$

$$\mathbf{w} = \begin{bmatrix} [B(\tau)]_2 \\ [-\mathsf{aSK}]_1 \\ [-Q_Z(\tau)]_1 \\ [-Q_x(\tau)]_1 \\ [\hat{Q}_x(\tau)]_1 \\ [\sigma^*]_2 \\ [-\hat{B}(\tau)]_1 \\ [-Q_0(\tau)]_2 \end{bmatrix}, \text{ and } \mathbf{b} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ [1]_T \end{bmatrix}. \tag{1}$$

This relation encodes five linear constraints that, together, ensure that at least t parties have provided valid decryption shares. Each party contributes a partial decryption, which can be interpreted as a BLS signature on the tag $[\gamma]_2$ using its secret key. Decryption

Game-SB-IND-CCA$_{\mathcal{A}}(1^\lambda)$

$c^\star \leftarrow \bot; b \leftarrow\$ \{0,1\}$
$\mathsf{pp} \leftarrow\$ \mathsf{Setup}(1^\lambda, B_{\max}, N)$
$(C, \mathsf{Cor}, \mathsf{st}_1) \leftarrow\$ \mathcal{A}_1(1^\lambda, \mathsf{pp})$ $\quad / \mathcal{H} \leftarrow C \setminus \mathsf{Cor}$
assert $\mathsf{Cor} \subseteq C \subseteq [N]$
$\forall \ell \in \mathcal{H}\colon (\mathsf{pk}_\ell, \mathsf{sk}_\ell) \leftarrow\$ \mathsf{KGen}(\mathsf{pp})$
$(\{\mathsf{pk}_\ell\}_{\ell\in\mathsf{Cor}}, m_0, m_1, i, t, \mathsf{st}_2)$
$\quad \leftarrow\$ \mathcal{A}_2^{\mathcal{O}^{\text{b-dec}}}(\mathsf{st}_1, \{\mathsf{pk}_\ell\}_{\ell\in\mathcal{H}})$
assert $\forall \ell \in \mathsf{Cor}\colon \mathsf{KVfy}(\mathsf{pk}_\ell) = 1$
assert $|\mathsf{Cor}| < t \ \wedge\ |m_0| = |m_1|$
$(\mathsf{ek}, \mathsf{ak}) \leftarrow \mathsf{Prep}(C, \{\mathsf{pk}_\ell\}_{\ell\in C})$
$(c^\star, \pi) \leftarrow \mathsf{Enc}(\mathsf{ek}, m_b, i, t)$
$b' \leftarrow\$ \mathcal{A}_3^{\mathcal{O}^{\text{b-dec}}}(\mathsf{st}_2, c^\star, \pi)$
return $b \stackrel{?}{=} b'$

$\mathcal{O}^{\text{b-dec}}(\{(c_j, \pi_j)\}_{j\in[B]}, \ell)$

if $\ell \notin \mathcal{H}$
  then return $\bot$
if $c^\star \in \{c_j\}_{j\in[B]}$
  then return $\bot$
for $j \in [B]$ **do**
  if $\mathsf{Vfy}(\mathsf{ek}, c_j, \pi_j) = 0$
    then return $\bot$
return $\mathsf{BatchDec}(\mathsf{sk}_\ell, \{c_j\}_{j\in[B]})$

Game-Rogue$_{\mathcal{A}}(1^\lambda)$

$\mathsf{pp} \leftarrow\$ \mathsf{Setup}(1^\lambda, B_{\max}, N); (C, \mathsf{Cor}, \mathsf{st}) \leftarrow\$ \mathcal{A}_1(1^\lambda, \mathsf{pp})$ $\quad / \mathcal{H} \leftarrow C \setminus \mathsf{Cor}$
assert $\mathsf{Cor} \subseteq C \subseteq [N] \wedge C \neq \emptyset$
$\forall \ell \in \mathcal{H}\colon (\mathsf{pk}_\ell, \mathsf{sk}_\ell) \leftarrow\$ \mathsf{KGen}(\mathsf{pp})$
$(\{\mathsf{pk}_\ell\}_{\ell\in\mathsf{Cor}}, m, i, t, \mathsf{st}) \leftarrow\$ \mathcal{A}_2(\mathsf{st}, \{\mathsf{pk}_\ell\}_{\ell\in\mathcal{H}})$
assert $\forall \ell \in \mathsf{Cor}\colon \mathsf{KVfy}(\mathsf{pk}_\ell) = 1 \wedge 0 < t \leq N$
$(\mathsf{ek}, \mathsf{ak}) \leftarrow \mathsf{Prep}(C, \{\mathsf{pk}_\ell\}_{\ell\in C}); (\mathsf{ct}_i, \pi_i^{\mathsf{ct}}) \leftarrow\$ \mathsf{Enc}(\mathsf{ek}, m, i, t)$
$(B, S, \{(\mathsf{ct}_j, \pi_j^{\mathsf{ct}})\}_{j\in[B]\setminus\{i\}}, \{\mathsf{pd}_\ell, \pi_\ell^{\mathsf{pd}}\}_{\ell\in S\cap\mathsf{Cor}}) \leftarrow\$ \mathcal{A}_3(\mathsf{st}, \mathsf{ct}_i, \pi_i^{\mathsf{ct}})$
assert $B \leq B_{\max} \wedge i \in [B] \wedge S \subseteq C \wedge |S| \geq t$
assert $\forall j \in [B]\colon \mathsf{Vfy}(\mathsf{ek}, \mathsf{ct}_j, \pi_j^{\mathsf{ct}}) = 1$
$\forall \ell \in S \cap \mathcal{H}\colon (\mathsf{pd}_\ell, \pi_\ell^{\mathsf{pd}}) \leftarrow \mathsf{BatchDec}(\mathsf{sk}_\ell, \{\mathsf{ct}_j\}_{j\in[B]})$
if $\exists \ell \in S \cap \mathcal{H}\colon \mathsf{PDVfy}(\mathsf{pd}_\ell, \{\mathsf{ct}_j\}_{j\in[B]}, \mathsf{pk}_\ell, \pi_\ell^{\mathsf{pd}}) = 0$ **then return** 1
assert $\forall \ell \in S \cap \mathsf{Cor}\colon \mathsf{PDVfy}(\mathsf{pd}_\ell, \{\mathsf{ct}_j\}_{j\in[B]}, \mathsf{pk}_\ell, \pi_\ell^{\mathsf{pd}}) = 1$
$\{m_j\}_{j\in[B]} \leftarrow \mathsf{Comb}(\mathsf{ak}, \{\mathsf{pd}_\ell\}_{\ell\in S}, \{\mathsf{ct}_j\}_{[B]})$
if $m_i \neq m$ **then return** 1 **else return** 0

Figure 1: The SB-IND-CCA **(left) and rogue ciphertext security game (right) of** SBTE**. We also define the semantic security game for silent setup encryption [45]** Game-S-IND-CPA **as the left game, but without providing access to $\mathcal{O}^{\text{b-dec}}$ to the adversary, and limiting $B_{\max} = 1$ (highlighted in gray).**

then requires a valid aggregated signature σ^* on $[\gamma]_2$ derived from t valid shares.

For every subset $S \subseteq [n]$ with $|S| \geq t$, there exists a unique aggregated public key $\mathsf{aPK}_S = [\mathsf{aSK}]_1$, derived from the corresponding individual public keys. The aggregated signature σ^* must then be verified against aPK_S. This verification is captured by the third linear constraint:

$$[\mathsf{aSK}]_1 \circ [\gamma]_2 = [1]_1 \circ [\sigma^*]_2.$$

To ensure that aPK_S is a valid aggregated key corresponding to a qualified set S with $|S| \geq t$, we define a polynomial B such that $B(\omega^i) = 0$ for all $i \notin S$. The correctness of aPK_S is then enforced by the first constraint, derived from the sumcheck lemma (Lemma 1):

$$[\mathsf{SK}(\tau)]_1 \circ [B(\tau)]_2 = \mathsf{aPK}_S \circ [1]_2 + [Q_x(\tau)]_1 \circ [\tau]_2 + [Q_Z(\tau)]_1 \circ [Z(\tau)]_2$$

We enforce three additional constraints:

- B must represent an authorized subset, i.e., it is non-zero for at least t positions. This is enforced by the fourth constraint, which verifies that $deg(B) \leq n - t$ by checking that $[\tau^t] \circ [B(\tau)]_2 = [\hat{B}(\tau)]_1 \circ [1]_2$, where $\hat{B}(X) = X^t B(X)$.
- The degree of Q_x must be at most $|\mathbb{H}| - 2$, per the requirements of the sumcheck lemma. This is enforced by the second constraint, which translates to a degree check on Q_x.
- $B(0) = 1$ to rule out the trivial (zero) polynomial. This is enforced by the fifth constraint which translates to KZG [54] verification: $[1]_1 \circ [B(\tau)]_2 = [\tau]_1 \circ [Q_0(\tau)]_2 + 1$, where $Q_0(X) = \frac{B(X)-1}{X}$.

These constraints collectively verify that $|S| \geq t$. We elaborate further in the decryption phase.

Encrypt: To encrypt a message $m \in \mathbb{G}_T$ to public keys $(\mathsf{pk}_1, \ldots, \mathsf{pk}_n)$, sample randomness $\mathbf{s} = (s_1, \ldots, s_5) \leftarrow\$ \mathbb{F}^5$. The ciphertext is then computed as: $\mathsf{ct} = \left(\mathsf{ct}_1 = \mathbf{s}^\top \cdot \mathbf{A}, \mathsf{ct}_2 = \mathbf{s}^\top \cdot \mathbf{b} + m\right)$.

Partial Decryption: The i-th party computes $\sigma_i = \mathsf{sk}_i \cdot [\gamma]_2$, and publishes the partial decryption $\mathsf{pd}_i = \mathsf{ct}_{1,6} \circ \sigma_i$. Note that the element $\mathsf{ct}_{1,6}$ corresponds to $[\mathbf{s}_3]_1$.

Combine: Given t partial decryptions, from a set of parties $S \subseteq [n]$. The combiner executes the following:

(1) Interpolate polynomial B at $B(0) = 1$, and $\{B(\omega^i) = 0\}_{i\in[n]\setminus S}$.
(2) Compute the aggregated public key corresponding to the set S: $[\mathsf{aSK}]_1 = \frac{1}{n}\left(\sum_{i\in S} B(\omega^i) \cdot [\mathsf{sk}_i]_1\right)$
(3) Define the polynomials $Q_x(X)$, and $Q_Z(X)$ that are required in the sumcheck lemma (Lemma 1). That is: $\mathsf{SK}(X) \cdot B(X) = \mathsf{aSK} + Q_x(X)X + Q_Z(X)Z(X)$. Evaluate $[B(\tau)]_2$, $[-Q_x(\tau)]_1$, and $[-Q_Z(\tau)]_1$.
(4) Aggregate the partial decryptions: $\mathsf{pd}^* = \frac{1}{n}\left(\sum_{i\in S} B(\omega^i) \cdot \mathsf{pd}_i\right)$.
(5) Set $\hat{Q}_x(X) = Q_x(X) \cdot X^2$ and evaluate $[\hat{Q}_x(\tau)]_1$. This is used to prove that Q_x is of degree at most $n - 2$.
(6) Set $Q_0(X) = (B(X) - 1)/X$ and evaluate $[Q_0(\tau)]_2$. This is a KZG proof that $B(0) = 1$.
(7) Set $\hat{B}(X) = B(X) \cdot X^t$ and evaluate $[\hat{B}(\tau)]_1$. This is used to prove that B is of degree at most $n - t$.

Then assemble the witness $\mathbf{w}$ as in Eq. (1), except that we ignore the sixth element (by putting 0 there) and instead we compute $[m]_T \leftarrow \mathsf{ct}_2 - (\mathsf{ct}_1 \circ \mathbf{w} + \mathsf{pd}^*)$.

Correctness and Additive Homomorphism. Follows by inspection for all ciphertexts that use the same tag $[\gamma]_2$.

Semantic Security. We provide a full security proof, accounting for our modifications over [45] and further optimizations introduced in Section 6, in Appendix D.1.

Our Modifications to [45]. First, in contrast to [45], where each ciphertext uses a fresh random γ, we use the same γ across all ciphertexts. Second, we modify the partial decryption. In [45], the partial decryption is simply σ_i. However, when a single γ is shared across all ciphertexts, such a partial decryption is no longer bound to any specific ciphertext, causing all ciphertexts to be decrypted simultaneously. To avoid this, in pd_i, we bind each partial decryption to a specific ciphertext using $[\mathbf{s}_3]_1$, which is unique to each ciphertext. Such binding is crucial for proving the CCA security of the silent batched threshold encryption scheme presented in Section 7.

5 Silent BTE from Σ_{HSTE}

In this section, we demonstrate how to apply the framework from [18] using an additive homomorphic STE. The construction outlined here focuses on the core ideas underlying the design of silent batched threshold encryption. At this stage, we do *not* address potential attacks that compromise CCA security (Definition 6) or rogue ciphertext security (Definition 7); these considerations are addressed in Section 7 as part of our final construction.

We refer the reader to Section 1.2 for a recap of BEAT-MEV, the batched threshold encryption scheme from [18]. Here we describe how to remove the interactive setup using the homomorphic STE scheme as described in Section 4. This requires a bit-decomposition of the PRF keys.

5.1 Baseline SBTE Construction

Next, we show a baseline SBTE that combines our Σ_{HSTE} with BEAT-MEV [18], followed by a discussion of its limitations. Let PRF be a key-homomorphic puncturable PRF. The scheme is as follows:

- Setup: Execute Σ_{HSTE}.Setup and PRF.Setup. The output of the setup is a crs.
- KGen: Each party intending to participate in decryption locally runs Σ_{HSTE}.KGen using the crs, and publishes its public key. Let pk_u denote the public key of party u.
- Enc: Given a message $m_i \in \mathbb{G}_T$, the crs, the public keys, and an index $i \in [B]$:
  - Sample a PRF key k_i and puncture it at index i as $k_i^* \leftarrow \mathsf{Punct}(k_i, i)$. Compute $\beta_i = m_i + \mathsf{PRF}(k_i, i)$.
  - Decompose k_i into λ_k bits, and encrypt each bit using Σ_{HSTE} to the committee, yielding $(v_{i,1}, \ldots, v_{i,\lambda_k})$, where each $v_{i,j} = \Sigma_{\mathsf{HSTE}}.\mathsf{Enc}(k_{i,j})$ encrypts the j-th bit of k_i under the public keys $\{\mathsf{pk}_1, \ldots, \mathsf{pk}_n\}$.
  - Output $\mathsf{ct}_i = (k_i^*, \beta_i, v_{i,1}, \ldots, v_{i,\lambda_k})$.
- BatchDec: Given a batch of ciphertexts $\{\mathsf{ct}_1, \ldots, \mathsf{ct}_B\}$, each party $u \in [n]$ in the committee performs:
  - Homomorphic aggregation of the encrypted bits $\{v_j \leftarrow \sum_i \mathsf{ct}_i.v_{i,j}\}_{j\in[\lambda_k]}$.
  - Computation and publication of the batch-bound partial decryption using sk_u: $\{\mathsf{pd}_{u,j} = \Sigma_{\mathsf{HSTE}}.\mathsf{Dec}(\mathsf{sk}_u, v_j)\}_{j\in[\lambda_k]}$. These partial decryptions are bound to the specific batch context, ensuring they cannot be reused to decrypt ciphertexts encrypted under the same tag.
- Comb: Let $k = \sum_{i=1}^{B} k_i$ denote the aggregate PRF key. Given partial decryptions from a set $S \subseteq [n]$ such that $|S| \geq t$, the j-th chunk of k, denoted $k^{(j)}$, can be recovered by aggregating the corresponding set of partial decryptions.[8] $[k^{(j)}]_T = \Sigma_{\mathsf{HSTE}}.\mathsf{Comb}(\{\mathsf{pd}_{u,j}\}_{u\in S}, v_j)$. Then, using k, we can recover $\{m_i\}_{i\in[B]}$ using a similar strategy as in BEAT-MEV [18].

Limitations of the Construction. While the above construction is *technically* a batched-silent threshold encryption scheme, its concrete efficiency falls significantly short of the requirements for practical encrypted mempools.

First, the bit decomposition of the PRF key introduces a λ_k-fold blowup in the ciphertext size. Specifically, each Σ_{HSTE} ciphertext includes $3\mathbb{G}_1$ elements, $5\mathbb{G}_2$ elements, and $1\mathbb{G}_T$ element, totaling approximately 1200 bytes at the 128-bit security level using the BLS-12-381 curve. Since the PRF key is $\lambda_k = 256$ bits, this results in a ciphertext of over 300 KiB + $|m|$. On blockchains, transactions are typically smaller than 1 KiB, resulting in a $\approx 300\times$ overhead.

Moreover, each party in the committee publishes 256 target group elements as their partial decryption amounting to $\approx$ 147 KiB!

To put these overheads in context, consider a naive scheme for encrypted mempools. Each committee member simply samples an El Gamal key pair and publishes their public key. To encrypt their transaction clients first secret share their message and encrypt the i-th share under the i-th committee members public key. Indeed, this scheme does not have an interactive setup, but the ciphertext size grows with $O(n)$, and the partial decryptions of each party are $O(B)$ in length. Concretely, each ciphertext is 64 bytes for a 32 byte message and each partial decryption is 32 bytes. For typical parameters of $n = 512$ and $B = 1024$, the ciphertext size is 32 KiB (order of magnitude smaller!) and the partial decryption size from each party is 32 KiB.

Although the baseline construction is theoretically interesting, as it establishes a feasibility result of a silent batched threshold encryption, the performance overhead renders it impractical to solve the problem of building encrypted mempools. In Section 6, we present a number of optimizations aimed at making the construction suitable for real-world deployment.

6 Making Σ_{HSTE} Practical

In this section we present a series of optimizations that are applied to the Σ_{HSTE} construction to make it feasible for practical use. When the resulting construction is instantiated within the framework of [18], it yields a practically efficient silent batched threshold encryption construction. The optimizations are summarized as follows:

- Chunking: Rather than performing bit-decomposition, we partition the key into chunks of size ℓ. This approach introduces the challenge of ensuring decryption remains efficient, both computationally and in terms of memory usage.
- Reusing randomness across ciphertexts: This is crucial to reducing the ciphertext size. In most cryptographic schemes, reusing randomness breaks semantic security. Therefore,

[8] Note that $k^{(j)} = \sum_{i\in B} k_{i,j} \in [0, B]$. Thus, the discrete logarithm can be efficiently recovered through brute force.

our goal is to design a method that allows randomness reuse without compromising the security of the scheme.
- Avoiding target group elements:
  - To compress the ciphertext, we "multiplicatively" share the $\mathbb{G}_T$ element in the ciphertext across $\mathbb{G}_1$ and $\mathbb{G}_2$.
  - A novel method for computing partial decryptions that are tied to individual ciphertexts, unlike the method employed in [45]. Notably, this allows us to move partial decryptions to $\mathbb{G}_1$. Further, it allows us to release one partial decryption per party instead of one per chunk per party, reducing partial decryption size by an additional factor of ℓ.

We next discuss these optimizations in more detail.

6.1 Chunking

Instead of encrypting every bit of k separately, resulting in λ ciphertexts, we decompose k into ℓ chunks $(k_1, \ldots, k_\ell)$ of size $z = \lambda_k/\ell$ bits such that $k = \sum_{j\in[\ell]} k_j \cdot 2^{(j-1)z}$. This allows us to reduce the number of Σ_{HSTE} ciphertexts from λ_k to ℓ, at the cost of now extracting the discrete logarithm from decrypted $(K_1, \ldots, K_\ell) \in \mathbb{G}_T^\ell$. For a reasonable choice of ℓ, this can be done efficiently using Pollard's rho method [67, 76] or baby-steps giant-steps [24] due to the restriction that $\mathsf{DL}(K_j) \in [0, 2^{z+\log(B_{\max})} - 1]$. In our evaluation, we choose $\ell = 16$ chunks, leading to a chunk size of 16 bits. Smaller ℓ leads to smaller ciphertext size, but more expensive dlog extraction during decryption.

6.2 Reusing Randomness across Chunks

When applying chunking to batched silent threshold encryption, we encrypt ℓ chunks of the PRF key using *independent* randomness to the same tag γ. This blows up the ciphertext and the partial decryption size by ℓ. To save on ciphertext cost, we propose *sharing* the randomness across the ciphertexts by *extending* the linear relation.[9] For every chunk $j \in [2, \ell]$, in the setup we sample $r_j \leftarrow\!\!\$\ \mathbb{F}_p$. For ease of notation, we set $r_1 \coloneqq 1$. We define $\mathbf{w}_j$ and $\mathbf{b}_j$ as follows

$$\mathbf{w}_j = r_j \cdot \mathbf{w}, \text{ and } \mathbf{b}_j = r_j \cdot \mathbf{b}. \tag{2}$$

The extended relation for silent threshold encryption can be represented as: $\mathbf{A} \circ \mathbf{w}_j = \mathbf{b}_j$, for every $j \in [\ell]$.

In the witness encryption, each column $\mathbf{b}_j$ for $j \in [\ell]$ will now result in a separate *mask* which can be used to encrypt the j-th message m_j. Concretely, we sample randomness $\mathbf{s} = [s_1, s_2, s_3, s_4, s_5] \leftarrow\!\!\$\ \mathbb{F}_p^5$ and set the ciphertext to be:

$$\begin{aligned}
\mathsf{ct}_1 &= s_1 \cdot [\mathsf{SK}(\tau)]_1 + s_4 \cdot [\tau^t]_1 + s_5 \cdot [1]_1,\\
\mathsf{ct}_2 &= s_1 \cdot [1]_2 + s_3 \cdot [\gamma]_2, \quad \mathsf{ct}_3 = s_1 \cdot [Z(\tau)]_2,\\
\mathsf{ct}_4 &= s_1 \cdot [\tau]_2 + s_2 \cdot [\tau^2]_2, \quad \mathsf{ct}_5 = s_2 \cdot [1]_2,\\
\mathsf{ct}_6 &= s_3 \cdot [1]_1, \quad \mathsf{ct}_7 = s_4 \cdot [1]_2, \quad \mathsf{ct}_8 = s_5 \cdot [\tau]_1,\\
\{\mathsf{ct}_{9,j} &= s_5 \cdot [r_j]_T + m_j \cdot [1]_T\}_{j\in[\ell]},
\end{aligned}$$

which effectively allows us to securely reuse randomness $\mathbf{s}$ across all chunks, and thus *rate-1* ciphertext size.

The ciphertext size is now $5 \cdot \mathbb{G}_2 + 3 \cdot \mathbb{G}_1 + \ell \cdot \mathbb{G}_T$ to encrypt ℓ chunks, which saves $(\ell - 1)(5\mathbb{G}_2 + 3\mathbb{G}_1)$ elements, which amounts to around 9.1 KiB in savings for $\ell = 16$.

[9] [58] used a similar strategy but for the El Gamal and Cramer-Shoup encryption schemes.

The decryption proceeds as in Σ_{HSTE}. However, we now run ℓ decryption instances: the i-th instance uses the column $\mathbf{b}_i$ and generates the witness $\mathbf{w}_i$.

Semantic Security. While we defer the security proof to the final scheme (including all optimizations), we provide some intuition for the security of this scheme at this point. In the GGM, by treating the randomness chosen during encryption, the trapdoor τ, as well as $\{r_i\}_{i\in[2,\ell]}$ as formal variables, all terms in the relation can be viewed as polynomials in these variables. Then, we argue that given the view of the adversary, if the column $\mathbf{b}_j$ is *independent* of the columns of $\mathbf{A}$ and the remaining columns $\mathbf{b}_v$ for every $v \in [\ell] \setminus \{j\}$, then this implies that the mask corresponding to the j-th column is indistinguishable from a uniformly random group element, which in turn guarantees semantic security of the underlying message.

It now remains to argue that in the silent threshold encryption scheme, this condition does hold for every column $\mathbf{b}_j$ for $j \in [\ell]$.

It was already shown in [45, Lemma 2] that $\mathbf{b}$ is indeed independent of the columns of $\mathbf{A}$ and other terms in the adversary's view in the baseline construction, unless the adversary does not receive at least one additional partial decryption (signature on $[\gamma]_2$). Intuitively, we can extrapolate because, given the adversary's view, any given $[r_j s_5]_T$ is independent of all $[r_v s_5]_T$ for $v \neq j$.

6.3 Avoiding Target Group Elements

We describe how to avoid using target group elements ($\mathbb{G}_T$) in both the ciphertext and the partial decryptions.

In the Ciphertext. Instead of including $\mathbb{G}_T$ elements $\{[r_i s_5]_T + [m_i]_T\}_{i\in[\ell]}$ in the ciphertext, we multiplicatively secret-share them across $\mathbb{G}_1$ and $\mathbb{G}_2$. Concretely, we replace each $\mathbb{G}_T$ element with one element in $\mathbb{G}_1$ and one in $\mathbb{G}_2$, and send: $v \cdot (s_5[r_i]_1 + [m_i]_1)$ and $v^{-1} \cdot [1]_2$, where $v \leftarrow\!\!\$\ \mathbb{F}_p$ is chosen at random at the time of encryption. This reduces the size of the ciphertext by $\ell \cdot (576 - (48 + 96))$ bytes. For $\ell = 16$, this amounts to 6.75 KiB in savings.

To further reduce the ciphertext size, we sample $v_i \leftarrow\!\!\$\ \mathbb{F}_p$ for every $i \in [\ell]$ during setup and include the shares $[r_i v_i]_1$, $[v_i^{-1}]_2$ as part of the crs. These values are then reused across all ciphertexts. As a result, each ciphertext only needs to include the $\mathbb{G}_1$ element, while the corresponding $\mathbb{G}_2$ element can be derived from the crs. At the time of encryption, we then set $\mathsf{ct}_{9,i} = (s_5 + m_i) \cdot [r_i v_i]_1$. This reduces the size of the ciphertext by an additional $\ell \cdot 96$ bytes, which amounts to 1.5 KiB for $\ell = 16$. Note that this works only if the message m is a field element. As a tradeoff, recovering $\{m_i\}_{i\in[\ell]}$ from $\{[m_i r_i]_T\}_{i\in[\ell]}$ requires running ℓ independent instances of the underlying discrete log solver, since each instance corresponds to a different base.

Putting It All Together. We now have a ciphertext that is

$$\underbrace{5 \cdot \mathbb{G}_2 + 3\mathbb{G}_1 + \ell \cdot \mathbb{G}_1}_{\mathsf{STE}} + \underbrace{\mathbb{G}_1}_{\mathsf{k}^*} + |m| = 1440 \text{ bytes} + |m|$$

on the BLS12-381 curve for $\ell = 16$.

In the Partial Decryption. Interestingly, we observe that the partial decryption share can be modified to be $[\mathbf{s}_3 \mathsf{sk}]_1$ instead of $[\mathbf{s}_3]_1 \circ [\mathsf{sk}\gamma]_2$. This modification does not affect the decryption process, as the term $[\mathbf{s}_3 \gamma \mathsf{sk} r_i]_T$ can still be computed via the pairing

$[\mathbf{s}_3\mathsf{sk}]_1 \circ [\gamma r_i]_2$. Importantly, this change does not compromise semantic security, since the ciphertext remains unaltered. As a result, the size of the partial decryption is now one $\mathbb{G}_1$ element, which is of size only 48 bytes. Crucially, the size of the partial decryption remains constant regardless of the parameter ℓ. Without this optimization, the size of the partial decryption is $\ell \cdot \mathbb{G}_T$, corresponding to 9 KiB when $\ell = 16$. Our optimization thus achieves a 192× size reduction per partial decryption.

We show later that this modification preserves CCA security, a necessary property for silent batched threshold encryption schemes.

6.4 The Optimized HSTE Construction

We next summarize the optimized construction Σ_{MSTE} which incorporates all optimizations discussed above.

Setup(1^λ): Sample $\tau, \gamma \leftarrow\!\!\$\ \mathbb{F}_p$, and $r_u, v_u \leftarrow\!\!\$\ \mathbb{F}_p$ for every $u \in [\ell]$, set $r_1 = 1$, and publish $\mathsf{crs} = \left\{[r_u]_{1,2}, [r_u\gamma]_2, [r_u\tau]_{1,2}, \ldots, [r_u\tau^n]_{1,2}, [r_u v_u]_1, [v_u^{-1}]_2\right\}_{u\in[\ell]}$.

KeyGen(crs): Sample $\mathsf{sk} \leftarrow\!\!\$\ \mathbb{F}_p$ and publish

$$\mathsf{pk} = \{[r_u \mathsf{sk} \cdot \tau^j]_1\}_{u\in[\ell], j\in[0,n-1]}.$$

We denote $\mathsf{pk}_{u,j} \coloneqq [r_u \mathsf{sk} \tau^j]_1$ for $u \in [\ell]$ and $j \in [0, n-1]$.

Preprocess: Compute the encryption public key as $[\mathsf{SK}(\tau)]_1 = [\sum_{i=1}^n \mathsf{sk}_i \cdot L_i(\tau)]_1$.

Encrypt: To encrypt a message $(m_1, \ldots, m_\ell) \in \mathbb{F}^\ell$ to public keys $(\mathsf{pk}_1, \ldots, \mathsf{pk}_n)$, sample $\mathbf{s} \leftarrow\!\!\$\ \mathbb{F}^5$. The ciphertext is computed as

$$\mathsf{ct} = \left(\mathsf{ct}_1 = \mathbf{s}^\top \cdot \mathbf{A}, \{\mathsf{ct}_{2,i} = (s_5 + m_i)[r_i v_i]_1\}_{i\in[\ell]}\right).$$

Dec$(\mathsf{sk}_i, \mathsf{ct})$: Output $\mathsf{pd}_i \leftarrow \mathsf{sk}_i \cdot \mathsf{ct}_{1,6}\big[= [\mathsf{sk}_i s_3]_1\big]$.

Combine$(\mathsf{crs}, \mathsf{ak}, \mathsf{ct}, \{\sigma_{i,j}\}_{i\in[n]})$: Given t partial decryptions, let $S \subseteq [n]$ be the set of parties that published partial decryptions. The combiner runs a similar algorirhm as in Σ_{HSTE}. In particular, it defines the polynomials $B, \hat{B}(X), Q, Q_Z, Q_x$, $\hat{Q}_x, Q_0(X)$ as in Σ_{HSTE}. Then, the witness $\mathbf{w}_j$ for every $j \in [\ell]$ is similar to the witness in Σ_{HSTE}, but with respect to the j-th pair of generators $([r_j]_1, [r_j]_2)$. I.e., the witness $\mathbf{w}_j$ is as in Eq. (2) (omitting the sixth element), where $[r_j \mathsf{aSK}]_1 = \frac{1}{n}\left(\sum_{i\in S} B(\omega^i) \cdot [r_j sk_i]_1\right)$ for every $j \in [\ell]$. It is straightforward to verify that each $\mathbf{w}_j$ is computable from the crs and the public keys. Next, for every $j \in [\ell]$ we compute: $\mathsf{pd}_j^* = \frac{1}{n}\left(\sum_{i\in S} B(\omega^i) \cdot \mathsf{pd}_i \circ [r_j\gamma]_2\right)$. Finally, given $\mathbf{w}_j$ and pd_j^*, compute: $[m_j r_j]_T = \mathsf{ct}_{2,j} \circ [v_j^{-1}]_2 - (\mathsf{ct}_1 \circ \mathbf{w}_j + \mathsf{pd}_j^*)$. Note that for a message $(m_1, \ldots, m_\ell) \in \mathbb{F}^\ell$, the decryption recovers it *in the target group with respect to generators* $\{[r_j]_T\}_{j\in[\ell]}$, i.e., as $([m_1 r_1]_T, \ldots, [m_\ell r_\ell]_T) \in \mathbb{G}_T^\ell$. This is sufficient for our use case in constructing SBTE as we later extract each m_i by solving the discrete logarithm with respect to the generator $[r_i]_T$.

A formal proof of Lemma 9 is presented in Appendix D.1.

Lemma 9 (Semantic Security). *The modified STE construction* Σ_{MSTE} *is semantically secure (Definition 20) in the GGM.*

7 The Final Construction

In this section, we introduce the full-fledged silent batched threshold encryption. It builds upon the baseline construction from Section 5, replacing Σ_{HSTE} by Σ_{MSTE} as the underlying STE. In addition, we enhance the construction with non-interactive zero-knowledge proofs to achieve CCA and rogue ciphertext security. A full construction overview is depicted in Figs. 2 and 3.

As in the baseline construction, in encryption, the client samples a PRF key $k^{(i)}$. Instead of bit-decompostion, we now decompose the key into $\ell = 16$ chunks $(k_1^{(i)}, \ldots k_\ell^{(i)})$ and encrypt each sub-key using Σ_{MSTE}. The client attaches a Pedersen commitment to each $k_j^{(i)}$, a range proof π^{range} that all $k_j^{(i)} \in [0, 2^z - 1]$, as well as a proof of knowledge π^{cca} of the randomness involved. Both proofs are required to prevent rogue ciphertext attacks, and the latter allows us to achieve CCA security, making use of extraction techniques.

In partial decryption for a batch of ciphertexts, each committee member verifies the corresponding proofs and then generates the partial decryption as in Σ_{MSTE}. Each partial decryption is accompanied by a publicly verifiable proof π^{pd} to ensure correct decryption.

Given partial decryptions $\{\mathsf{pd}_i\}_{i\in S}$ from a set S of at least t committee members, the combiner first verifies the proofs associated with each partial decryption, and continues to run Comb with only the valid partial decryptions. I.e., the combiner aggregates the partial decryptions as in Σ_{MSTE}, yielding $(K_1, \ldots, K_\ell) \in \mathbb{G}_T^\ell$, where $K_m = \sum_{i\in[B]} k_m^{(i)} \cdot [r_m]_T$. Then it runs DL extraction ℓ times to recover $(k_1, \ldots, k_\ell) \in \mathbb{F}_p^\ell$, and reconstructs the key as $k = \sum_{j\in[\ell]} k_j \cdot 2^{(j-1)z}$. Observe that $k = \sum_{i\in[B]} k^{(i)}$, thus decryption proceeds as in BEAT-MEV.

Finally, we elaborate in more detail on the proof systems needed for CCA and rogue ciphertext security within Appendix C.

Theorem 10 (Security of Σ_{SB}). *The* SBTE *scheme* Σ_{SB} *achieves* SB-IND-CCA *and rogue ciphertext security.*

For a formal proof of Theorem 10, we defer the reader to Appendix D.2 (proof sketch and full proof of SB-IND-CCA security) and D.3 (full proof of rogue ciphertext security).

8 Experimental Evaluation

We experimentally evaluate our batched threshold encryption scheme with silent setup. We present our benchmarks alongside those of prior work on batched threshold encryption [3, 18, 22, 23] and threshold encryption with silent setup [45]. We emphasize that this is only to provide context, as our results are incomparable to all prior work, which either rely on an *interactive* setup or do not support batched decryption. In large-scale decentralized systems such as Ethereum, where the committee in charge of voting for the next block changes every 12 seconds, a DKG-based setup is simply not feasible. Our implementation is available at: https://github.com/guruvamsi-policharla/batched-silent-threshold-encryption

Setup. All of our experiments were run on a MacBook Pro with an Apple M5 Pro chip and 48 GB of RAM in single-threaded mode. We use `arkworks` [5] for implementations of pairing-friendly curves and their associated algebra, with benchmarks run over the BLS12-381 curve. Throughout the evaluation, we instantiate our scheme with $\ell = 16$ chunks of $z = 16$ bits each (cf. Section 6). We benchmark

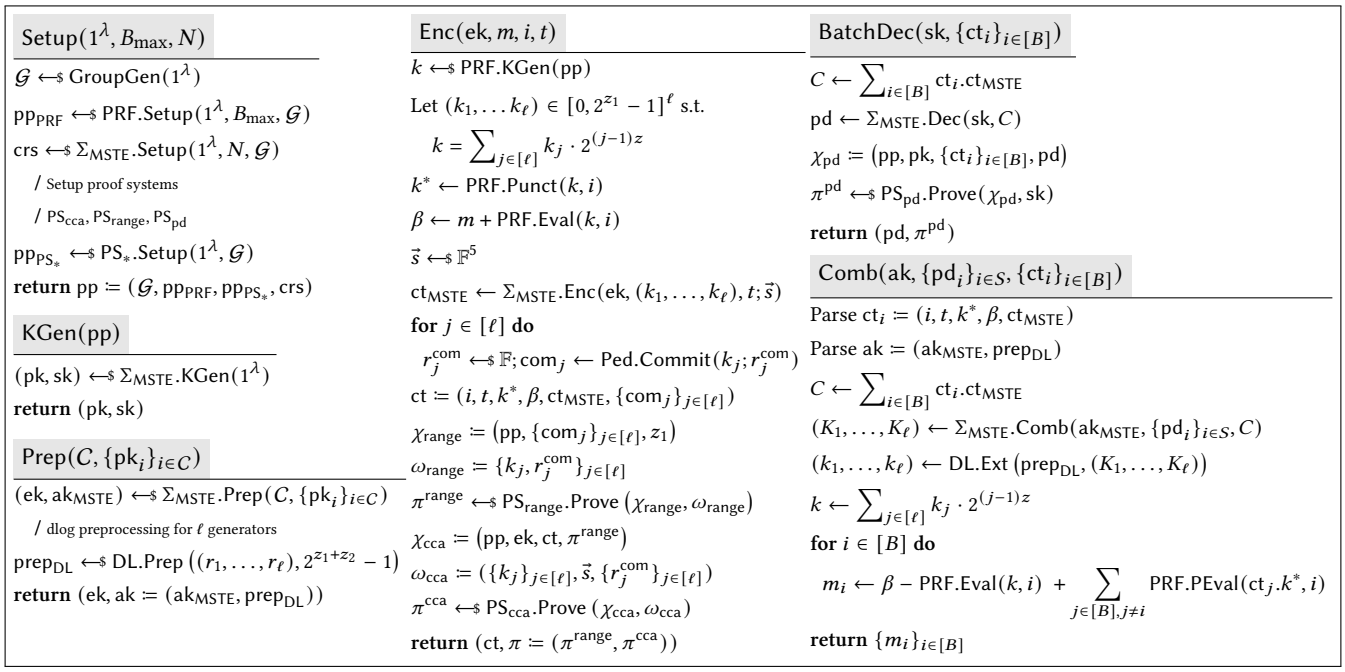


Figure 2: Construction of the SBTE scheme Σ_{SB} with ℓ chunks. We denote $z_1 = \lceil \lambda_k/\ell \rceil$ and $z_2 = \lceil \log(B_{max}) \rceil$.

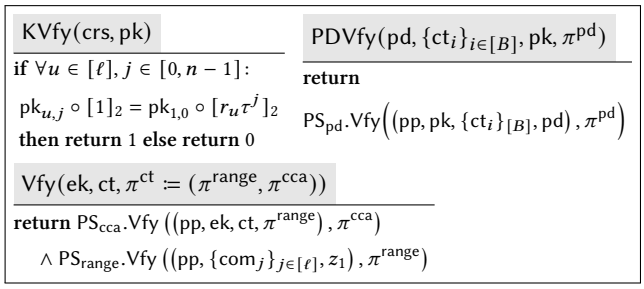


Figure 3: Verification algorithms of Σ_{SB}

Batch size	Partial Decr. (ms)			Reconstruction (s)		
	[3, 23]	[18]	Σ_{SB}	[3, 23]	[18]	Σ_{SB}
8	0.33	0.05	0.05	0.01	0.02	0.37
32	0.77	0.07	0.06	0.05	0.21	0.47
128	1.97	0.13	0.09	0.21	3.06	0.98
512	6.13	0.39	0.21	0.91	48.15	3.28

Figure 4: Time taken to compute partial decryptions and reconstruct messages for varying batch size using $n = 128$ with threshold $t = n/2$. Reported times exclude the generation and verification of proofs of correct decryption (cf. Fig. 6).

Reconstruction Breakdown for Σ_{SB} (ms)				
Batch size	STE Decrypt	DLog	FFT	Total
8	272	73	25	370
32	273	76	126	475
128	277	87	612	977
512	291	86	2899	3277

Figure 5: Component-wise breakdown of reconstruction time for Σ_{SB} using the FFT-based PPRF evaluation, with $n = 128$ parties and threshold $t = n/2$.

the fully secure version of our scheme, which attaches the NIZK proofs from Appendix C required for CCA and rogue ciphertext security. A component-wise breakdown of the proof costs is given in Fig. 6.

Encryption. Given an aggregated public key ek, encryption—including the generation of the attached Pedersen commitments, the range proof π^{range}, and the linkage proof π^{cca}—takes ≈ 106 ms (≈ 9.1 ms for the encryption itself and ≈ 97 ms for generating the attached proofs), independent of the batch size or the number of parties in the committee (cf. Fig. 6 for a breakdown of the proof costs). Verifying the proofs attached to a ciphertext takes ≈ 95 ms. Importantly, decryptors can start verifying proofs as soon as encrypted transactions hit the mempool, meaning verification can largely occur outside of the critical path between block publication and the execution of decrypted transactions. Our optimizations bring the ciphertext down to just 1.41 KiB, with the attached commitments

and proofs adding another 5.69 KiB. We give a comprehensive comparison of the size of relevant parameters (ciphertext, public keys, CRS, partial decryptions, etc.) in the Appendix (Fig. 9).

Partial Decryption. Decrypting a batch of ciphertexts in our scheme requires B group additions and one group exponentiation. This is similar to [18], but [3, 23] require a multi scalar multiplication of size B and a group exponentiation, making the partial decryption more expensive (c.f. Fig. 4). Each partial decryption is accompanied by a proof of correct decryption π^{pd}, instantiated as a compact Schnorr proof of just 2 field elements. For a batch of $B = 512$ ciphertexts, computing a partial decryption together with its proof takes $\approx$ 33 ms, as does verifying it, with the cost dominated by recomputing the aggregated batch ciphertext and hashing the batch into the Fiat–Shamir transcript. As this cost scales linearly in the batch size (cf. Fig. 6 for timings at $B \in \{8, 32, 128, 512\}$) and is incurred once per batch, it amortizes to $\approx$ 0.06 ms per ciphertext. Partial decryptions are very small at just one $\mathbb{G}_1$ element (48 bytes), plus the 64-byte proof.

Reconstruction. Given sufficiently many partial decryptions, we run the combine algorithm of silent threshold encryption [45] to first recover the *aggregate* key of the batch of ciphertexts. However, the key is actually decomposed and stored in the exponent of the target group, requiring us to solve the discrete logarithm problem for small exponents. Once we have the aggregate key, we then need to run the combine algorithm from [18] to recover the messages. Naïvely, this requires $O(B^2)$ pairings. Following the approach of [1], we observe that the aggregate PPRF evaluation across all batch indices can be cast as a circular cross-correlation, and replace the $O(B^2)$ loop with an FFT-based $O(B \log B)$ computation: we compute an FFT over $\mathbb{G}_1$ on the PPRF keys, perform $O(B)$ pointwise pairings in the frequency domain, and apply an inverse FFT over $\mathbb{G}_T$. This brings the asymptotic cost in line with [3, 23], who require $O(B \log^2 B)$ time,[10] but with the downside that messages must be encrypted to a particular "epoch" and they can only batch ciphertexts that have been encrypted to the same epoch. This results in a more complicated block-building strategy. We note that the optimization of introducing sub-batches in [18] simultaneously works for our scheme. Given the fact that we need to solve more discrete logarithms with more sub-batches, we can carefully choose the number of sub-batches to find the optimal setting. The partial decryption size grows linearly with the number of sub-batches. We provide benchmarks in Fig. 4 and a component-wise breakdown of our reconstruction in Fig. 5. The FFT optimization yields significant speedups at larger batch sizes: 3.5× at $B = 128$ and 14.8× at $B = 512$ compared to the naïve $O(B^2)$ approach.

Computing Discrete Logarithm. The strategy to solve the discrete logarithm for exponents smaller than E is to first store 'markers' $\mathcal{M} = \{i \cdot (E/M)\}_{i \in [0,M)}$ at M evenly spaced points in $[0, E)$ in a constant time look-up hash-map. Given the target instance t, we first compute $\mathcal{T} = \{t + i\}_{i \in [0, E/M)}$, and from the common element in $\mathcal{T} \cap \mathcal{M}$, we can recover the discrete logarithm. As a further storage optimization, we store a truncated representation of each marker in the hash-map. E is chosen based on the chunk bit-width and batch size. Concretely, we set $E = 2^{25}$ and $M = 2^{13}$, which requires only $\approx$ 200 KiB of storage. This allows us to support a maximum batch size of 2^9 with 16-bit chunks, and solving all discrete logarithms takes $\approx$ 90 ms. We emphasize that this is in single-thread mode, and this algorithm is trivially parallelizable. Furthermore, our implementation offers an effective time-space tradeoff by simply changing the number of markers stored.

NIZK Benchmarks for Σ_{SB} (ms)		
Component	**Prove**	**Verify**
Range proof π^{range}	78.9	70.1
Linkage proof π^{cca}	16.3	24.9
Ciphertext proof π^{ct} (total)	97.2	95.4
Partial decryption proof π^{pd} ($B = 8$)	0.6	0.6
Partial decryption proof π^{pd} ($B = 32$)	2.2	2.1
Partial decryption proof π^{pd} ($B = 128$)	8.3	8.3
Partial decryption proof π^{pd} ($B = 512$)	32.9	32.9

Figure 6: Time to generate and verify the NIZK proofs required for CCA and rogue ciphertext security cf. Appendix C) with $\ell = 16$ chunks. The ciphertext proof π^{ct} comprises the Pedersen commitments, the range proof π^{range}, and the linkage proof π^{cca}.

Index Collision Problem. As is inherent for BEAT-MEV-style constructions, our scheme can suffer *index collisions*. That is, decryption requires an index-collision-free assignment of transactions to (sub-)batches, and encryptors sample indices from a polynomially-sized domain.[11] Depending on the parameter choice, this can lead to censorship attacks, in which attackers suppress certain transactions by posting competing transactions with higher fees offered for the same index. Fortunately, both known mitigations can be directly applied to our scheme: (1) Users can encrypt their transactions to multiple indices [29], which is feasible thanks to our optimizations reducing ciphertext size. (2) We can use sub-batching [18] and sort ciphertext into sub-batches to avoid index collisions at the time of decryption. The above countermeasures can be tuned (i.e., ciphertext multiplicity and the number of sub-batches) to both reduce the likelihood of benign index collisions to a conventional statistical-correctness target (e.g., 2^{-40}) and make censorship attacks expensive to execute. In particular, if users encrypt their transactions to d indices and we allow at most s sub-batches, then there are $d \cdot s$ possible slots for the transaction within the batch. Thus, under fee-prioritized selection, and disregarding honest competing transactions for simplicity, an attacker must submit $d \cdot s$ competing transactions with higher fees.

Deployment Considerations. Our construction focuses on cryptographic security and efficiency, whereas a complete encrypted-mempool design must also handle practical considerations such

[10] This can be improved to $O(B \log B)$ with a smooth multiplicative domain such as the roots of unity of order n.

[11] The constraint here is that the crs-size of the key-homomorphic puncturable PRF grows linearly with the index domain size.

as invalid decrypted transactions, fee payment, and spam resistance. We believe these concerns should be addressed at the system level and fall outside the direct scope of this work. This separation is also adopted by current Ethereum proposals [31, 61], which leave the encryption scheme open and thus keep such mechanisms largely independent of the specific (S)BTE instantiation. The LUCID proposal [31], for example, validates plaintext transactions after decryption, charges sealed-transaction tickets before decryption, and notes the need for mempool-level spam filtering.

Acknowledgments

The work of the fourth and the sixth authors is supported in part by the AFOSR Award FA9550-24-1-0156, and research grants from the Bakar Fund, J. P. Morgan Faculty Research Award, Supra Inc., Sui Foundation, and the Stellar Development Foundation. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of these institutes.

This work is funded by the European Research Council (ERC) under the European Union's Horizon 2020 and Horizon Europe research and innovation programs (grant CRYPTOLAYER-101044770). Furthermore, this research is supported by the German Federal Ministry of Education and Research, as well as the Hessen State Ministry for Higher Education, Research and the Arts, through their joint sponsorship of the National Research Center for Applied Cybersecurity ATHENE.

References

[1] Amit Agarwal, Kushal Babel, Sourav Das, Babak Poorebrahim Gilkalaye, Arup Mondal, Benny Pinkas, Peter Rindal, and Aayush Yadav. 2025. Weighted Batched Threshold Encryption with Applications to Mempool Privacy. Cryptology ePrint Archive, Report 2025/2115. https://eprint.iacr.org/2025/2115

[2] Amit Agarwal, Sourav Das, Babak Poorebrahim Gilkalaye, Peter Rindal, and Victor Shoup. 2026. BTX: Simple and Efficient Batch Threshold Encryption. Cryptology ePrint Archive, Paper 2026/754. https://eprint.iacr.org/2026/754

[3] Amit Agarwal, Rex Fernando, and Benny Pinkas. 2025. Efficiently-Thresholdizable Batched Identity Based Encryption, with Applications. In *CRYPTO 2025, Part III (LNCS, Vol. 16002)*, Yael Tauman Kalai and Seny F. Kamara (Eds.). Springer, Cham, 69–100. doi:10.1007/978-3-032-01881-6_3

[4] Guillermo Angeris, Alex Evans, and Tarun Chitra. 2021. A Note on Bundle Profit Maximization. *Stanford University* (2021).

[5] arkworks contributors. 2022. `arkworks` zkSNARK ecosystem. https://arkworks.rs.

[6] Gennaro Avitabile, Nico Döttling, Bernardo Magri, Christos Sakkas, and Stella Wohnig. 2024. Signature-based Witness Encryption with Compact Ciphertext. Cryptology ePrint Archive, Report 2024/1477. https://eprint.iacr.org/2024/1477

[7] Foteini Baldimtsi, Konstantinos Kryptos Chalkias, Francois Garillot, Jonas Lindstrom, Ben Riva, Arnab Roy, Mahdi Sedaghat, Alberto Sonnino, Pun Waiwitlikhit, and Joy Wang. 2023. Subset-optimized BLS Multi-signature with Key Aggregation. Cryptology ePrint Archive, Report 2023/498. https://eprint.iacr.org/2023/498

[8] Joseph Bebel and Dev Ojha. 2022. Ferveo: Threshold Decryption for Mempool Privacy in BFT networks. Cryptology ePrint Archive, Report 2022/898. https://eprint.iacr.org/2022/898

[9] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. 2019. Aurora: Transparent Succinct Arguments for R1CS. In *EUROCRYPT 2019, Part I (LNCS, Vol. 11476)*, Yuval Ishai and Vincent Rijmen (Eds.). Springer, Cham, 103–128. doi:10.1007/978-3-030-17653-2_4

[10] Iddo Bentov, Yan Ji, Fan Zhang, Lorenz Breidenbach, Philip Daian, and Ari Juels. 2019. Tesseract: Real-Time Cryptocurrency Exchange Using Trusted Hardware. In *ACM CCS 2019*, Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz (Eds.). ACM Press, 1521–1538. doi:10.1145/3319535.3363221

[11] Alexandra Boldyreva. 2003. Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme. In *PKC 2003 (LNCS, Vol. 2567)*, Yvo Desmedt (Ed.). Springer, Berlin, Heidelberg, 31–46. doi:10.1007/3-540-36288-6_3

[12] Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. 2018. Verifiable Delay Functions. In *CRYPTO 2018, Part I (LNCS, Vol. 10991)*, Hovav Shacham and Alexandra Boldyreva (Eds.). Springer, Cham, 757–788. doi:10.1007/978-3-319-96884-1_25

[13] Dan Boneh and Xavier Boyen. 2004. Secure Identity Based Encryption Without Random Oracles. In *CRYPTO 2004 (LNCS, Vol. 3152)*, Matthew Franklin (Ed.). Springer, Berlin, Heidelberg, 443–459. doi:10.1007/978-3-540-28628-8_27

[14] Dan Boneh, Xavier Boyen, and Eu-Jin Goh. 2005. Hierarchical Identity Based Encryption with Constant Size Ciphertext. In *EUROCRYPT 2005 (LNCS, Vol. 3494)*, Ronald Cramer (Ed.). Springer, Berlin, Heidelberg, 440–456. doi:10.1007/11426639_26

[15] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. 2020. Efficient polynomial commitment schemes for multiple points and polynomials. Cryptology ePrint Archive, Paper 2020/081. https://eprint.iacr.org/2020/081

[16] Dan Boneh, B Fisch, A Gabizon, and Z Williamson. 2020. A simple range proof from polynomial commitments (2020). *URL: https://hackmd.io/@dabo/B1U4kx8XI* (2020).

[17] Dan Boneh, Rohit Nema, Arnab Roy, and Ertem Nusret Tas. 2026. Efficient Batch Threshold Encryption Using Partial Fraction Techniques. Cryptology ePrint Archive, Paper 2026/674. https://eprint.iacr.org/2026/674

[18] Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyan Qu. 2025. BEAT-MEV: Epochless Approach to Batched Threshold Encryption for MEV Prevention. In *USENIX Security 2025*. USENIX Association, 3457–3476. https://www.usenix.org/conference/usenixsecurity25/presentation/bormet

[19] Xavier Boyen. 2008. The Uber-Assumption Family (Invited Talk). In *PAIRING 2008 (LNCS, Vol. 5209)*, Steven D. Galbraith and Kenneth G. Paterson (Eds.). Springer, Berlin, Heidelberg, 39–56. doi:10.1007/978-3-540-85538-5_3

[20] Chris Brzuska, Michael Klooß, and Ivy K. Y. Woo. 2025. Threshold Public-Key Encryption: Definitions, Relations, and CPA-to-CCA Transforms. Cryptology ePrint Archive, Report 2025/1665. https://eprint.iacr.org/2025/1665

[21] Matteo Campanelli, Anca Nitulescu, Carla Ràfols, Alexandros Zacharakis, and Arantxa Zapico. 2022. Linear-Map Vector Commitments and Their Practical Applications. In *ASIACRYPT 2022, Part IV (LNCS, Vol. 13794)*, Shweta Agrawal and Dongdai Lin (Eds.). Springer, Cham, 189–219. doi:10.1007/978-3-031-22972-5_7

[22] Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. 2024. Mempool Privacy via Batched Threshold Encryption: Attacks and Defenses. In *USENIX Security 2024*, Davide Balzarotti and Wenyuan Xu (Eds.). USENIX Association. https://www.usenix.org/conference/usenixsecurity24/presentation/choudhuri

[23] Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. 2025. Practical Mempool Privacy via One-time Setup Batched Threshold Encryption. In *USENIX Security 2025*. USENIX Association, 3477–3495. https://www.usenix.org/conference/usenixsecurity25/presentation/choudhuri

[24] Henri Cohen. 2013. *A course in computational algebraic number theory*. Vol. 138. Springer Science & Business Media.

[25] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. 2020. Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability. In *2020 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 910–927. doi:10.1109/SP40000.2020.00040

[26] Sourav Das, Philippe Camacho, Zhuolun Xiang, Javier Nieto, Benedikt Bünz, and Ling Ren. 2023. Threshold Signatures from Inner Product Argument: Succinct, Weighted, and Multi-threshold. In *ACM CCS 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM Press, 356–370. doi:10.1145/3576915.3623096

[27] Lalita Devadas, Abhishek Jain, Brent Waters, and David J. Wu. 2025. Succinct Witness Encryption for Batch Languages and Applications. Cryptology ePrint Archive, Report 2025/944. https://eprint.iacr.org/2025/944

[28] Nico Döttling, Lucjan Hanzlik, Bernardo Magri, and Stella Wohnig. 2023. McFly: Verifiable Encryption to the Future Made Practical. In *FC 2023, Part I (LNCS, Vol. 13950)*, Foteini Baldimtsi and Christian Cachin (Eds.). Springer, Cham, 252–269. doi:10.1007/978-3-031-47754-6_15

[29] Jesko Dujmovic, Rachit Garg, and Giulio Malavolta. 2024. Time-Lock Puzzles with Efficient Batch Solving. In *EUROCRYPT 2024, Part II (LNCS, Vol. 14652)*, Marc Joye and Gregor Leander (Eds.). Springer, Cham, 311–341. doi:10.1007/978-3-031-58723-8_11

[30] Stefan Dziembowski, Sebastian Faust, and Jannik Luhn. 2024. Shutter Network: Private Transactions from Threshold Cryptography. *IACR Cryptol. ePrint Arch.* (2024), 1981. https://eprint.iacr.org/2024/1981

[31] Anders Elowsson, Justin Florentine, and Julian Ma. 2026. EIP-8184: LUCID Encrypted Mempool. Ethereum Improvement Proposals. https://eips.ethereum.org/EIPS/eip-8184 draft.

[32] Shayan Eskandari, Seyedehmahsa Moosavi, and Jeremy Clark. 2019. Sok: Transparent dishonesty: front-running attacks on blockchain. In *International Conference on Financial Cryptography and Data Security*.

[33] Rex Fernando, Guru-Vamsi Policharla, Andrei Tonkikh, and Zhuolun Xiang. 2025. TrX: Encrypted Mempools in High Performance BFT Protocols. Cryptology ePrint Archive, Report 2025/2032. https://eprint.iacr.org/2025/2032

[34] Flashbots. 2022. MEV-Boost. https://github.com/flashbots/mev-boost.

[35] Danilo Francati, Daniele Friolo, Monosij Maitra, Giulio Malavolta, Ahmadreza Rahimi, and Daniele Venturi. 2023. Registered (Inner-Product) Functional Encryption. In *ASIACRYPT 2023, Part V (LNCS, Vol. 14442)*, Jian Guo and Ron Steinfeld (Eds.). Springer, Singapore, 98–133. doi:10.1007/978-981-99-8733-7_4

[36] Cody Freitag, Brent Waters, and David J. Wu. 2023. How to Use (Plain) Witness Encryption: Registered ABE, Flexible Broadcast, and More. In *CRYPTO 2023, Part IV (LNCS, Vol. 14084)*, Helena Handschuh and Anna Lysyanskaya (Eds.). Springer, Cham, 498–531. doi:10.1007/978-3-031-38551-3_16

[37] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. 2018. The Algebraic Group Model and its Applications. In *CRYPTO 2018, Part II (LNCS, Vol. 10992)*, Hovav Shacham and Alexandra Boldyreva (Eds.). Springer, Cham, 33–62. doi:10.1007/978-3-319-96881-0_2

[38] Georg Fuchsbauer, Antoine Plouviez, and Yannick Seurin. 2020. Blind Schnorr Signatures and Signed ElGamal Encryption in the Algebraic Group Model. In *EUROCRYPT 2020, Part II (LNCS, Vol. 12106)*, Anne Canteaut and Yuval Ishai (Eds.). Springer, Cham, 63–95. doi:10.1007/978-3-030-45724-2_3

[39] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Report 2019/953. https://eprint.iacr.org/2019/953

[40] Rachit Garg, George Lu, Brent Waters, and David J. Wu. 2023. Realizing Flexible Broadcast Encryption: How to Broadcast to a Public-Key Directory. In *ACM CCS 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM Press, 1093–1107. doi:10.1145/3576915.3623168

[41] Sanjam Garg, Craig Gentry, Amit Sahai, and Brent Waters. 2013. Witness encryption and its applications. In *45th ACM STOC*, Dan Boneh, Tim Roughgarden, and Joan Feigenbaum (Eds.). ACM Press, 467–476. doi:10.1145/2488608.2488667

[42] Sanjam Garg, Mohammad Hajiabadi, Mohammad Mahmoody, and Ahmadreza Rahimi. 2018. Registration-Based Encryption: Removing Private-Key Generator from IBE. In *TCC 2018, Part I (LNCS, Vol. 11239)*, Amos Beimel and Stefan Dziembowski (Eds.). Springer, Cham, 689–718. doi:10.1007/978-3-030-03807-6_25

[43] Sanjam Garg, Mohammad Hajiabadi, Mohammad Mahmoody, Ahmadreza Rahimi, and Sruthi Sekar. 2019. Registration-Based Encryption from Standard Assumptions. In *PKC 2019, Part II (LNCS, Vol. 11443)*, Dongdai Lin and Kazue Sako (Eds.). Springer, Cham, 63–93. doi:10.1007/978-3-030-17259-6_3

[44] Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. 2023. hinTS: Threshold Signatures with Silent Setup. Cryptology ePrint Archive, Report 2023/567. https://eprint.iacr.org/2023/567

[45] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. 2024. Threshold Encryption with Silent Setup. In *CRYPTO 2024, Part VII (LNCS, Vol. 14926)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, Cham, 352–386. doi:10.1007/978-3-031-68394-7_12

[46] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. 1999. Secure Distributed Key Generation for Discrete-Log Based Cryptosystems. In *EUROCRYPT'99 (LNCS, Vol. 1592)*, Jacques Stern (Ed.). Springer, Berlin, Heidelberg, 295–310. doi:10.1007/3-540-48910-X_21

[47] Noemi Glaeser, Dimitris Kolonelos, Giulio Malavolta, and Ahmadreza Rahimi. 2023. Efficient Registration-Based Encryption. In *ACM CCS 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM Press, 1065–1079. doi:10.1145/3576915.3616596

[48] Junqing Gong, Brent Waters, Hoeteck Wee, and David J. Wu. 2025. Threshold Batched Identity-Based Encryption from Pairings in the Plain Model. Cryptology ePrint Archive, Report 2025/2103. https://eprint.iacr.org/2025/2103

[49] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *EUROCRYPT 2016, Part II (LNCS, Vol. 9666)*, Marc Fischlin and Jean-Sébastien Coron (Eds.). Springer, Berlin, Heidelberg, 305–326. doi:10.1007/978-3-662-49896-5_11

[50] Jens Groth. 2021. Non-interactive distributed key generation and key resharing. Cryptology ePrint Archive, Report 2021/339. https://eprint.iacr.org/2021/339

[51] Lioba Heimbach and Roger Wattenhofer. 2022. Eliminating Sandwich Attacks with the Help of Game Theory. In *ASIACCS 22*, Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako (Eds.). ACM Press, 153–167. doi:10.1145/3488932.3517390

[52] Susan Hohenberger, George Lu, Brent Waters, and David J. Wu. 2023. Registered Attribute-Based Encryption. In *EUROCRYPT 2023, Part III (LNCS, Vol. 14006)*, Carmit Hazay and Martijn Stam (Eds.). Springer, Cham, 511–542. doi:10.1007/978-3-031-30620-4_17

[53] Aljosha Judmayer, Nicholas Stifter, Philipp Schindler, and Edgar R. Weippl. 2021. Estimating (Miner) Extractable Value is Hard, Let's Go Shopping! *IACR Cryptology ePrint Archive* (2021).

[54] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *ASIACRYPT 2010 (LNCS, Vol. 6477)*, Masayuki Abe (Ed.). Springer, Berlin, Heidelberg, 177–194. doi:10.1007/978-3-642-17373-8_11

[55] Mahimna Kelkar, Soubhik Deb, Sishan Long, Ari Juels, and Sreeram Kannan. 2023. Themis: Fast, Strong Order-Fairness in Byzantine Consensus. In *ACM CCS 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM Press, 475–489. doi:10.1145/3576915.3616658

[56] Mahimna Kelkar, Fan Zhang, Steven Goldfeder, and Ari Juels. 2020. Order-Fairness for Byzantine Consensus. In *CRYPTO 2020, Part III (LNCS, Vol. 12172)*, Daniele Micciancio and Thomas Ristenpart (Eds.). Springer, Cham, 451–480. doi:10.1007/978-3-030-56877-1_16

[57] Dimitris Kolonelos, Giulio Malavolta, and Hoeteck Wee. 2023. Distributed Broadcast Encryption from Bilinear Groups. In *ASIACRYPT 2023, Part V (LNCS, Vol. 14442)*, Jian Guo and Ron Steinfeld (Eds.). Springer, Singapore, 407–441. doi:10.1007/978-981-99-8733-7_13

[58] Kaoru Kurosawa. 2002. Multi-recipient Public-Key Encryption with Shortened Ciphertext. In *PKC 2002 (LNCS, Vol. 2274)*, David Naccache and Pascal Paillier (Eds.). Springer, Berlin, Heidelberg, 48–63. doi:10.1007/3-540-45664-3_4

[59] Klaus Kursawe. 2020. Wendy, the good little fairness widget: Achieving order fairness for blockchains. In *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies*. 25–36.

[60] Jia Liu, Tibor Jager, Saqib A. Kakvi, and Bogdan Warinschi. 2018. How to build time-lock encryption. *DCC* 86, 11 (2018), 2549–2586. doi:10.1007/s10623-018-0461-x

[61] Jannik Luhn. 2025. EIP-8105: Universal Enshrined Encrypted Mempool. Ethereum Improvement Proposals. https://eips.ethereum.org/EIPS/eip-8105 Draft.

[62] Ueli M. Maurer. 2005. Abstract Models of Computation in Cryptography (Invited Paper). In *10th IMA International Conference on Cryptography and Coding (LNCS, Vol. 3796)*, Nigel P. Smart (Ed.). Springer, Berlin, Heidelberg, 1–12. doi:10.1007/11586821_1

[63] Conor McMenamin, Vanesa Daza, Matthias Fitzi, and Padraic O'Donoghue. 2022. FairTraDEX: A Decentralised Exchange Preventing Value Extraction. In *Proceedings of the 2022 ACM CCS Workshop on Decentralized Finance and Security (DeFi'22)*. doi:10.1145/3560832.3563439

[64] Silvio Micali, Leonid Reyzin, Georgios Vlachos, Riad S. Wahby, and Nickolai Zeldovich. 2021. Compact Certificates of Collective Knowledge. In *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 626–641. doi:10.1109/SP40001.2021.00096

[65] Peyman Momeni, Sergey Gorbunov, and Bohan Zhang. 2022. FairBlock: Preventing Blockchain Front-Running with Minimal Overheads. In *SecureComm (Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, Vol. 462)*. Springer, 250–271.

[66] Guru-Vamsi Policharla. 2026. A Simple Batched Threshold Encryption Scheme. Cryptology ePrint Archive, Paper 2026/760. https://eprint.iacr.org/2026/760

[67] John M Pollard. 1978. Monte Carlo methods for index computation (mod p). *Mathematics of computation* 32, 143 (1978), 918–924.

[68] Kaihua Qin, Liyi Zhou, and Arthur Gervais. 2021. Quantifying Blockchain Extractable Value: How dark is the forest? *arXiv preprint* (2021).

[69] Carla Ràfols and Arantxa Zapico. 2021. An Algebraic Framework for Universal and Updatable SNARKs. In *CRYPTO 2021, Part I (LNCS, Vol. 12825)*, Tal Malkin and Chris Peikert (Eds.). Springer, Cham, Virtual Event, 774–804. doi:10.1007/978-3-030-84242-0_27

[70] Hany Ragab, Alyssa Milburn, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2021. CrossTalk: Speculative Data Leaks Across Cores Are Real. In *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 1852–1867. doi:10.1109/SP40001.2021.00020

[71] Leonid Reyzin, Adam Smith, and Sophia Yakoubov. 2021. Turning HATE into LOVE: Compact Homomorphic Ad Hoc Threshold Encryption for Scalable MPC. In *Cyber Security Cryptography and Machine Learning*.

[72] Victor Shoup. 1997. Lower Bounds for Discrete Logarithms and Related Problems. In *EUROCRYPT'97 (LNCS, Vol. 1233)*, Walter Fumy (Ed.). Springer, Berlin, Heidelberg, 256–266. doi:10.1007/3-540-69053-0_18

[73] Shutter Network contributors. 2021. The Shutter Network. https://shutter.network.

[74] Chrysoula Stathakopoulou, Signe Rüsch, Marcus Brandenburger, and Marko Vukolić. 2021. Adding fairness to order: Preventing front-running attacks in bft protocols using tees. In *2021 40th International Symposium on Reliable Distributed Systems (SRDS)*. 34–45.

[75] Sora Suegami, Shinsaku Ashizawa, and Kyohei Shibano. 2024. Constant-Cost Batched Partial Decryption in Threshold Encryption. Cryptology ePrint Archive, Paper 2024/762. https://eprint.iacr.org/2024/762

[76] Edlyn Teske. 1998. *Speeding up Pollard's rho method for computing discrete logarithms*. Springer.

[77] Christof Ferreira Torres, Ramiro Camino, et al. 2021. Frontrunner Jones and the Raiders of the Dark Forest: An Empirical Study of Frontrunning on the Ethereum Blockchain. In *30th USENIX Security Symposium*.

[78] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. 2018. Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution. In *USENIX Security 2018*, William Enck and Adrienne Porter Felt (Eds.). USENIX Association, 991–1008. https://www.usenix.org/conference/usenixsecurity18/presentation/bulck

[79] Ye Wang, Yan Chen, Shuiguang Deng, and Roger Wattenhofer. 2021. Cyclic Arbitrage in Decentralized Exchange Markets. *Available at SSRN 3834535* (2021).

[80] Brent Waters and David J. Wu. 2025. Silent Threshold Cryptography from Pairings: Expressive Policies in the Plain Model. Cryptology ePrint Archive, Report 2025/1547. https://eprint.iacr.org/2025/1547
[81] Haoqian Zhang, Louis-Henri Merino, Ziyan Qu, Mahsa Bastankhah, Vero Estrada-Galiñanes, and Bryan Ford. 2023. F3B: A Low-Overhead Blockchain Architecture with Per-Transaction Front-Running Protection. In *5th Conference on Advances in Financial Technologies (AFT 2023)*. doi:10.4230/LIPIcs.AFT.2023.3
[82] Liyi Zhou, Kaihua Qin, Antoine Cully, Benjamin Livshits, and Arthur Gervais. 2021. On the just-in-time discovery of profit-generating transactions in defi protocols. In *2021 IEEE Symposium on Security and Privacy (SP)*.
[83] Liyi Zhou, Kaihua Qin, Christof Ferreira Torres, Duc V Le, and Arthur Gervais. 2021. High-frequency trading on decentralized on-chain exchanges. In *2021 IEEE Symposium on Security and Privacy (SP)*.
[84] Ziqi Zhu, Jiangtao Li, Kai Zhang, Junqing Gong, and Haifeng Qian. 2024. Registered Functional Encryptions from Pairings. In *EUROCRYPT 2024, Part II (LNCS, Vol. 14652)*, Marc Joye and Gregor Leander (Eds.). Springer, Cham, 373–402. doi:10.1007/978-3-031-58723-8_13
[85] Ziqi Zhu, Kai Zhang, Junqing Gong, and Haifeng Qian. 2023. Registered ABE via Predicate Encodings. Cryptology ePrint Archive, Report 2023/1383. https://eprint.iacr.org/2023/1383

Open Science

To facilitate reproducible research and enable verification of our results, we provide a public artifact accompanying this submission. The artifact consists of (i) an implementation of our silent batched threshold encryption scheme, including the code used for our experiments and benchmarks, as well as (ii) the raw benchmark results underpinning the evaluation in Section 8. We ensure the persistent availability of the artifact through the following repository:

Artifact Repository: https://github.com/guruvamsi-policharla/batched-silent-threshold-encryption

For documentation and instructions on how to reproduce our results, please consider the `README.md` file contained within the repository.

Ethical Considerations

This work involves no human subjects, personal data, or vulnerability analysis, and does not introduce attack techniques. It enhances the privacy of user transactions in blockchain. Accordingly, we identify no specific ethical risks requiring additional mitigation.

A Additional Related Work

Mempool Privacy: MEV poses a significant challenge in blockchain systems. A promising mitigation strategy involves achieving mempool privacy, where threshold encryption has emerged as the de facto approach.

In constructions based on traditional threshold encryption [8, 81], every transaction needs to be decrypted individually, which introduces a heavy $O(nB)$ communication overhead for n committee members and B ciphertexts. Subsequent works [28, 65, 73] employ identity-based encryption (IBE) to improve efficiency. Users now encrypt their transactions to a future block identity, and all transactions encrypted to the same block get decrypted once. This reduces decryption communication to $O(n)$. However, such approach compromises pending transaction privacy. As detailed earlier, batched threshold encryption [3, 18, 22, 23, 75] resolves the limitation efficiently.

Other primitives, like time-lock puzzles [12, 60], indistinguishability obfuscation (iO) [6], or general-purpose witness encryption [27, 41], can also be used to achieve mempool privacy. However, limitations like heavy computational overhead or reliance on strong cryptographic assumptions make these schemes less preferred for blockchain applications. Designing practical constructions under these primitives remains an open research question.

We note here work concurrent with or subsequent to ours that targets objectives orthogonal to those considered in this paper:

- **Weighted Batched Threshold Encryption:** [1] extend BEAT-MEV to support weighted committees, while keeping server communication independent of the committee weights.
- **Censorship-Resistance Epochless Batched Threshold Encryption:** A recent line of work [2, 17, 33, 66] studies epochless BTE schemes that avoid index-collision attacks. Such attacks arise in schemes where users encrypt to batch indices and at most one ciphertext per index can be decrypted in a block: an adversary can copy the index of an honest ciphertext and thereby prevent it from being decrypted, leading to the potential for censorship. Epoch-based schemes [3, 48] avoid this particular issue because ciphertexts are tied to a block or epoch, but this comes at the cost of losing the epochless property.

Alternative MEV Mitigation Strategies: Beyond cryptographic solutions for mempool privacy, several alternative approaches address MEV. In Ethereum, Flashbots and MEV-boost are used to regulate MEV extraction through auctions [25, 34]. While reducing its negative effects, they do not eliminate the problem and introduce centralization risks. Solutions based on Trusted-Execution Environments (TEEs) are also explored to solve MEV [10, 74]. The secret key of a public key encryption scheme is stored in secure enclaves, and the TEE will decrypt and publish the blocks of transactions encrypted to the corresponding public key. They inherit hardware trust assumptions and remain vulnerable to side-channel attacks [70, 78], which creates risks and makes them impractical solutions. Another line of work approaches the MEV problem by achieving fair ordering [55, 56, 59]. They enforce consensus over all miners on the transaction ordering, thus limiting any reordering by a single miner. However, only weakened order fairness is achieved so far, which does not eliminate MEV entirely. MEV-resistant DEXs such as [51, 63] employ game theoretic mechanisms to prevent MEV. These works are mostly complex and specific to certain applications, and cannot be used as a general solution. Motivated by these limitations, we build a mempool encryption scheme with the desired properties that practically mitigates MEV.

Silent Setup for Threshold Cryptography: DKG remains a persistent bottleneck in deploying threshold cryptosystems despite decades of research [46, 50], imposing substantial communication and computation overhead.

Threshold signatures are an essential primitive for decentralized systems like Ethereum, and similar to threshold decryption, they commonly rely on DKG protocols for key setup. Some prior works have attempted to remove the need for DKG by using multisignature [7, 11], which can be made to support non-interactive t-out-of-n key aggregation by having the signature aggregator include all signers, and letting the verifier aggregate the public keys of the desired t signers. This incurs linear signature size and verification costs, which are expensive for blockchain deployment. An

intuitive way to improve the overhead is to let the aggregator also compute the public key aggregation. A succinct non-interactive argument (SNARK) is required to prove the correctness of this aggregated public key. Since a simple plug-in of generic SNARKs [39, 49] introduces high aggregation overhead, recent works [26, 44, 64] design custom SNARK algorithms to achieve concretely efficient aggregation time.

Beyond threshold signatures, there has been a line of research on silent setup for other advanced encryption schemes, including distributed/flexible broadcast encryption [36, 40, 57], registration-based encryption [42, 43, 47], registered attribute-based encryption [52, 85] and registered functional encryption [35, 84]. Recently, Garg et al. [45] present the first efficient construction of threshold encryption with silent setup, which inspires this work. Alternatively, other works attempt to construct threshold encryption with silent setup using iO [27, 71].

Critically, no prior work achieves silent setup for batched threshold encryption, which we believe to be a promising primitive for scalable, privacy-preserving mempools. Our work bridges this gap by defining and constructing the first concretely practical Silent Batched Threshold Encryption (SBTE) scheme.

B Additional Preliminaries

In the following, we present some theorems, lemmas, and definitions for cryptographic building blocks we use in our constructions.

B.1 Master Theorem

Theorem 11 (Master Theorem [14, 19]). *Let $p \in \mathbb{N}$ be a prime, let $n, v_1, v_2, v_t \in \mathbb{N}$, let $\mathbf{L}_1 \in \mathbb{F}_p[X_1, \ldots, X_n]^{v_1}$, $\mathbf{L}_2 \in \mathbb{F}_p[X_1, \ldots, X_n]^{v_2}$, $\mathbf{L}_T \in \mathbb{F}_p[X_1, \ldots X_n]^{v_t}$, and $f \in \mathbb{F}_p[X_1, \ldots, X_n]$ be tuples of n-variate polynomials of maximum degree $d_{\mathbf{L}_1}, d_{\mathbf{L}_2}, d_{\mathbf{L}_T}$, and d_f respectively. Denote $d = \max\left(d_{\mathbf{L}_1} + d_{\mathbf{L}_2}, d_{\mathbf{L}_T}, d_f\right)$, and $v = v_1 + v_2 + v_t$. If f is independent of $(\mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_T)$, then for any generic adversary $\mathcal{A}$ that makes at most q group oracle queries:*

$$\left| \Pr\left[\mathcal{A}\begin{pmatrix} p, \mathsf{h}_1[\mathbf{L}_1(x)], \mathsf{h}_2[\mathbf{L}_2(x)], \\ \mathsf{h}_T[\mathbf{L}_T(x), f(x)] \end{pmatrix} = 1\right] - \Pr\left[\mathcal{A}\begin{pmatrix} p, \mathsf{h}_1[\mathbf{L}_1(x)], \mathsf{h}_2[\mathbf{L}_2(x)], \\ \mathsf{h}_T[\mathbf{L}_T(x), r] \end{pmatrix} = 1\right] \right| \leq \frac{(q+v+2)^2 \cdot d}{2p},$$

where $\mathsf{h}_1, \mathsf{h}_2$, and h_T denote the corresponding handles, and the probabilities are taken over the choice of $x \leftarrow\!\!\$\ \mathbb{F}_p^n$ and $r \leftarrow\!\!\$\ \mathbb{F}_p$.

Definition 12 (Independence). Let the completion $\mathbf{C}(\mathbf{L}) = \{\mathbf{L}_1 \otimes \mathbf{L}_2\} \cup \mathbf{L}_T$ of size D for lists $(\mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_T)$ be of the form

$$\mathbf{C}(\mathbf{L}) = \{g_1(X_1, \ldots, X_n), \ldots g_D(X_1, \ldots, X_n)\}.$$

We say that an element f is dependent on lists $(\mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_T)$, if there exist coefficients $\kappa_i \in \mathbb{F}_p$ such that

$$f(X_1, \ldots, X_n) = \sum_{i=1}^{D} \kappa_i \cdot g_i(X_1, \ldots, X_n).$$

Otherwise, we say that f is independent of $(\mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_T)$.

B.2 Commitment Schemes

Definition 13 (Commitment Schemes). A commitment scheme Σ for message space $\mathcal{M}$ and randomness space $\mathcal{R}$ is a tuple of algorithms $\Sigma = (\mathsf{Setup}, \mathsf{Commit}, \mathsf{Vfy})$ such that:

- $\mathsf{pp} \leftarrow\!\!\$\ \mathsf{Setup}(1^\lambda)$. Given the security parameter λ, Setup generates public parameters pp that are implicit input to all subsequent algorithms.
- $\mathsf{com} \leftarrow \mathsf{Commit}(m; r)$. Given a message m and randomness r, the Commit algorithm generates a commitment com.
- $1/0 \leftarrow \mathsf{Vfy}(\mathsf{com}, m, r)$. The verification algorithm returns 1 if the decommitment r opens the commitment com to message m and 0 otherwise.

A commitment scheme must satisfy the following properties:

- **Correctness.** For all messages $m \in \mathcal{M}$ and randomness $r \in \mathcal{R}$ it holds that $\mathsf{Vfy}(\mathsf{Commit}(m; r), m, r) = 1$.
- **Perfect Hiding.** For all unbounded adversaries $\mathcal{A}$, all $\lambda \in \mathbb{N}$, all $\mathsf{pp} \in \mathsf{Setup}(1^\lambda)$ and all messages $m_0, m_1 \in \mathcal{M}$ of equal length, it holds that

$$\Pr[\mathcal{A}(\mathsf{pp}, m_0, m_1, \mathsf{Commit}(m_0)) = 1] = \Pr[\mathcal{A}(\mathsf{pp}, m_0, m_1, \mathsf{Commit}(m_1)) = 1].$$

- **Computational Binding.** For all PPT adversaries $\mathcal{A}$ there exists a negligible function negl such that

$$\Pr\begin{bmatrix} m \neq m' \wedge & \mathsf{pp} \leftarrow\!\!\$\ \mathsf{Setup}(1^\lambda) \\ \mathsf{Vfy}(\mathsf{com}, m, r) & : (\mathsf{com}, m, r, m', r') \\ \wedge \mathsf{Vfy}(\mathsf{com}, m', r') & \leftarrow\!\!\$\ \mathcal{A}(1^\lambda, \mathsf{pp}) \end{bmatrix} \leq \mathsf{negl}(\lambda).$$

B.3 NIZK PoKs

Definition 14 (NIZK-PoK). Let R be an NP-relation of statement-witness pairs $(\chi, \omega) \in \mathsf{R}$ a non-interactive zero-knowledge proof system of knowledge PS_R for R is a tuple of algorithms $\mathsf{PS}_\mathsf{R} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Vfy})$ with the following syntax:

- $\mathsf{crs} \leftarrow\!\!\$\ \mathsf{Setup}(1^\lambda)$. Given the security parameter λ, the Setup algorithm outputs a common reference string crs. The crs is implicit input to all subsequent algorithms.
- $\pi \leftarrow\!\!\$\ \mathsf{Prove}(\chi, \omega)$. For a statement-witness pair (χ, ω), the Prove algorithm returns a proof π.
- $1/0 \leftarrow \mathsf{Vfy}(\chi, \pi)$. Given a statement χ and a proof π, the Vfy algorithm outputs 1, iff π is valid for χ.

A proof system PS_R must fulfill *correctness*: For all $(\chi, \omega) \in \mathsf{R}$ it holds that $\mathsf{Vfy}(\chi, \mathsf{Prove}(\chi, \omega)) = 1$.

Additionally, we require our proof systems to satisfy *simulation extractability*, following the definition of (weak) simulation extractability of [20].

Definition 15 (Weak Simulation Extractability). A proof system PS_R for R is weakly simulation-extractable if there exists a PPT simulator $\mathsf{Sim} = (\mathsf{Setup}, \mathsf{Sim}, \mathsf{Ext})$ such that for all PPT adversaries $\mathcal{A}$ there exists a negligible function negl such that

$$\Pr\left[\mathsf{Game\text{-}SimExt}_{\mathcal{A}}^{\mathsf{PS}_\mathsf{R}, \mathsf{Sim}}(1^\lambda) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda),$$

where Game-SimExt is defined in Figure 7.

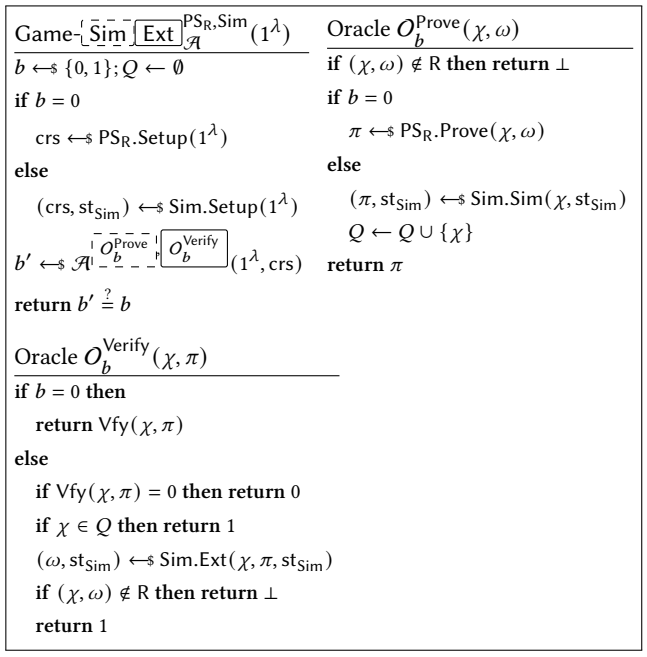


Figure 7: The zero-knowledge game Game-Sim**, the knowledge-soundness game** Game-Ext **and the simulation-extractability game** Game-SimExt **for NIZK-PoKs. Code in dashed boxes highlights components present only in** Game-Sim **and** Game-SimExt**, and code in solid boxes highlights the components present only in** Game-Ext **and** Game-SimExt**.**

We also define zero-knowledge and knowledge-soundness separately. Clearly, both zero-knowledge and knowledge soundness are implied by (weak) simulation extractability.

Definition 16 (Zero-Knowledge). A proof system $\mathsf{PS_R}$ for R is zero-knowledge if there exists a PPT simulator $\mathsf{Sim} = (\mathsf{Setup}, \mathsf{Sim})$ such that for all PPT adversaries $\mathcal{A}$ there exists a negligible function negl such that

$$\Pr\left[\text{Game-Sim}^{\mathsf{PS_R},\mathsf{Sim}}_{\mathcal{A}}(1^\lambda) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda),$$

where Game-Sim is defined in Figure 7.

Definition 17 (Knowledge-Soundness). A proof system $\mathsf{PS_R}$ for R is knowledge-sound if there exists a PPT simulator $\mathsf{Sim} = (\mathsf{Setup}, \mathsf{Ext})$ such that for all PPT adversaries $\mathcal{A}$ there exists a negligible function negl such that

$$\Pr\left[\text{Game-Ext}^{\mathsf{PS_R},\mathsf{Sim}}_{\mathcal{A}}(1^\lambda) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda),$$

where Game-Ext is defined in Figure 7.

B.4 Key-Homomorphic Puncturable PRF

Definition 18 (Pseudorandomness of PRF*).* A puncturable PRF is pseudorandom if for all PPT adversaries $\mathcal{A}$, there exists a negligible function negl such that

$$\Pr[\text{Game-Pr}^{\mathsf{PRF}}_{\mathcal{A}}(1^\lambda) = 1] \leq \frac{1}{2} + \mathsf{negl}(\lambda),$$

where Game-Pr is defined in Figure 8.

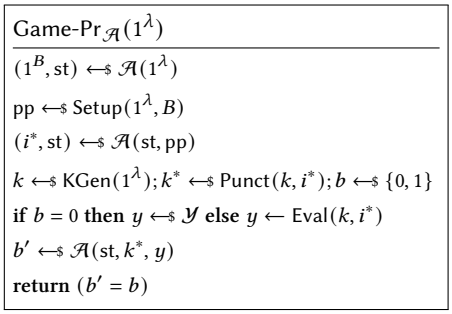


Figure 8: Pseudorandomness game of PRF **where $\mathcal{Y}$ denotes the evaluation space of the** PRF**.**

B.5 Deferred Definitions of Silent (Batched) Threshold Encryption Schemes

Definition 19 (Correctness). An SBTE scheme Σ is correct, if

$$\forall i \in C\colon \mathsf{KVfy}(\mathsf{pk}_i) = 1$$
$$\forall j \in [B]\colon \mathsf{Vfy}(\mathsf{ek}, \mathsf{ct}_j, \pi^{\mathsf{ct}}_j) = 1$$
$$\forall i \in S\colon \mathsf{PDVfy}(\mathsf{pd}_i, \{\mathsf{ct}_j\}_{j\in[B]}, \mathsf{pk}_i, \pi^{\mathsf{pd}}_i) = 1$$
$$\mathsf{Comb}(\mathsf{ak}, \{\mathsf{pd}_i\}_{i\in S}, \{\mathsf{ct}_j\}_{j\in[B]}) = \{m_j\}_{j\in[B]},$$

for all $\lambda, B, t, B_{\max}, N \in \mathbb{N}$, $B \leq B_{\max}$, $C \subseteq [N]$, and $S \subseteq C$ with $t \leq |C|$, and $|S| \geq t$, and for all messages $\{m_j\}_{j\in[B]}$, and all

- $\mathsf{pp} \in \mathsf{Setup}(1^\lambda, B_{\max}, N)$
- $\{(\mathsf{pk}_i, \mathsf{sk}_i) \in \mathsf{KGen}(\mathsf{pp})\}_{i\in C}$
- $(\mathsf{ek}, \mathsf{ak}) \in \mathsf{Prep}(C, \{\mathsf{pk}_i\}_{i\in C})$
- $\{(\mathsf{ct}_j, \pi^{\mathsf{ct}}_j) \in \mathsf{Enc}(\mathsf{ek}, m_j, j, t)\}_{j\in[B]}$
- $\{(\mathsf{pd}_i, \pi^{\mathsf{pd}}_i) \in \mathsf{BatchDec}(\mathsf{sk}_i, \{\mathsf{ct}_j\}_{j\in[B]})\}_{i\in S}$.

For completeness, we recall the semantic security definition for silent setup encryption (without batching), following [45].

*Definition 20 (*S-IND-CPA*-security).* We say that an STE scheme Σ is S-IND-CPA-secure if for all PPT-adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ there exists a negligible function negl such that

$$\Pr[\text{Game-S-IND-CPA}^{\Sigma}_{\mathcal{A}}(1^\lambda) = 1] \leq \frac{1}{2} + \mathsf{negl}(\lambda),$$

where Game-S-IND-CPA is defined in Figure 1.

C NIZK Proofs for Rogue Ciphertext and CCA Security

In our construction, we make use of NIZK proofs to ensure both rogue ciphertext security and CCA security. We next give more details.

NIZK proofs for Rogue Ciphertext Security. Rogue ciphertext security ensures that an encryption of a message m always decrypts

to the same message m, even if all other ciphertexts in the batch and all committee members are controlled by a malicious adversary (Definition 7). There are four possible attack vectors through which an adversary could launch a rogue ciphertext attack against our construction. We address and mitigate each of these using tailored NIZK proofs and verification techniques.

Injecting malformed public keys. A malicious adversary could publish malformed public keys during the silent setup phase, causing the aggregation step of the decryption to return an incorrect message. Observe that, similar to [45], our public keys are verifiable as is, i.e., we do not need to introduce a special proof system. We can verify that a public key is sound by checking that $\mathsf{pk}_{u,j} \circ [1]_2 = \mathsf{pk}_{1,0} \circ [r_u \tau^j]_2$, for all $u \in [\ell], j \in [0, n-1]$ without sending a NIZK proof.

Issuing malformed partial decryptions. Malicious committee members could also attempt to issue malformed partial decryptions in order to manipulate the final decrypted message. We mitigate this attack by requiring committee members to prove the correctness of their partial decryption using a NIZK proof system, denoted $\mathsf{PS}_{\mathsf{pd}}$. The proof system is defined over statements $\chi \coloneqq (\mathsf{crs}, \mathsf{pk}, \{\mathsf{ct}_i\}_{i\in[B]}, \mathsf{pd})$ and witnesses $\omega \coloneqq \mathsf{sk}$, with the following relation: $\mathsf{R}_{\mathsf{pd}} = \{(\chi, \omega) \colon [\mathsf{sk}]_1 = \mathsf{pk}_{1,0} \wedge \mathsf{pd} = \mathsf{sk} \cdot \sum_{i\in[B]} \mathsf{ct}_i.\mathsf{ct}_6\}$. This proof system can be similarly realized using a simple Schnorr proof. The proof size is 2 field elements, i.e., 64 bytes.

Inducing overflow in chunking. Recall that in our construction, the PRF key is decomposed into ℓ chunks of size $z_1 = \lceil \lambda_k / \ell \rceil$ bits, such that $k = \sum_{j\in[\ell]} k_j \cdot 2^{(j-1)z_1}$. We then encrypt the scalar vector $(k_1, \ldots, k_\ell)$ in Σ_{MSTE}. During decryption, we sum up each chunk across all ciphertexts in a batch and learn $[r_j K_j]_T$ through STE decryption, where $K_j \coloneqq \sum_{i\in[B]} k_j^{(i)}$ and $k_j^{(i)}$ denotes chunk j of ciphertext i. Then, we run an algorithm to extract each K_j from $[r_j K_j]_T$ with base $[r_j]_T$, using the range $K_j \in [0, 2^{z_1 + \log(B)} - 1]$.

Critically, an adversary could inject a ciphertext, where it samples a chunk key k_j outside of the permitted range. As a result, the discrete logarithm for the aggregated chunk key across all ciphertexts in the batch K_j could be outside the permitted range causing the employed discrete logarithm algorithm to either fail entirely or not terminate in polynomial time.

Hence, we introduce range proof $\mathsf{PS}_{\mathsf{range}}$, forcing the encryptor to prove that each k_j is within the range $[0, 2^{z_1} - 1]$. For this, we can use an efficient range proof technique [15, 16], which requires us to add Pedersen commitments $\{\mathsf{com}_j\}_{j\in[\ell]}$ for each k_j to the ciphertext. The commitments and range proof add $\ell \cdot 6 \cdot \mathbb{G}_1$ elements to the ciphertext, which amounts to 4.5 KiB for $\ell = 16$. Note that we also need to prove that the $\{k_j\}_{j\in[\ell]}$ hidden within com_j are the same chunk keys that are encrypted within Σ_{MSTE}. We explain this in the next step.

Mismatching punctured keys and STE ciphertexts. An adversary could attempt the same attack described in [18], where mismatched keys are used between the punctured PRF key and the one encrypted within the ciphertext. Hence, we also need to prove that the chunking $\{k_j\}_{j\in[\ell]}$ that is (validly) encrypted in Σ_{MSTE}, is actually a valid decomposition of a key k matching the punctured key k^*. Additionally, to tie the range proofs to the ciphertext, we must also prove that the same chunk keys are hidden in the Pedersen commitments $\{\mathsf{com}_j\}_{j\in[\ell]}$.

To enforce this, we introduce a NIZK proof system $\mathsf{PS}_{\mathsf{cca}}$ for the relation $\mathsf{R}_{\mathsf{cca}}$ defined over pairs of statements $\chi = (\mathsf{crs}, \{\mathsf{pk}_i\}_{i\in[n]}, \mathsf{ct} \coloneqq (i, t, k^*, \beta, \mathsf{ct}_{\mathsf{MSTE}}, \{\mathsf{com}_j\}_{j\in[\ell]}, \pi^{\mathsf{range}}))$[12] and witnesses $\omega \coloneqq (\{k_j\}_{j\in[\ell]}, \vec{s}, \{r_j^{\mathsf{com}}\}_{j\in[\ell]})$, such that

$$\mathsf{R}_{\mathsf{cca}} = \Bigg\{ (\chi, \omega) \colon k^* = \mathsf{PRF.Punct}\Big(\sum_{j\in[\ell]} k_j \cdot 2^{(j-1)\cdot z_1}, i \Big)$$
$$\wedge\ \mathsf{ct}_{\mathsf{MSTE}} = \Sigma_{\mathsf{MSTE}}.\mathsf{Enc}\left(\{\mathsf{pk}_i\}_{i\in[n]}, \{k_j\}_{j\in[\ell]}, t; \vec{s} \right)$$
$$\wedge\ \forall j \in [\ell] \colon \mathsf{com}_j = \mathsf{Ped.Commit}(k_j, r_j^{\mathsf{com}}) \Bigg\}.$$

We instantiate $\mathsf{PS}_{\mathsf{cca}}$ with a Schnorr proof with a proof size of $2\ell + 6$ field elements, i.e., 1.19 KiB. Together, π^{range} and π^{cca} constitute the ciphertext proof π^{ct}.

CCA Security. Finally, observe that the proof system $\mathsf{PS}_{\mathsf{cca}}$ is a proof of knowledge, which is straight-line simulation-extractable in the algebraic group model (AGM) [37, 38]. As a result, we can extract s_3 from its witness and use it to simulate the partial decryption oracle in the CCA security proof, ensuring CCA security for our construction.

D Security

D.1 Semantic Security of Σ_{MSTE}

Proof of Lemma 9. Our proof follows the technique from [45], but accounts for the fact that we encrypt ℓ chunks instead of a single message. To do so, we prove in the GGM [62, 72] that the following two games are computationally indistinguishable.

Note that in our proof we consider a variant of our construction, where we generate a random generator $[r_j]_{1,2}$ for *all* chunks $j \in [\ell]$ instead of setting $r_1 = 1$. This analysis carries over to the Σ_{MSTE} construction detailed in Section 6.4.

$\underline{\mathsf{Game}_0}$: Game_0 is equivalent to the Game-S-IND-CPA security game (Figure 1), instantiated with Σ_{MSTE}. In Game_0, the challenger first samples $\tau, \gamma, r_1, \ldots, r_\ell \leftarrow\!\$\ \mathbb{F}^{\ell+2}$ and computes the crs as $\{[r_m]_{1,2}, [r_m\gamma]_2, [r_m\tau]_{1,2}, \ldots, [r_m\tau^n]_{1,2}\}_{m\in[\ell]}$. The challenger sends the corresponding handles to the adversary $\mathcal{A}$, who responds with a universe $C \subseteq [N]$, a set of corrupted parties $\mathsf{Cor} \subseteq C$, and a threshold $t > |\mathsf{Cor}|$. Then, the challenger computes $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow\!\$\ \mathsf{KGen}(\mathsf{pp})$ for all $i \in \mathcal{H}$, where $\mathcal{H} = C \setminus \mathsf{Cor}$ and sends the public keys $\{\mathsf{pk}_i\}_{i\in\mathcal{H}}$ to $\mathcal{A}$. We define $\mathcal{H}^* = \mathcal{H} \cup \{0\}$. To compute the public key of the corrupted parties, the adversary needs to query the GGM oracle with all $\{sk_i\}_{i\in\mathsf{Cor}}$, receiving handles to the corresponding public keys in return. Then, $\mathcal{A}$ sends $m^{(0)}$ and $m^{(1)}$. The challenger samples a random bit b and computes $\mathsf{ct}^* \leftarrow\!\$\ \mathsf{Enc}(\mathsf{pp}, \{\mathsf{pk}_i\}_{i\in C}, t, m_b)$, sending the handles to elements in ct^* to $\mathcal{A}$. The view of the adversary now consists of the tuple $(N, t, C, \{\mathsf{sk}_i\}_{i\in\mathsf{Cor}})$ and handles to $(\mathsf{crs}, \{\mathsf{pk}_i\}_{i\in C}, \mathsf{ct}^*)$. Finally, the adversary returns a bit b' and wins whenever $b = b'$. Throughout the course of the game, $\mathcal{A}$ can query the challenger to make arbitrary generic group operations.

[12] We include the range proof within the statements to avoid ciphertext malleability and ensure CCA security.

Game$_1$: In Game$_1$, we modify how $\mathsf{ct}_{9,i}$ is computed in the challenge ciphertext. Instead of padding the message $[r_i m_i^{(b)}]_T$ with $[r_i s_5]_T$ for all $i \in [\ell]$, the challenger samples uniformly random $f_i \leftarrow\!\!\$\ \mathbb{Z}_p$ and sets $\mathsf{ct}_{9,i} = [v_i \cdot (f_i + m_i^{(b)} r_i)]_1$, effectively replacing $r_i s_5$ with f_i.

Claim 1. *It holds that* Game$_0 \approx_c$ Game$_1$ *in the generic group model.*

Proof of Claim 1. For a matrix A, we denote with $A^{[\ell]}$ the matrix that results from repeating each column of A a total of ℓ times. Hence, if A has dimensions $u \times v$, then $A^{[\ell]}$ has dimensions $u \times \ell v$. We denote as $\mathbf{a}_i$ the i-th column of A and as $\mathbf{b}_i$ the i-th column of B. Further, we write $B_{\neq j} = (\mathbf{b}_1, \ldots, \mathbf{b}_{j-1}, \mathbf{b}_{j+1}, \ldots, \mathbf{b}_\ell)$ to denote the matrix B without column j. Given two matrices A and B, we write $A|B$ to denote concatenation of the columns of A and B. We denote the formal variables as $\mathbf{K} = \{K_i\}_{i\in\mathcal{H}}$, $\mathbf{S} = (S_1, \ldots, S_5)$, $\mathbf{R} = \{R_i\}_{i\in[\ell]}$, $\mathbf{V} = \{V_i\}_{i\in[\ell]}$, and $\mathbf{Y} = (X, \mathbf{K}, \Gamma, \mathbf{S}, \mathbf{R}, \mathbf{V})$, corresponding to the secret elements τ, $\{\mathsf{sk}_i\}_{i\in\mathcal{H}}$, γ, $\mathbf{s}$, $\{r_i\}_{i\in[\ell]}$, and $\{v_i\}_{i\in[\ell]}$.

Inspecting the view of the adversary, we define the following lists of polynomials over the handles known to the adversary. For ease of notation, we henceforth denote $R_0 \coloneqq 1$.

$$\mathbf{L}_1(\mathbf{Y}) = \Bigg(\overbrace{\{R_m, R_m X, \ldots, R_m X^N\}}^{\mathsf{crs}}{}_{m\in[0,\ell]},$$
$$\overbrace{\{R_m K_i, R_m K_i X, \ldots, R_m K_i X^{N-1}\}}^{\{\mathsf{pk}_i\}_{i\in\mathcal{H}}}{}_{i\in\mathcal{H}, m\in[0,\ell]},$$
$$\overbrace{S_1\left(\sum_{i\in\mathcal{H}} K_i L_i(X) + \sum_{i\in\mathsf{Cor}} \mathsf{sk}_i L_i(X)\right) + S_4 X^t + S_5}^{\mathbf{S}^\intercal \mathbf{a}_1}, \underbrace{S_3}_{\mathbf{S}^\intercal \mathbf{a}_6}, \underbrace{S_5 X}_{\mathbf{S}^\intercal \mathbf{a}_8},$$
$$\{V_m R_m\}_{m\in[\ell]}, [V_m R_m S_5]_{m\in[\ell]} \Bigg)$$

$$\mathbf{L}_2(\mathbf{Y}) = \Bigg(\underbrace{\{R_m, R_m\Gamma, R_m X, \ldots, R_m X^N\}_{m\in[0,\ell]}}_{\mathsf{crs}}, \underbrace{S_1 + S_3\Gamma}_{\mathbf{S}^\intercal \mathbf{a}_2}, \underbrace{S_1(X^N - 1)}_{\mathbf{S}^\intercal \mathbf{a}_3},$$
$$\underbrace{XS_1 + X^2 S_2}_{\mathbf{S}^\intercal \mathbf{a}_4}, \underbrace{S_2}_{\mathbf{S}^\intercal \mathbf{a}_5}, \underbrace{S_4}_{\mathbf{S}^\intercal \mathbf{a}_7}, \{V_m^{-1}\}_{m\in[\ell]} \Bigg)$$

$$\mathbf{L}_T(\mathbf{Y}) = (1)$$
$$f_j(\mathbf{Y}) = R_j S_5 \quad \forall j \in [\ell]$$

Ultimately, we want to show that for all $j \in [\ell]$, it holds that $f_j(\mathbf{Y})$ is independent of $F_j = (\mathbf{C}(\mathbf{L}) \setminus \{V_j R_j S_5 \cdot V_j^{-1}\}) \cup \{R_i S_5\}_{i\in[\ell], i\neq j}$,[13] where $\mathbf{C}$ refers to the completion. Henceforth, we denote $D = |F_j|$

To do so, we first show that, if f_j depends on F_j, then there exists a vector of polynomials $\mathbf{g} \in (\mathbb{Z}_p[X, \mathbf{K}, \Gamma, \mathbf{R}])^{8\ell+\ell-1}$ that does not depend on $\mathbf{S}$ and $\mathbf{V}$ such that $\mathbf{S}^\intercal (A^{[\ell]}|B_{\neq j}) \cdot \mathbf{g} = \mathbf{S}^\intercal \mathbf{b}_j$. Since $f_j(\mathbf{Y}) = R_j S_5$, it can only be dependent on F_j if there exists a linear combination $R_j S_5 = \sum_{i\in[D]} \kappa_i q_i(\mathbf{Y})$, where q_i are the elements of $F_j = \bigcup q_i$. We divide F_j into three types:

(1) Terms that do not involve $\mathbf{S}$. E.g., $(K_1 X^{N-1}) \cdot X$.
(2) Terms that involve $\mathbf{S}$ on both sides. E.g., $S_3 \cdot (S_1 + S_3\Gamma)$.
(3) Terms that involve $\mathbf{S}$ on exactly one side. E.g., $S_3 \cdot X$.

Since the first two types of elements can not symbolically play a role in the linear combination $R_j S_5 = \sum_{i\in[D]} \kappa_i q_i(\mathbf{Y})$, we are only left with the third type of elements. Additionally, we observe that elements containing V_m or V_m^{-1} can not symbolically play any role in a combination of $R_j S_5$, which leaves us with

$$\mathbf{S}^\intercal (A^{[\ell]}|B_{\neq j}) \cdot \mathbf{g}(X, \mathbf{K}, \Gamma, \mathbf{R}) = \mathbf{S}^\intercal \mathbf{b}_j.$$

As $\mathbf{S}$ appears on both sides, we conclude that it must symbolically hold that

$$(A^{[\ell]}|B_{\neq j}) \cdot \mathbf{g}(X, \mathbf{K}, \Gamma, \mathbf{R}) = \mathbf{b}_j,$$

where the polynomials in $\mathbf{g}$ do not depend on $\mathbf{S}$.

Next, we will show that this holds if $|\mathsf{Cor}| < t$, which we summarize in the following claim:

Claim 2. *Let* $\mathbf{L}_1 \setminus (\mathbf{S}, \mathbf{V})$ *and* $\mathbf{L}_2 \setminus (\mathbf{S}, \mathbf{V})$ *be the elements from the* $\mathbf{L}_1$ *and* $\mathbf{L}_2$*, respectively, that do not depend on* $\mathbf{S}$ *or* $\mathbf{V}$*. Let*

$$A = \begin{pmatrix} W(\mathbf{Y}) & 1 & X^N - 1 & X & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & X^2 & 1 & 0 & 0 & 0 \\ 0 & \Gamma & 0 & 0 & 0 & 1 & 0 & 0 \\ X^t & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & X \end{pmatrix},$$

where $W(\mathbf{Y}) = \sum_{i\in\mathcal{H}} K_i L_i(X) + \sum_{i\in\mathsf{Cor}} \mathsf{sk}_i L_i(X)$*. Let* $A^{[\ell]}$ *be the the matrix of dimension* $5 \times 8\ell$*, where each column of* A *is repeated* ℓ *times. Let* $\mathbf{b}_i$ *be the vector* $(0, 0, 0, 0, R_i)^\intercal$*. Let* $B_{\neq j} = (\mathbf{b}_1 | \ldots | \mathbf{b}_j - 1 | \mathbf{b}_j + 1 | \ldots | \mathbf{b}_\ell)$*.*

If $|\mathsf{Cor}| < t$ *then for all* $j \in [\ell]$*, there does not symbolically exist any non-zero* $\mathbf{g} \in \mathbb{Z}_p[X, \mathbf{K}, \Gamma, \mathbf{R}]^{8\ell+\ell-1}$ *with* $g_{1,i}, g_{6,i}$*, and* $g_{8,i}$ *coming from* $\mathbf{L}_2$*, and* $g_{2,i}, g_{3,i}, g_{4,i}, g_{5,}, g_{7,i}$ *coming from* $\mathbf{L}_1$ *for all* $i \in [\ell]$*, and* $g_{9,i}$ *for* $i \neq j$ *coming from* $\mathbf{L}_T$ *such that*

$$A^{[\ell]}|B_{\neq j}\mathbf{g} = b_j.$$

Proof of Claim 2. Suppose towards contradiction that $\mathsf{Cor} < t$ and there exists such $\mathbf{g}$ for some $j \in [\ell]$. W.l.o.g., for ease of notation, assume $j = \ell$.

First, we note the equation system.

$$W\bar{g}_1 + \bar{g}_2 + (X^N - 1)\bar{g}_3 + X\bar{g}_4 = 0 \tag{3}$$
$$X^2\bar{g}_4 + \bar{g}_5 = 0 \tag{4}$$
$$\Gamma\bar{g}_2 + \bar{g}_6 = 0 \tag{5}$$
$$X^t\bar{g}_1 + \bar{g}_7 = 0 \tag{6}$$
$$\bar{g}_1 + X\bar{g}_8 + \sum_{i=1}^{\ell-1} g_{9,i} R_i = R_\ell \tag{7}$$

In the above, we denote $\bar{g}_k \coloneqq \sum_{i\in[\ell]} g_{k,i}$.

Next, let us inspect the structure of the different $\bar{g}_k$. For convenience, denote $R_0 \coloneqq 1$. For $k = 2, 3, 4, 5, 7$, the function $\bar{g}_k$ is a linear combination from elements of $\mathbf{L}_1 \setminus \mathbf{S}$, thus

$$\bar{g}_k = \sum_{m=0}^{\ell} R_m \left(\sum_{j=0}^{N} \bar{\alpha}_{j,m}^{(k)} X^j + \sum_{u\in\mathcal{H}} \sum_{j=0}^{N-1} \bar{\beta}_{u,j,m}^{(k)} K_u X^j \right).$$

[13] Here, we remove the trivial combination of $V_j R_j S_5 \cdot V_j^{-1}$ to account for the fact that the adversary must compute $R_j S_5$ in some other way than pairing $\mathsf{ct}_{9,j}$ with $[v_j^{-1}]_2$ to distinguish f_j from random, as the factor $R_j S_5$ is replaced with f_j in Game$_1$.

Note that this implies that, e.g. for $k = 1$, it holds that $\bar{\alpha}^{(1)}_{j,m} = \sum_{i\in[\ell]} \alpha^{(1,i)}_{j,m}$, and similarly for $\bar{\beta}$. Also, we define

$$\bar{g}_{k,m} = \sum_{j=0}^{N} \bar{\alpha}^{(k)}_{j,m} X^j + \sum_{u\in\mathcal{H}} \sum_{j=0}^{N-1} \bar{\beta}^{(k)}_{u,j,m} K_u X^j,$$

where $\bar{g}_{k,m}$ *does not depend on* any $R_1, \ldots, R_\ell$. As a result, we get

$$\bar{g}_k = \sum_{m=0}^{\ell} \bar{g}_{k,m}.$$

Further, for $k = 1, 6, 8$, the function $\bar{g}_k$ is a linear combination form elements of $\mathbf{L}_2 \setminus \mathbf{S}$, thus

$$\bar{g}_k = \sum_{m=0}^{\ell} R_m \left(\sum_{j=0}^{N} \bar{\alpha}^{(k)}_{j,m} X^j + \bar{\delta}^{(k)}_m \Gamma \right).$$

We define $\bar{g}_{k,m}$ analogously, such that they do not depend on $R_1, \ldots, R_\ell$.

Finally, the $g_{9,i}$ for $1 \le i < \ell$ are just constants, as they stem from $\mathbf{L}_T$.

In the following steps, we inspect the equations to find constraints on the different $\bar{g}_{k,\ell}$, i.e., the terms that get multiplied by R_ℓ. In the end, we aim to apply the sumcheck lemma on $R_\ell \cdot W \cdot \bar{g}_{1,\ell}$ in Equation 3, and reach a contradiction with the assumption that $|\mathrm{Cor}| < t$.

We start with $\bar{g}_1$.

- Equation 3: Clearly, $R_m\Gamma$ can only come from $\bar{g}_1$, hence we know that $\bar{\delta}^{(1)}_m = 0$ for all $0 \le m \le \ell$. Thus
$$\bar{g}_{1,m} = \sum_{j=0}^{N} \bar{\alpha}^{(k)}_{j,m} X^j.$$
- Equation 6: We insert $\bar{g}_1$ and $\bar{g}_7$:
$$X^t \sum_{m=0}^{\ell} R_m \left(\sum_{j=0}^{N} \bar{\alpha}^{(1)}_{j,m} X^j \right) + \sum_{m=0}^{\ell} R_m \left(\sum_{j=0}^{N} \bar{\alpha}^{(7)}_{j,m} X^j + \sum_{u\in\mathcal{H}} \sum_{j=0}^{N-1} \bar{\beta}^{(7)}_{u,j,m} K_u X^j \right) = 0$$
  By inspection, we can deduce the following constraints:
  - $\bar{\beta}^{(7)}_{u,j,m} = 0$ for all u, j, m, because K_u does not appear elsewhere.
  - $\bar{\alpha}^{(1)}_{j,m} = 0$ for all m, and all $j > N - t$, because the corresponding $\bar{g}_{7,m}$ have maximum degree N in X. Hence, we can write
$$\bar{g}_{1,m} = \sum_{j=0}^{N-t} \bar{\alpha}^{(k)}_{j,m} X^j.$$
- Equation 7: We can rearrange the equation as follows:
$$\bar{g}_1 - \left(R_\ell + \sum_{i=1}^{\ell-1} g_{9,i} R_i \right) = -X \cdot \bar{g}_8$$
$$\sum_{m=0}^{\ell} R_m \bar{g}_{1,m} - \left(R_\ell + \sum_{i=1}^{\ell-1} g_{9,i} R_i \right) = -X \sum_{m=0}^{\ell} R_m \left(\sum_{j=0}^{N} \bar{\alpha}^{(8)}_{j,m} X^j + \bar{\delta}^{(8)}_m \Gamma \right)$$
  First, observe that $\bar{\delta}^{(8)}_m = 0$ for all m, because $R_m\Gamma$ does not appear elsewhere. Next, observe that, for $X = 0$, it holds that $\bar{g}_1(X = 0, R_1, \ldots, R_\ell) = R_\ell + \sum_{i=1}^{\ell-1} g_{9,i} R_i$. Because of the term R_ℓ, we can conclude that $\bar{g}_{1,\ell}(0) = 1$, as no factor of R_ℓ appears otherwise in $\bar{g}_1$. Also, recall that $\bar{g}_{1,\ell}$ only depends on the variable X.

Next, we collect constraints on $\bar{g}_2$ using Equation 5.

$$\underbrace{\Gamma \cdot \sum_{m=0}^{\ell} R_m \left(\overbrace{\sum_{j=0}^{N} \bar{\alpha}^{(2)}_{j,m} X^j + \sum_{u\in\mathcal{H}} \sum_{j=0}^{N-1} \bar{\beta}^{(2)}_{u,j,m} K_u X^j}^{\bar{g}_{2,m}} \right)}_{\bar{g}_2} + \overbrace{\sum_{m=0}^{\ell} R_m \left(\underbrace{\sum_{j=0}^{N} \bar{\alpha}^{(6)}_{j,m} X^j + \bar{\delta}^{(6)}_m \Gamma}_{\bar{g}_{6,m}} \right)}^{\bar{g}_6} = 0$$

First, observe that $\bar{\alpha}^{(6)}_{j,m} = 0$ for all j, m, because all other terms contain a factor of Γ. Thus, it holds that $\bar{g}_{6,m} = \delta^{(6)}_m \Gamma$. Further, it must hold that $\bar{\beta}^{(2)}_{u,j,m} = 0$, because a factor of $\Gamma R_m K_u X^j$ does not appear elsewhere. Finally, it holds that $\bar{\alpha}^{(2)}_{j,m} = 0$ for $j > 0$ and all m, because $R_m\Gamma X$ does not appear elsewhere. We are left with $\bar{g}_{2,m} = \alpha^{(2)}_{0,m}$.

Next, we observe constraints on $\bar{g}_4$ and $\bar{g}_5$ from Equation 4:

$$\bar{g}_5 = -\bar{g}_4 X^2$$

As $\bar{g}_4$ has an additional factor of X^2, it must hold that $\bar{\alpha}^{(5)}_{j,m} = 0$ and $\bar{\beta}_{u,j,m} = 0$ for all u, m, and $j \in \{0, 1\}$.

Finally, we rewrite Equation 3, applying the sum-check lemma to $W \cdot R_\ell \bar{g}_{1,\ell}$. For this, recall that $\bar{g}_{1,\ell} = \sum_{j=0}^{N-t} \bar{\alpha}^{(1)}_{j,\ell} X^j$ and $\bar{g}_{1,\ell}(0) = 1$. We write $\bar{g}_{1,\ell}$ in Lagrange basis form as $\bar{g}_{1,\ell} = \sum_{i=1}^{N} b_i L_i(X)$, where $b_i = \bar{g}_{1,\ell}(\omega^i)$:

$$R_\ell \cdot \left(\sum_{i\in\mathcal{H}} b_i K_i + \sum_{i\in\mathrm{Cor}} b_i \mathsf{sk}_i + Q_x(X)X + Q_Z(X)Z(X) \right) + \sum_{m=0}^{\ell-1} R_m W \bar{g}_{1,m}(X) = -\bar{g}_2 - \bar{g}_3(X^N - 1) - \bar{g}_4 X$$

We can rewrite this as follows:

$$R_\ell \cdot \left(\overbrace{\sum_{i\in\mathcal{H}} b_i K_i + \sum_{i\in\mathsf{Cor}} b_i \mathsf{sk}_i + Q_x(X)X + Q_Z(X)Z(X)}^{T} \right) + F$$
$$= R_\ell \left(\underbrace{-\bar{g}_{2,\ell} - \bar{g}_{3,\ell}(X^N - 1) - \bar{g}_{4,\ell}X}_{T'} \right) + F',$$

where both F and F' do not depend on R_ℓ. Hence, it must hold that $T = T'$. We insert $\bar{g}_{2,\ell}, \bar{g}_{3,\ell}$, and replace $X \cdot \bar{g}_{4,\ell}$ with $\bar{g}_{5,\ell}$.

$$\sum_{i\in\mathcal{H}} b_i K_i + \sum_{i\in\mathsf{Cor}} b_i \mathsf{sk}_i + Q_x(X)X + Q_Z(X)(X^N - 1)$$
$$= -\alpha^{(2)}_{0,\ell} - \overbrace{\left(\sum_{j=0}^{N} \bar{\alpha}^{(3)}_{j,\ell} X^j + \sum_{u\in\mathcal{H}} \sum_{j=0}^{N-1} \bar{\beta}^{(3)}_{u,j,\ell} K_u X^j \right)}^{\bar{g}_{3,\ell}} (X^N - 1)$$
$$- \underbrace{\left(\sum_{j=2}^{N} \bar{\alpha}^{(5)}_{j,\ell} X^{j-2} + \sum_{u\in\mathcal{H}} \sum_{j=1}^{N-1} \bar{\beta}^{(5)}_{i,j,\ell} K_u X^{j-2} \right) X}_{\bar{g}_{5,\ell}/X}$$

First, observe that it must hold that $Q_Z(X) - \bar{g}_{3,\ell} = 0$, because there does not appear any factor of X^j for $j \geq N$ anywhere else. For a similar reason, it must hold that $Q_x(X) - (\bar{g}_{5,j}/X) = 0$, as there do not appear any other factors of X.

Thus, we can simplify the equation to

$$\sum_{i\in\mathcal{H}} b_i K_i + \sum_{i\in\mathsf{Cor}} b_i \mathsf{sk}_i = -\alpha^{(2)}_{0,\ell}.$$

We can deduce that $b_i = 0$ for all $i \in \mathcal{H}$. Combining with $\bar{g}_{1,\ell}(0) = 1$, we can lower-bound the degree of $\bar{g}_{1,\ell}$ to be $\deg(\bar{g}_{1,\ell}) \geq |\mathcal{H}|$. At the same time, however, we know that $\deg(\bar{g}_{1,\ell}) \leq N - t$, giving us $|\mathcal{H}| \leq N - t$. Since $|\mathcal{H}| = N - |\mathsf{Cor}|$ by definition, we get that $\mathsf{Cor} \geq t$, which is a contradiction to the initial assumption of $\mathsf{Cor} < t$. As a result, the system can not have a symbolic solution g. □

Following the Master Theorem 11, and applying a hybrid argument across all ℓ chunks, it holds that

$$\Pr[\mathsf{Game}_{0,\mathcal{A}}(1^\lambda) = 1]$$
$$\leq \Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) = 1] + \ell \cdot \frac{(q+v+2)^2 \cdot (n+2)}{2^{\lambda+1}},$$

where q is the number of group oracle queries made by the adversary, and v is the number of elements in the lists. Since $\mathcal{A}$ is PPT, the advantage is negligible. □

To close the proof, observe that since $c_{9,i} = [v_i \cdot (f_i + r_i m_i^{(b)})]_1$ for uniform random f_i, it holds that the adversary's view in Game_1 is identically distributed for $b = 0$ and $b = 1$ and thus

$$\Pr[\mathsf{Game}_{0,\mathcal{A}}(1^\lambda) = 1]$$
$$\leq \Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) = 1] + \mathsf{negl}(\lambda) \leq \frac{1}{2} + \mathsf{negl}(\lambda).$$

□

D.2 CCA Security of Σ_{SB}

Lemma 21 (CCA-Security of Σ_{SB}). *Σ_{SB} is* SB-IND-CCA*-secure, given the semantic security of Σ_{MSTE} (which holds in the GGM), the pseudorandomness of the* PRF*, the simulation-extractability of $\mathsf{PS}_{\mathsf{cca}}$, and the zero-knowledge property of $\mathsf{PS}_{\mathsf{range}}$ and $\mathsf{PS}_{\mathsf{pd}}$.*

We first present a sketch of our SB-IND-CCA proof.

Proof Sketch of Lemma 21. We prove CCA security through a series of computationally indistinguishable game hops, ultimately removing the challenge ciphertext's dependency on m_b, and thus rendering the final game identical for $b = 0$ and $b = 1$. We do so in the following steps:

(1) Replace the proofs π^{cca} and π^{range} with simulated ones for the challenge ciphertext. Indistinguishability follows from their respective zero-knowledge properties.
(2) Replace the commitments $\{\mathsf{com}_i\}_{i\in[\ell]}$ to the chunk keys with commitments to a constant, say 1, in the challenge ciphertext. We can do so because the proofs π^{cca} and π^{range} no longer depend on the committed value and randomness involved, as they are simulated. This step is indistinguishable due to the perfect hiding property of Pedersen commitments.
(3) As we later want to make a reduction to the semantic security of Σ_{MSTE}, we will have to simulate oracle queries to $\mathcal{O}^{\text{b-dec}}(\{\mathsf{ct}_j, \pi^{\mathsf{ct}}_j\}_{j\in[B]})$ without access to honest parties' secret keys. We do so in three steps:
   (a) We replace the proof π^{pd} with a simulated one, which no longer requires us to know sk_i. This step is indistinguishable following the zero-knowledge property of $\mathsf{PS}_{\mathsf{pd}}$.
   (b) We extract the witnesses ω_j from all proofs π^{cca}_j. In particular, this allows us to learn $s_{3,j}$, such that $\mathsf{ct}_{\mathsf{MSTE},j}.\mathsf{ct}_6 = [s_{3,j}]_1$. Extraction yields the correct witness following simulation-extractability of $\mathsf{PS}_{\mathsf{cca}}$.
   (c) We simulate partial decryption by computing $\mathsf{pd} \leftarrow \sum_{j\in[B]} s_{3,j} \cdot \mathsf{pk}^{(i)}_{1,0}$. Note that this is equivalent to a "real" partial decryption $\mathsf{pd} = \mathsf{sk}_i \cdot \sum_{j\in[B]} \mathsf{ct}_{\mathsf{MSTE},j}.\mathsf{ct}_6$, but computation no longer requires sk_i. As a result, we have removed all dependencies of $\mathcal{O}^{\text{b-dec}}$ on sk_i.
(4) We replace encryptions of chunk keys $(k_1, \ldots, k_\ell)$ in the challenge ciphertext with encryptions to a constant, say $(1, \ldots, 1)$, reducing to semantic security of Σ_{MSTE}. The reduction can simulate $\mathcal{O}^{\text{b-dec}}$ to its internal adversary, as it no longer depends on the honest parties' secret keys.
(5) Instead of padding the message as $\beta \leftarrow \mathsf{PRF}(k, i) + m_b$, we sample a uniform random $r \leftarrow\!\!\$\ \mathbb{G}_T$ and set $\beta \leftarrow r + m_b$. We prove indistinguishability by reduction to the pseudorandomness of the PRF, which is possible because all dependencies on the PRF key k (i.e., as witness for the proofs π^{cca} and π^{range}, and as (chunked) message in Σ_{MSTE}) were removed in prior game hops. The only remaining dependency is the punctured key, which is provided by the pseudorandomness game, and thus can be simulated by the reduction.
(6) Finally, we replace $\beta \leftarrow r + m_b$ with $\beta \leftarrow r$, which is identically distributed.

As a result, we have eliminated all dependencies of Game-SB-IND-CCA on b through a series of computationally indistinguishable game hops. Hence, no PPT adversary can guess b with non-negligible advantage.

We next present the full proof.

The proof follows the structure of the CCA-security proof in [18] with some extra steps. In particular, we replace the Pedersen commitments $\{\mathsf{com}_i\}_{i\in[\ell]}$ to k_i with commitments to 1 in the challenge ciphertext at the appropriate time. Also, we now simulate partial decryption by extracting s_3 from $\mathsf{PS}_{\mathsf{cca}}$.

Proof of Lemma 21. We prove SB-IND-CCA security of Σ_{SB} (Definition 6) through a series of game-hops where we eliminate all dependencies on the internal bit b of Game-SB-IND-CCA. Let $\mathsf{Game}_{0,\mathcal{A}}$ be the original game defined in Figure 1, instantiated with Σ_{SB}.

$\underline{\mathsf{Game}_{1,\mathcal{A}}}$: In this game, we make modifications concerning the proof system $\mathsf{PS}_{\mathsf{cca}}$, making use of its simulator/extractor $\mathsf{Sim}_{\mathsf{PS}_{\mathsf{cca}}}$.

- We replace the setup of the proof system $\mathsf{PS}_{\mathsf{cca}}$ by one generated by the simulator:

$$(\mathsf{pp}_{\mathsf{cca}}, \mathsf{st}_{\mathsf{Sim}}) \leftarrow\!\$\ \mathsf{Sim}_{\mathsf{PS}_{\mathsf{cca}}}.\mathsf{Setup}(1^\lambda, \mathcal{G})$$

- We replace the proof π^{cca} attached to the challenge ciphertext $\mathsf{ct}^\star$ with a simulated one:

$$(\pi^{\mathsf{cca}}, \mathsf{st}_{\mathsf{Sim}}) \leftarrow\!\$\ \mathsf{Sim}_{\mathsf{PS}_{\mathsf{cca}}}.\mathsf{Sim}((\mathsf{pp}, \mathsf{ek}, \mathsf{ct}^\star, \pi^{\mathsf{range}}), \mathsf{st}_{\mathsf{Sim}})$$

- In the $\mathcal{O}^{\text{b-dec}}$ oracle, on input $\{(\mathsf{ct}_j, (\pi_j^{\mathsf{range}}, \pi_j^{\mathsf{cca}}))\}_{j\in[B]}$ (after verifying all proofs, and if $\mathsf{ct}^\star \notin \{\mathsf{ct}_j\}_{j\in[B]}$), we use the simulator $\mathsf{Sim}_{\mathsf{PS}_{\mathsf{cca}}}$ to extract the witnesses $\{\omega_j\}_{j\in[B]}$ from all proofs $\{\pi_j^{\mathsf{cca}}\}_{j\in[B]}$. For all $j \in [B]$:

$$(\omega_j, \mathsf{st}_{\mathsf{Sim}}) \leftarrow\!\$\ \mathsf{Sim}_{\mathsf{PS}_{\mathsf{cca}}}.\mathsf{Ext}((\mathsf{pp}, \mathsf{ek}, \mathsf{ct}_j, \pi_j^{\mathsf{range}}), \pi_j^{\mathsf{cca}}, \mathsf{st}_{\mathsf{Sim}}).$$

  If for any $j \in [B]$ it holds that $((\mathsf{pp}, \mathsf{ek}, \mathsf{ct}_j, \pi_j^{\mathsf{range}}), \omega_j) \notin \mathsf{R}_{\mathsf{cca}}$, we output $\perp$ as a response to the query.

Claim 3. *If the proof system* $\mathsf{PS}_{\mathsf{cca}}$ *satisfies simulation-extractability (Definition 15), then for all PPT adversaries* $\mathcal{A}$ *there exists a negligible function* negl *such that*

$$\left|\Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{0,\mathcal{A}}(1^\lambda) = 1]\right| \le \mathsf{negl}(\lambda)$$

Proof of Claim 3. The proof follows straightforwardly from the simulation-extractability of $\mathsf{PS}_{\mathsf{cca}}$. Consider a reduction $\mathcal{B}$ playing in $\text{Game-SimExt}_{\mathcal{B}}^{\mathsf{PS}_{\mathsf{cca}},\mathsf{Sim}_{\mathsf{PS}_{\mathsf{cca}}}}$ for an appropriate simulator $\mathsf{Sim}_{\mathsf{PS}_{\mathsf{cca}}}$. $\mathcal{B}$ runs the distinguisher $\mathcal{A}$ internally, embedding the setup $\mathsf{pp}_{\mathsf{cca}}$ of $\mathsf{PS}_{\mathsf{cca}}$ it receives from its challenger. Note that in its simulation, $\mathcal{B}$ internally samples a random bit β to decide which message m_β to encrypt. To generate the proof π^{cca} for the challenge ciphertext, $\mathcal{B}$ simply queries the $\mathcal{O}^{\mathsf{Prove}}$ oracle, which $\mathcal{B}$ can do as it knows the corresponding witness for the challenge ciphertext. For every $\mathcal{O}^{\text{b-dec}}$ query made by $\mathcal{A}$, instead of verifying the proofs $\{\pi_j^{\mathsf{cca}}\}_{j\in[B]}$, $\mathcal{B}$ queries its $\mathcal{O}^{\mathsf{Verify}}((\mathsf{pp}, \mathsf{ek}, \mathsf{ct}_j, \pi_j^{\mathsf{range}}), \pi_j^{\mathsf{cca}})$ oracle, and outputs $\perp$ if any of the queries returns a value other than 1. Finally, when $\mathcal{A}$ outputs a bit β', $\mathcal{B}$ outputs 1 whenever $\beta = \beta'$, and 0 otherwise.

Analysis. Denote with b the internal bit of the challenger in $\text{Game-SimExt}_{\mathcal{B}}^{\mathsf{PS}_{\mathsf{cca}},\mathsf{Sim}_{\mathsf{PS}_{\mathsf{cca}}}}$, and with b' the output of the reduction $\mathcal{B}$. Observe that, if $b = 0$, then $\mathcal{B}$ perfectly simulates Game_0 to $\mathcal{A}$. If, however, $b = 1$, then $\mathcal{B}$ perfectly simulates Game_1. Hence, we get that

$$\begin{aligned}
&\left|\Pr\left[\text{Game-SimExt}_{\mathcal{B}}^{\mathsf{PS}_{\mathsf{cca}},\mathsf{Sim}_{\mathsf{PS}_{\mathsf{cca}}}}(1^\lambda) = 1\right] - \frac{1}{2}\right| \\
&= \left|\Pr\left[b' = b\right] - \frac{1}{2}\right| \\
&= \left|\frac{1}{2}\Pr\left[\beta' \neq \beta \mid b = 0\right] + \frac{1}{2}\Pr\left[\beta' = \beta \mid b = 1\right] - \frac{1}{2}\right| \\
&= \left|\frac{1}{2}(1 - \Pr\left[\beta' = \beta \mid b = 0\right] + \Pr\left[\beta' = \beta \mid b = 1\right]) - \frac{1}{2}\right| \\
&= \frac{1}{2}\left|\Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{0,\mathcal{A}}(1^\lambda) = 1]\right|.
\end{aligned}$$

Given the simulation-extractability of $\mathsf{PS}_{\mathsf{cca}}$, applied also to $\mathcal{B}$ with complemented output, we conclude that

$$\begin{aligned}
&\left|\Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{0,\mathcal{A}}(1^\lambda) = 1]\right| \\
&\le 2 \cdot \left|\Pr\left[\text{Game-SimExt}_{\mathcal{B}}^{\mathsf{PS}_{\mathsf{cca}},\mathsf{Sim}_{\mathsf{PS}_{\mathsf{cca}}}}(1^\lambda) = 1\right] - \frac{1}{2}\right| \\
&\le 2 \cdot \mathsf{negl}(\lambda),
\end{aligned}$$

for a negligible function negl. □

$\underline{\mathsf{Game}_{2,\mathcal{A}}}$: In this game, we now simulate the proof π^{range} in a similar manner. To do so, let $\mathsf{Sim}_{\mathsf{PS}_{\mathsf{range}}}$ be the simulator for $\mathsf{PS}_{\mathsf{range}}$. We leave simulator state implicit in this game and Game_4. We initially set $\mathsf{pp}_{\mathsf{range}} \leftarrow\!\$\ \mathsf{Sim}_{\mathsf{PS}_{\mathsf{range}}}.\mathsf{Setup}(1^\lambda, \mathcal{G})$ and then simulate $\pi^{\mathsf{range}} \leftarrow\!\$\ \mathsf{Sim}_{\mathsf{PS}_{\mathsf{range}}}.\mathsf{Sim}((\mathsf{pp}, \{\mathsf{com}_i\}_{i\in[\ell]}, z_1))$ for the challenge ciphertext.

Claim 4. *If the proof system* $\mathsf{PS}_{\mathsf{range}}$ *is zero-knowledge (Appendix B.3) then* $\mathsf{Game}_{2,\mathcal{A}}$ *is computationally indistinguishable from* $\mathsf{Game}_{1,\mathcal{A}}$.

$$\left|\Pr[\mathsf{Game}_{2,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) = 1]\right| \le \mathsf{negl}(\lambda)$$

Proof of Claim 4. The claim follows straight from the zero-knowledge property of the proof system $\mathsf{PS}_{\mathsf{range}}$. □

$\underline{\mathsf{Game}_{3,\mathcal{A}}}$: In this game, we change how the Pedersen commitments $\{\mathsf{com}_i\}_{i\in[\ell]}$ are computed for the challenge ciphertext. Instead of committing with $\mathsf{com}_i \leftarrow\!\$\ \mathsf{Ped.Commit}(k_i)$ to each k_i, we instead set $\mathsf{com}_i \leftarrow\!\$\ \mathsf{Ped.Commit}(1)$ for all $i \in [\ell]$.

Claim 5. *For all (even unbounded) adversaries* $\mathcal{A}$ *it holds that*

$$\Pr[\mathsf{Game}_{3,\mathcal{A}}(1^\lambda) = 1] = \Pr[\mathsf{Game}_{2,\mathcal{A}}(1^\lambda) = 1].$$

Proof of Claim 5. The claim follows directly from the perfect hiding property of the Pedersen commitment scheme. □

In a following game-hop, we want to make a reduction to the semantic security of the underlying Σ_{MSTE} scheme (Definition 20). In order to do that, we need to simulate the batch-decryption oracle $\mathcal{O}^{\text{b-dec}}$, which has a dependency on the threshold silent setup secret keys sk_i for $i \in \mathcal{H}$. Because we only require Σ_{MSTE} to be semantically secure, we do not have access to a decryption oracle, which we could use to simulate the partial decryptions of Σ_{MSTE}. Hence, we remove the dependency on sk_i of $\mathcal{O}^{\text{b-dec}}$ within the following three

game-hops using the zero-knowledge property of the proof system $\mathsf{PS}_{\mathsf{pd}}$, and simulation-extractability of the proof system $\mathsf{PS}_{\mathsf{cca}}$ to simulate decryption shares.

$\underline{\mathsf{Game}_{4,\mathcal{A}}}$: In this game, we replace the proof of valid partial decryption π^{pd} that is generated within $O^{\text{b-dec}}$ with a simulated one. Let $\mathsf{Sim}_{\mathsf{PS}_{\mathsf{pd}}}$ be the simulator for the proof system $\mathsf{PS}_{\mathsf{pd}}$. To do so, we initially generate $\mathsf{pp}_{\mathsf{pd}} \leftarrow\!\!\$\ \mathsf{Sim}_{\mathsf{PS}_{\mathsf{pd}}}.\mathsf{Setup}(1^\lambda, \mathcal{G})$, and, whenever the adversary queries $O^{\text{b-dec}}(\{\mathsf{ct}_j, \pi_j\}_{j\in[B]}, i)$, we compute pd normally, but simulate

$$\pi^{\mathsf{pd}} \leftarrow\!\!\$\ \mathsf{Sim}_{\mathsf{PS}_{\mathsf{pd}}}.\mathsf{Sim}((\mathsf{pp}, \mathsf{pk}_i, \{\mathsf{ct}_j\}_{j\in[B]}, \mathsf{pd})).$$

We call the resulting oracle $O_4^{\text{b-dec}}$

Claim 6. *If the proof system* $\mathsf{PS}_{\mathsf{pd}}$ *is zero-knowledge (Appendix B.3) then* $\mathsf{Game}_{4,\mathcal{A}}$ *is computationally indistinguishable from* $\mathsf{Game}_{3,\mathcal{A}}$.

$$\left|\Pr[\mathsf{Game}_{4,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{3,\mathcal{A}}(1^\lambda) = 1]\right| \le \mathsf{negl}(\lambda)$$

Proof of Claim 6. The claim follows directly from the zero-knowledge property of the proof system $\mathsf{PS}_{\mathsf{pd}}$. □

$\underline{\mathsf{Game}_{5,\mathcal{A}}}$: In this game we make another change to $O^{\text{b-dec}}$, which finally removes its dependency on sk_i for $i \in \mathcal{H}$. Instead of computing partial decryption in Σ_{MSTE} on the aggregated ciphertext[14] $\mathsf{ct} = \sum_{j\in[B]} \mathsf{ct}_j$ using sk_i, we simulate partial decryption using the extracted witnesses $\{\omega_j\}_{j\in[B]}$, particularly $\{\tilde{s_3}\}_{j\in[B]}$. To do so, we parse for all $j \in [B]$

$$\left(\vec{\tilde{k}}_j, \vec{\tilde{s}}_j, \vec{\tilde{r}}_j\right) \leftarrow \omega_j,$$

where $\vec{\tilde{k}}_j$ corresponds to the chunk keys of ciphertext j, $\vec{\tilde{s}}_j$ corresponds to the randomness during encryption in Σ_{MSTE} for ciphertext j, and $\vec{\tilde{r}}_j$ corresponds to the randomness used for the Pedersen commitments in ciphertext j. Next, we compute the partial decryption as follows: First, set $\tilde{s_3} = \sum_{j\in[B]} \tilde{s_3}_j$. Then, compute

$$\mathsf{pd}_i \leftarrow \tilde{s_3} \cdot \mathsf{pk}_{1,0}^{(i)}\big[= [\mathsf{sk}_i \cdot \tilde{s_3}]_1\big],$$

which can be computed without knowledge of sk_i. We denote the resulting oracle as $O_5^{\text{b-dec}}$

Claim 7. *The games are identical, thus*

$$\Pr[\mathsf{Game}_{5,\mathcal{A}}(1^\lambda) = 1] = \Pr[\mathsf{Game}_{4,\mathcal{A}}(1^\lambda) = 1].$$

Proof of Claim 7. The partial decryption as computed from the secret key (Game_4) can only differ from the one computed using $\tilde{s_3}$ (Game_5), if there exists at least one $j \in [B]$ such that $[\tilde{s_3}_j]_1 \neq \mathsf{ct}_6^{(j)}$. In this case, however, both games would have aborted due to the abort condition that was first introduced in Game_1 (i.e., the assertion that $((\mathsf{pp}, \mathsf{ek}, \mathsf{ct}_j, \pi_j^{\mathsf{range}}), \omega_j) \in \mathsf{R}_{\mathsf{cca}}$), leading both oracles to return $\perp$ instead. □

Having removed the dependency on the threshold silent setup secret keys sk_i from the batch-decryption oracle $O_5^{\text{b-dec}}$, we can proceed with a game hop that replaces encryption of $(k_1, \ldots, k_\ell)$ in the challenge ciphertext with an encryption of an arbitrary constant (say $(1, \ldots, 1)$). We can reduce the indistinguishability of this game hop to the semantic security of Σ_{MSTE}. We do not run into issues with simulating $O^{\text{b-dec}}$ queries in this reduction, because we no longer need the secret keys to do so.

$\underline{\mathsf{Game}_{6,\mathcal{A}}}$: In this game we change how the challenge ciphertext is computed and replace the encryption of $(k_1, \ldots, k_\ell)$ with an encryption of $(1, \ldots, 1)$ (of length ℓ). Instead of computing $\mathsf{ct} \leftarrow\!\!\$\ \Sigma_{\mathsf{MSTE}}.\mathsf{Enc}(\{\mathsf{pk}_i\}_{[n]}, \vec{k})$, we set $\mathsf{ct} \leftarrow\!\!\$\ \Sigma_{\mathsf{MSTE}}.\mathsf{Enc}(\{\mathsf{pk}_i\}_{[n]}, \vec{1})$.

Claim 8. *If the underlying* Σ_{MSTE} *scheme is semantically secure (Definition 20), then* $\mathsf{Game}_{6,\mathcal{A}}$ *is computationally indistinguishable from* $\mathsf{Game}_{5,\mathcal{A}}$.

$$\left|\Pr[\mathsf{Game}_{6,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{5,\mathcal{A}}(1^\lambda) = 1]\right| \le \mathsf{negl}(\lambda)$$

Proof of Claim 8. We prove this claim by reduction to the semantic security of Σ_{MSTE}. Let $\mathcal{A}$ be a PPT adversary such that

$$\left|\Pr[\mathsf{Game}_{6,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{5,\mathcal{A}}(1^\lambda) = 1]\right| \ge \epsilon(\lambda)$$

for a non-negligible ϵ. We construct a PPT reduction $\mathcal{B}$ that runs within the Game-S-IND-CPA$_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ challenger (Figure 1) and internally uses $\mathcal{A}$ to win.

$\mathcal{B}$ receives crs from Game-S-IND-CPA$_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ and runs $\mathcal{A}$ with (crs) as input, receiving a universe $C \subseteq [n]$ and a set of corrupted parties $\mathsf{Cor} \subseteq C$, which $\mathcal{B}$ forwards to Game-S-IND-CPA$_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$, receiving a set of public keys $\{\mathsf{pk}_i\}_{i\in\mathcal{H}}$. $\mathcal{B}$ sends the honest parties' public keys to $\mathcal{A}$ and receives a set of public keys for the corrupted parties $\{\mathsf{pk}_i\}_{i\in\mathsf{Cor}}$. It follows the (identical) steps from Game_5 and Game_6 up to the point where it is supposed to encrypt $\vec{k} \coloneqq (k_1, \ldots, k_\ell)$ or $\vec{1} \coloneqq (1, \ldots, 1)$ respectively as part of the challenge ciphertext. Instead, $\mathcal{B}$ sends

$$(m_0 \coloneqq \vec{k}, m_1 \coloneqq \vec{1})$$

to Game-S-IND-CPA$_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ and receives ct as a response. $\mathcal{B}$ then continues following the steps from Game_5 and Game_6.

Note that $\mathcal{B}$ can simulate the batch decryption oracle $O_6^{\text{b-dec}} = O_5^{\text{b-dec}}$ to $\mathcal{A}$, as it no longer depends on any unknown secret key share sk_i for $i \in \mathcal{H}$. This dependency was removed in the previous game hop.

Analysis. Let b' be the internal bit of Game-S-IND-CPA$_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$. If $b' = 0$, then Game-S-IND-CPA$_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ returns an encryption of $\vec{k}$ and $\mathcal{B}$ simulates $\mathsf{Game}_{5,\mathcal{A}}$ to $\mathcal{A}$. If $b' = 1$, then Game-S-IND-CPA$_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ returns an encryption of $\vec{1}$ and $\mathcal{B}$ simulates $\mathsf{Game}_{6,\mathcal{A}}$ to $\mathcal{A}$. Denote $\mathcal{B}$'s output to the Game-S-IND-CPA$_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ challenger with d, setting $d = 1$ exactly when $\mathcal{A}$ guesses the internally sampled SB-IND-CCA

[14] Here we abuse notation about and define aggregation of ciphertexts in a batch through the $\sum$ operator.

challenge bit correctly.

$$\begin{aligned}
&\left|\Pr\left[\text{Game-S-IND-CPA}_{\mathcal{B}}^{\Sigma_{\text{MSTE}}}(1^\lambda)=1\right]-\frac{1}{2}\right| \\
&=\left|\frac{1}{2}\Pr\left[d=0\mid b'=0\right]+\frac{1}{2}\Pr\left[d=1\mid b'=1\right]-\frac{1}{2}\right| \\
&=\left|\frac{1}{2}(1-\Pr\left[d=1\mid b'=0\right]+\Pr\left[d=1\mid b'=1\right])-\frac{1}{2}\right| \\
&=\frac{1}{2}\cdot\left|-\Pr[\text{Game}_{5,\mathcal{A}}(1^\lambda)=1]+\Pr[\text{Game}_{6,\mathcal{A}}(1^\lambda)=1]\right| \\
&\geq\frac{\epsilon(\lambda)}{2}.
\end{aligned}$$

This contradicts the semantic security of Σ_{MSTE} (also considering $\mathcal{B}$ with complemented output). □

$\underline{\text{Game}_{7,\mathcal{A}}}$: In this game, we replace the result of the PRF-evaluation during encryption with a random group element from $\mathbb{G}_T$. Instead of computing $\beta \leftarrow m_b + \text{PRF}(k,i)$ we set $\beta \leftarrow m_b + r$ for a random $r \leftarrow\!\!\$\ \mathbb{G}_T$.

Claim 9. *If* PRF *is pseudorandom (Definition 18) then* $\text{Game}_{7,\mathcal{A}}$ *is computationally indistinguishable from* $\text{Game}_{6,\mathcal{A}}$.

$$\left|\Pr[\text{Game}_{7,\mathcal{A}}(1^\lambda)=1]-\Pr[\text{Game}_{6,\mathcal{A}}(1^\lambda)=1]\right|\leq\text{negl}(\lambda)$$

Proof Sketch of Claim 9. We prove this claim by reduction to the pseudorandomness Game-Pr of PRF. The PPT reduction $\mathcal{B}$ playing in Game-$\text{Pr}_{\mathcal{B}}$ sends index i to the pseudorandomness game and receives k' and value r as a response. $\mathcal{B}$ then computes β as $\beta \leftarrow m_b + r$. If the r is sampled uniformly random by the pseudorandomness game ($b=0$), $\mathcal{B}$ simulates $\text{Game}_{7,\mathcal{A}}$ to $\mathcal{A}$. If r is the result of the PRF evaluation ($b=1$), $\mathcal{B}$ simulates $\text{Game}_{6,\mathcal{A}}$ to $\mathcal{A}$. Hence, if $\mathcal{A}$ can distinguish between $\text{Game}_{6,\mathcal{A}}$ and $\text{Game}_{7,\mathcal{A}}$, then $\mathcal{B}$ can distinguish between $b=0$ and $b=1$. □

$\underline{\text{Game}_{8,\mathcal{A}}}$: In this game we directly sample β from $\mathbb{G}_T$ instead of computing it as $m_b + r$ for a random $r \leftarrow\!\!\$\ \mathbb{G}_T$.

Claim 10. *Both games are identically distributed, and thus it holds that*

$$\Pr\left[\text{Game}_{8,\mathcal{A}}(1^\lambda)=1\right]=\Pr\left[\text{Game}_{7,\mathcal{A}}(1^\lambda)=1\right]$$

Proof of Claim 10. Clearly, β is identically distributed in both games. □

We conclude the proof by arguing that the adversary's view in $\text{Game}_{8,\mathcal{A}}$ no longer depends on the internal bit b. Hence

$$\Pr\left[\text{Game}_{8,\mathcal{A}}(1^\lambda)=1\right]=\frac{1}{2}.$$

Further, we have derived $\text{Game}_{8,\mathcal{A}}$ from Game-SB-IND-CCA$_{\mathcal{A}}^{\Sigma_{\text{SB}}}$ through a series of game-hops where each game is computationally indistinguishable from the previous one. We conclude that for all PPT adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\lambda)$ such that

$$\begin{aligned}
&\Pr[\text{Game-SB-IND-CCA}_{\mathcal{A}}^{\Sigma_{\text{SB}}}(1^\lambda)=1] \\
&\leq\Pr[\text{Game}_{8,\mathcal{A}}(1^\lambda)=1]+\text{negl}(\lambda)=\frac{1}{2}+\text{negl}(\lambda)
\end{aligned}$$

which satisfies Definition 6. □

D.3 Rogue Ciphertext Security of Σ_{SB}

Lemma 22 (Rogue Ciphertext Security of Σ_{SB}). *The Σ_{SB} construction is secure against rogue ciphertext attacks given the correctness of the* PRF *and Σ_{MSTE}, computational binding of the Pedersen commitment scheme, and the knowledge-soundness of* PS_{cca}, PS_{range}, *and* PS_{pd}.

Proof of Lemma 22. The rogue ciphertext security of Σ_{SB} follows from the perfect soundness of KVfy and the knowledge-soundness of the proof systems PS_{cca}, PS_{pd}, and PS_{range}, and the computational binding property of the Pedersen commitment scheme Ped. To summarize, in Game-Rogue, the adversary can control the keys of an arbitrary set of corrupted parties Cor in the universe C and pick some index i and a message m_i. The challenger then encrypts a challenge ciphertext ct_i. The adversary can generate $B-1$ additional ciphertexts $\{\text{ct}_j, \pi_j^{\text{ct}}\}_{j\in[B]\setminus\{i\}}$, for some $B \leq B_{\max}$ with $i \in [B]$. Also, the adversary picks a subset of committee members S of size $\geq t$, and issues partial decryptions $\{\text{pd}_u, \pi_u^{\text{pd}}\}_{u\in S\cap\text{Cor}}$ on behalf of all corrupted committee members on the batch of ciphertexts, including ct_i at position i. The challenger then computes the remaining partial decryptions $\{\text{pd}_u, \pi_u^{\text{pd}}\}_{u\in S\cap\mathcal{H}}$ on behalf of $S\cap\mathcal{H}$, where $\mathcal{H} = C \setminus \text{Cor}$. The adversary wins if all proofs π_j^{ct} for $j \in [B]$ verify and one of the following conditions is met:

- At least one of the honest partial decryptions is invalid, i.e. $\text{PDVfy}(\text{pd}_u, \{\text{ct}_j\}_{j\in[B]}, \text{pk}_u, \pi_u^{\text{pd}}) = 0$ for some $u \in S \cap \mathcal{H}$.
- All proofs π_u^{pd} for $u \in S$ verify, but the decryption returns a message $m_i' \neq m_i$ for index i.

Observe that, by the correctness of PS_{pd}, the first condition can never occur. Henceforth, we consider the second winning condition.

First, KVfy ensures the validity of the public keys chosen by the adversary, i.e., for every $u \in \text{Cor}$ there exists a secret key sk_u such that $(\text{pk}_u, \text{sk}_u) \in \text{KGen}(\text{pp})$. Since the challenger generates the remaining public keys honestly, the same holds for every $u \in C$.

We make explicit that batch validation in Game-Rogue checks that each ct_j carries index j and the common threshold t, for every $j \in [B]$.

Let $\text{Game}_0 = \text{Game-Rogue}_{\mathcal{A}}^{\Sigma_{\text{SB}}}$. We make a series of modifications to Game_0, showing that $\mathcal{A}$'s success probability decreases by at most a negligible amount at each step. Throughout the proof, let $z = \lceil \lambda_k/\ell \rceil$ be the chunk width, as in the construction, where λ_k is the PRF key length. All extraction checks below are performed after the corresponding original proof verification has succeeded; we return 0 if the extractor outputs $\perp$ or its output cannot be parsed as the specified witness.

$\underline{\text{Game}_1}$. In Game_1, we use simulator $\text{Sim}_{\text{PS}_{\text{cca}}} = (\text{Setup}, \text{Ext})$ of PS_{cca} to extract the witnesses ω_j^{cca} from $(\text{ct}_j, \pi_j^{\text{cca}})$. In particular, we replace the setup of PS_{cca} by pp_{cca}, where $(\text{pp}_{\text{cca}}, \text{st}_{\text{Sim}_{\text{PS}_{\text{cca}}}}) \leftarrow\!\!\$\ \text{Sim}_{\text{PS}_{\text{cca}}}.\text{Setup}(1^\lambda, \mathcal{G})$. Then, after receiving $\{\text{ct}_j, \pi_j^{\text{cca}}\}_{j\in[B]\setminus\{i\}}$, we compute for all $j \in [B] \setminus \{i\}$

$$(\omega_j^{\text{cca}}, \text{st}_{\text{Sim}_{\text{PS}_{\text{cca}}}}) \leftarrow\!\!\$\ \text{Sim}_{\text{PS}_{\text{cca}}}.\text{Ext}((\text{pp}, \text{ek}, \text{ct}_j), \pi_j^{\text{cca}}, \text{st}_{\text{Sim}_{\text{PS}_{\text{cca}}}}).$$

where $\text{ct}_j = (j, t, k^*, \beta, \text{ct}_{\Sigma_{\text{MSTE}}}, \text{com}_1, \ldots, \text{com}_\ell, \pi_j^{\text{range}})$ includes the range proof for PS_{cca}, as in Appendix C. We parse ω_j^{cca} as

$$((k_1^{\text{cca}}, \ldots, k_\ell^{\text{cca}}), \vec{s}, (r_1^{\text{com,cca}}, \ldots, r_\ell^{\text{com,cca}})),$$

omitting the index j for readability. Finally, we add an additional abort condition and return 0, if for any $j \in [B] \setminus \{i\}$, one of the following conditions is met:

- $k^* \neq \mathsf{PRF.Punct}\left(\sum_{i\in[\ell]} k_i^{\mathsf{cca}} \cdot 2^{(i-1)z}, j\right)$
- $\mathsf{ct}_{\Sigma_{\mathsf{MSTE}}} \neq \Sigma_{\mathsf{MSTE}}.\mathsf{Enc}(\mathsf{ek}, \{k_i^{\mathsf{cca}}\}_{i\in[\ell]}, t; \vec{s})$
- $\exists i \in [\ell]\colon \mathsf{com}_i \neq \mathsf{Ped.Commit}(k_i^{\mathsf{cca}}, r_i^{\mathsf{com,cca}})$

Claim 11. *For all PPT adversaries $\mathcal{A}$, there exists a reduction $\mathcal{B}$ such that*

$$\Pr[\mathsf{Game}_{0,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) = 1] \leq 2 \cdot \mathsf{Adv}^{\mathsf{Game\text{-}Ext}}_{\mathcal{B},\mathsf{PS}_{\mathsf{cca}},\mathsf{Sim}_{\mathsf{PS}_{\mathsf{cca}}}}(1^\lambda).$$

Proof of Claim 11. The claim follows a straightforward reduction to knowledge-soundness of $\mathsf{PS}_{\mathsf{cca}}$. To confirm this, observe that if any of the three abort conditions is met, then $(\chi_j^{\mathsf{cca}}, \omega_j^{\mathsf{cca}}) \notin \mathsf{R}_{\mathsf{cca}}$ for some $j \in [B] \setminus \{i\}$, where $\chi_j^{\mathsf{cca}} = (\mathsf{pp}, \mathsf{ek}, \mathsf{ct}_j)$. □

Game$_2$: Next, we make a similar modification for $\mathsf{PS}_{\mathsf{range}}$. We use the extractor $\mathsf{Sim}_{\mathsf{PS}_{\mathsf{range}}} = (\mathsf{Setup}, \mathsf{Ext})$ to extract a witness $\omega_j^{\mathsf{range}}$ from $(\mathsf{ct}_j, \pi_j^{\mathsf{range}})$. More formally, we embed the simulated setup $\mathsf{pp}_{\mathsf{range}}$ as $(\mathsf{pp}_{\mathsf{range}}, \mathsf{st}_{\mathsf{Sim}_{\mathsf{PS}_{\mathsf{range}}}}) \leftarrow\!\$\ \mathsf{Sim}_{\mathsf{PS}_{\mathsf{range}}}.\mathsf{Setup}(1^\lambda, \mathcal{G})$. Then, after receiving $\{\mathsf{ct}_j, \pi_j^{\mathsf{range}}\}_{j\in[B]\setminus\{i\}}$, we extract $\omega_j^{\mathsf{range}}$ as

$$(\omega_j^{\mathsf{range}}, \mathsf{st}_{\mathsf{Sim}_{\mathsf{PS}_{\mathsf{range}}}}) \leftarrow\!\$\ \mathsf{Sim}_{\mathsf{PS}_{\mathsf{range}}}.\mathsf{Ext}((\mathsf{pp}, \{\mathsf{com}_i\}_{i\in[\ell]}, z), \pi_j^{\mathsf{range}}, \mathsf{st}_{\mathsf{Sim}_{\mathsf{PS}_{\mathsf{range}}}}),$$

Here z is the chunk width defined above, and the commitments are those of ct_j. We parse

$$\omega_j^{\mathsf{range}} \coloneqq (k_1^{\mathsf{range}}, \ldots, k_\ell^{\mathsf{range}}, r_1^{\mathsf{com,range}}, \ldots, r_\ell^{\mathsf{com,range}}),$$

omitting the index j for readability. We add an additional abort condition and return 0 if one of the following conditions is met for any $j \in [B] \setminus \{i\}$:

- $\exists i \in [\ell]\colon \mathsf{com}_i \neq \mathsf{Ped.Commit}(k_i^{\mathsf{range}}, r_i^{\mathsf{com,range}})$
- $\exists i \in [\ell]\colon k_i^{\mathsf{range}} \notin [0, 2^z - 1]$.

Claim 12. *For all PPT adversaries $\mathcal{A}$ there exists a PPT reduction $\mathcal{B}$ such that*

$$\begin{aligned}&\Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{2,\mathcal{A}}(1^\lambda) = 1]\\ \leq\ &2 \cdot \mathsf{Adv}^{\mathsf{Game\text{-}Ext}}_{\mathcal{B},\mathsf{PS}_{\mathsf{range}},\mathsf{Sim}_{\mathsf{PS}_{\mathsf{range}}}}(1^\lambda).\end{aligned}$$

Proof of Claim 12. The claim follows a straightforward reduction to knowledge-soundness of $\mathsf{PS}_{\mathsf{range}}$. In particular, observe that if any of the new abort conditions is met, then $(\chi_j^{\mathsf{range}}, \omega_j^{\mathsf{range}}) \notin \mathsf{R}_{\mathsf{range}}$ for some $j \in [B] \setminus \{i\}$, where $\chi_j^{\mathsf{range}} = (\mathsf{pp}, \{\mathsf{com}_i\}_{i\in[\ell]}, z)$. □

Game$_3$: In Game$_3$, we add an additional abort condition. In particular, we return 0 if for any $j \in [B] \setminus \{i\}$, in ω_j^{cca} and $\omega_j^{\mathsf{range}}$, there exists a chunk index $h \in [\ell]$ such that $k_h^{\mathsf{cca}} \neq k_h^{\mathsf{range}}$.

Claim 13. *For all PPT adversaries $\mathcal{A}$, there exists a PPT adversary $\mathcal{B}$ such that*

$$\Pr[\mathsf{Game}_{2,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{3,\mathcal{A}}(1^\lambda) = 1] \leq \mathsf{Adv}^{\mathit{binding}}_{\mathcal{B},\mathsf{Ped}}(1^\lambda).$$

Proof of Claim 13. The claim follows by reduction to the computational binding property of the Pedersen commitment scheme Ped. Clearly, a PPT adversary $\mathcal{A}$ can only distinguish by triggering the additional abort condition. In this case, however, for some $j \in [B] \setminus \{i\}$ there exists a chunk index $h \in [\ell]$ such that, since the preceding games did not abort,

$$\mathsf{com}_h = \mathsf{Ped.Commit}(k_h^{\mathsf{cca}}, r_h^{\mathsf{com,cca}}) = \mathsf{Ped.Commit}(k_h^{\mathsf{range}}, r_h^{\mathsf{com,range}})$$

, but $k_h^{\mathsf{cca}} \neq k_h^{\mathsf{range}}$. The reduction submits the tuple

$$(\mathsf{com}_h, k_h^{\mathsf{cca}}, r_h^{\mathsf{com,cca}}, k_h^{\mathsf{range}}, r_h^{\mathsf{com,range}})$$

to the binding game and wins, whenever $\mathcal{A}$ hits the additional abort condition. □

Game$_4$: In Game$_4$, we use the extractor $\mathsf{Sim}_{\mathsf{PS}_{\mathsf{pd}}} = (\mathsf{Setup}, \mathsf{Ext})$ to extract a witness ω_u from all $(\mathsf{pd}_u, \pi_u^{\mathsf{pd}})$ for $u \in S \cap \mathsf{Cor}$. In particular, we embed the simulator setup $\mathsf{pp}_{\mathsf{pd}}$ as $(\mathsf{pp}_{\mathsf{pd}}, \mathsf{st}_{\mathsf{Sim}_{\mathsf{PS}_{\mathsf{pd}}}}) \leftarrow\!\$\ \mathsf{Sim}_{\mathsf{PS}_{\mathsf{pd}}}.\mathsf{Setup}(1^\lambda, \mathcal{G})$. Then, after receiving the adversary's partial decryptions, we extract ω_u^{pd} for all $u \in S \cap \mathsf{Cor}$ as

$$(\omega_u^{\mathsf{pd}}, \mathsf{st}_{\mathsf{Sim}_{\mathsf{PS}_{\mathsf{pd}}}}) \leftarrow\!\$\ \mathsf{Sim}_{\mathsf{PS}_{\mathsf{pd}}}.\mathsf{Ext}((\mathsf{pp}, \mathsf{pk}_u, \{\mathsf{ct}_j\}_{j\in[B]}, \mathsf{pd}_u), \pi_u^{\mathsf{pd}}, \mathsf{st}_{\mathsf{Sim}_{\mathsf{PS}_{\mathsf{pd}}}}).$$

We parse ω_u^{pd} as (sk_u). We add an additional abort condition and return 0 if $(\chi_u^{\mathsf{pd}}, \omega_u^{\mathsf{pd}}) \notin \mathsf{R}_{\mathsf{pd}}$ for some $u \in S \cap \mathsf{Cor}$, where $\chi_u^{\mathsf{pd}} = (\mathsf{pp}, \mathsf{pk}_u, \{\mathsf{ct}_j\}_{j\in[B]}, \mathsf{pd}_u)$. This checks both that the extracted secret key matches pk_u and that pd_u is the corresponding partial decryption of the batch.

Claim 14. *For all PPT adversaries $\mathcal{A}$ there exists a PPT reduction $\mathcal{B}$ such that*

$$\Pr[\mathsf{Game}_{3,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{4,\mathcal{A}}(1^\lambda) = 1] \leq 2 \cdot \mathsf{Adv}^{\mathsf{Game\text{-}Ext}}_{\mathcal{B},\mathsf{PS}_{\mathsf{pd}},\mathsf{Sim}_{\mathsf{PS}_{\mathsf{pd}}}}(1^\lambda).$$

Proof of Claim 14. The claim follows a straightforward reduction to knowledge-soundness of $\mathsf{PS}_{\mathsf{pd}}$. In particular, observe that if the additional abort condition is met, then $(\chi_u^{\mathsf{pd}}, \omega_u^{\mathsf{pd}}) \notin \mathsf{R}_{\mathsf{pd}}$ for some $u \in S \cap \mathsf{Cor}$, where $\chi_u^{\mathsf{pd}} = (\mathsf{pp}, \mathsf{pk}_u, \{\mathsf{ct}_j\}_{j\in[B]}, \mathsf{pd}_u)$. □

Because of (a) the perfect correctness and key homomorphism of the PRF, (b) the perfect correctness and additive homomorphism of Σ_{MSTE}, and (c) the perfect correctness and efficiency of the dlog extraction algorithm on aggregated chunks in $[0, B(2^z - 1)]$, it can only happen that $m_i \neq m_i'$ if one of the following bad events occurs:

(1) There exists $u' \in C$ for which no $\mathsf{sk}_{u'}$ satisfies $(\mathsf{pk}_{u'}, \mathsf{sk}_{u'}) \in \mathsf{KGen}(\mathsf{pp})$. This is impossible due to the perfect soundness of KVfy.
(2) $\exists j' \in [B] \setminus \{i\}$ such that $k_{j'}^*$ and $\mathsf{ct}_{\Sigma_{\mathsf{MSTE}}}^{(j')}$ do not belong to the same chunking $(k_1, \ldots, k_\ell)$ or there exists at least one k_i such that $k_i \notin [0, 2^z - 1]$.
(3) $\exists u \in S \cap \mathsf{Cor}$ such that pd_u is not a valid partial decryption of the batch of ciphertexts.

The challenge ciphertext is honestly generated, and all other ciphertexts that survive the preceding checks have valid chunks, so each aggregate chunk is at most $B(2^z - 1)$, within the extraction

range fixed by the construction. Since conditions 2 and 3 cause the adversary to trigger an abort condition in Game_4, it holds that

$$\Pr[\mathsf{Game}_{4,\mathcal{A}}(1^\lambda) = 1] = 0.$$

Summing the bounds of Claims 11–14, we can bound the advantage of any PPT adversary $\mathcal{A}$ to execute a successful rogue ciphertext attack as follows:

$$\begin{aligned}&\Pr[\mathsf{Game\text{-}Rogue}^{\Sigma_{\mathsf{SB}}}_{\mathcal{A}}(1^\lambda) = 1] \leq \\ &2 \cdot \mathsf{Adv}^{\mathsf{Game\text{-}Ext}}_{\mathcal{B}_1,\mathsf{PS}_{\mathsf{cca}},\mathsf{Sim}_{\mathsf{PS}_{\mathsf{cca}}}}(1^\lambda) + 2 \cdot \mathsf{Adv}^{\mathsf{Game\text{-}Ext}}_{\mathcal{B}_2,\mathsf{PS}_{\mathsf{range}},\mathsf{Sim}_{\mathsf{PS}_{\mathsf{range}}}}(1^\lambda) \\ &+ \mathsf{Adv}^{\mathsf{binding}}_{\mathcal{B}_3,\mathsf{Ped}}(1^\lambda) + 2 \cdot \mathsf{Adv}^{\mathsf{Game\text{-}Ext}}_{\mathcal{B}_4,\mathsf{PS}_{\mathsf{pd}},\mathsf{Sim}_{\mathsf{PS}_{\mathsf{pd}}}}(1^\lambda),\end{aligned}$$

for appropriate reductions $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$, and $\mathcal{B}_4$. □

E Size Comparison of Various Parameters

We compare our construction with other schemes that do not achieve silent setup (except [48]) with respect to the size of various parameters in Fig. 9.

	Size (Elements)				Size (KiB)			
	Σ_{SB}	[18]	[3, 23]	[48]	Σ_{SB}	[18]	[3, 23]	[48]
crs	$[\ell(N+2)+B_{\max}]\cdot\mathbb{G}_1$ $+\ell(N+3)\cdot\mathbb{G}_2+$ $(2B_{\max}-1)\cdot\mathbb{G}_2$	$B_{\max}\cdot\mathbb{G}_1+$ $2B_{\max}\cdot\mathbb{G}_2$ $-1\cdot\mathbb{G}_2$	$B_{\max}\cdot\mathbb{G}_1+$ $2\cdot\mathbb{G}_2$	$(2N(B_{\max}+1)+3)\cdot\mathbb{G}_1$ $+(2N(B_{\max}+1)+N)\cdot\mathbb{G}_2$ $+(B_{\max}+3)\cdot\mathbb{G}_2+1\cdot\mathbb{G}_T$	413.91	119.91	24.19	18, 528.98
pk	$\ell N\cdot\mathbb{G}_1$	$1\cdot\mathbb{G}_2$	$N\cdot\mathbb{G}_2$	$3\cdot\mathbb{G}_1+1\cdot\mathbb{G}_T$	96.00	0.09	6.00	0.70
ek	$6\cdot\mathbb{G}_1+5\cdot\mathbb{G}_2$	$1\cdot\mathbb{G}_1+1\cdot\mathbb{G}_2$	$1\cdot\mathbb{G}_2+2$ bytes	$2\cdot\mathbb{G}_1+3\cdot\mathbb{G}_2+1\cdot\mathbb{G}_T$	0.75	0.14	0.10	0.94
ct	$(4+\ell)\cdot\mathbb{G}_1+5\cdot\mathbb{G}_2$ $+\log B_{\max}$	$4\cdot\mathbb{G}_1+1\cdot\mathbb{G}_2$ +2 bytes	$1\cdot\mathbb{G}_1+3\cdot\mathbb{G}_2$ +2 bytes	$2\cdot\mathbb{G}_1+2\cdot\mathbb{G}_2+1\cdot\mathbb{G}_T$	1.41	0.28	0.33	0.84
π^{ct}	$(2\ell+6)\cdot\mathbb{F}+6\ell\cdot\mathbb{G}_1$	$3\cdot\mathbb{G}_1+2\cdot\mathbb{F}$	$4\cdot\mathbb{F}$	—	5.69	0.20	0.13	—
pd	$1\cdot\mathbb{G}_1$	$1\cdot\mathbb{G}_1$	$1\cdot\mathbb{G}_1$	$1\cdot\mathbb{F}+2\cdot\mathbb{G}_2$	0.05	0.05	0.05	0.22
π^{pd}	$2\cdot\mathbb{F}$	$2\cdot\mathbb{F}$	$2\cdot\mathbb{F}$	—	0.06	0.06	0.06	—

Figure 9: Sizes of various parameters and messages in our batched threshold encryption scheme Σ_{SB} with silent setup, and comparison between Σ_{SB}, [18], [3, 23], and [48] ([3, 18, 23] require DKG setup). Concrete numbers are reported for $\ell = 16, B_{\max} = 512$, and $N = 128$.